DOCKER TASK - 03

1.Create a Docker image from running container.

1. Objective

The objective is to create a new Docker image from an existing/running Docker container using the docker commit command.

The basic flow is:

Running Container → docker commit → New Docker Image

---

2. Check the Running Containers

First, check which containers are currently running.

docker ps

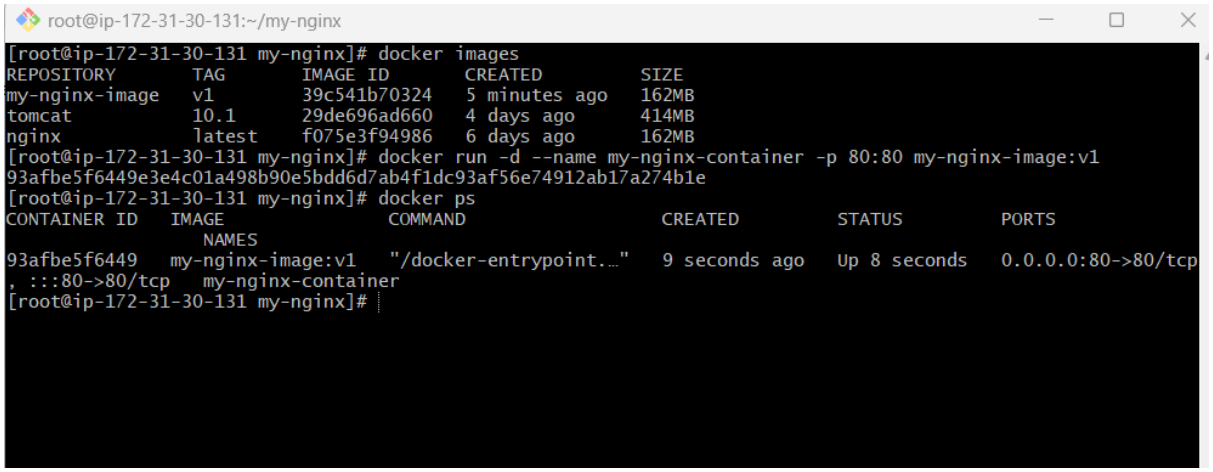
Example:

CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

cb9a128373f3 my-nginx-image "nginx -g 'daemon of…" 1 hour ago Up 1 hour 0.0.0.0:80->80/tcp my-nginx-container

Here:

- **Container ID:** cb9a128373f3
- **Container Name:** my-nginx-container
- **Current Image:** my-nginx-image



3. Make Changes Inside the Running Container

Enter the running container.

docker exec -it my-nginx-container /bin/bash

If Bash is not available, use:

docker exec -it my-nginx-container /bin/sh

For example, check the Nginx web files:

cd /usr/share/nginx/html

ls

You can create or modify a file:

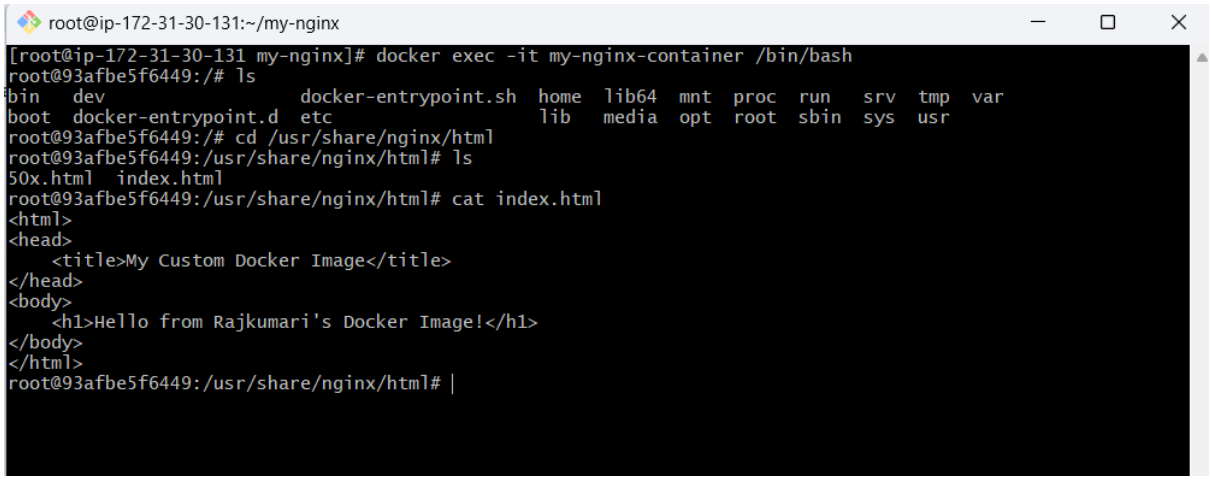
echo "This is my modified Nginx container" > /usr/share/nginx/html/index.html

Verify the change:

cat /usr/share/nginx/html/index.html

Exit the container:

Exit



4. Verify the Container

Check that the container is still running:

docker ps

You can also test the application from the Docker host:

curl http://localhost

You should see the modified HTML content.

Hello from Rajkumari's Docker Image!

5. Create an Image from the Running Container

Now use the docker commit command.

Syntax

docker commit <container-name-or-id> <new-image-name>:<tag>

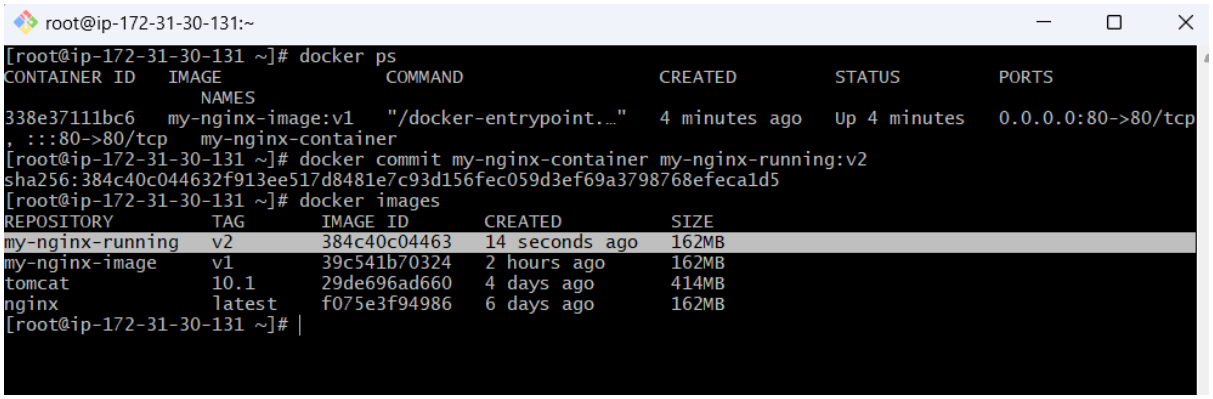
Example

docker commit my-nginx-container my-nginx-backup:v1

Output will look similar to:

sha256:xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

This means Docker successfully created a new image from the container.



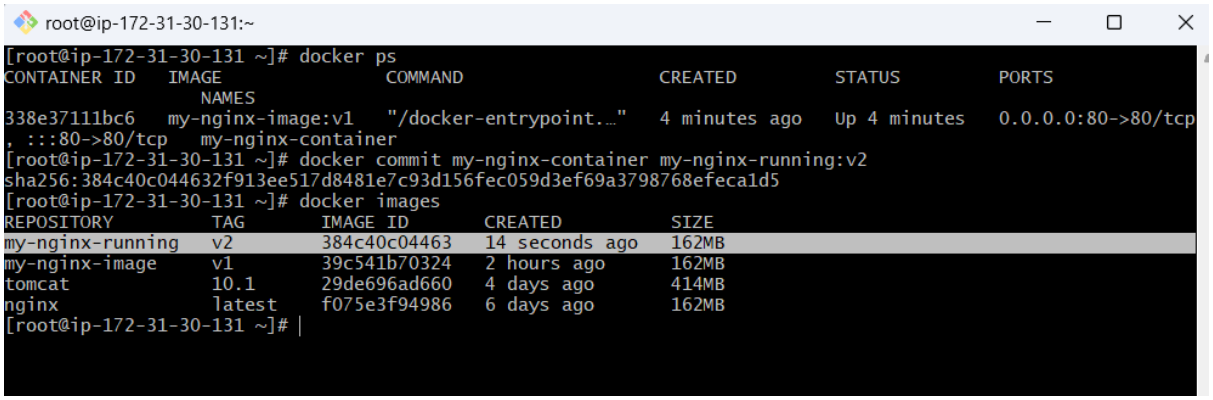
6. Verify the New Image

Run:

docker images

The newly created image is: docker commit <Container name or ID> <New Image name>: version

my-nginx-running:v2



7. Run a New Container from the Created Image

Create a new container using the new image:

docker run -d --name nginx-run-cont -p 8080:80 my-nginx-running:v2

Check the container:

docker ps

You should see:

CONTAINER ID IMAGE PORTS NAMES

xxxxxx my-nginx-running:v1 0.0.0.0:81->80/tcp nginx-backup-container

```
[root@ip-172-31-30-131 ~]# docker images
REPOSITORY          TAG       IMAGE ID       CREATED         SIZE
my-nginx-running    v2        384c40c04463   4 minutes ago   162MB
my-nginx-image      v1        39c541b70324   2 hours ago     162MB
tomcat              10.1      29de696ad660   4 days ago      414MB
nginx               latest    f075e3f94986   6 days ago      162MB
[root@ip-172-31-30-131 ~]# docker run -d --name my-nginx-run-cont -p 81:80 my-nginx-running:v2
da4916be1c8685c2dd70b4c91e034ccc0655c66e4e7cd658b52c1ce2e097d538
[root@ip-172-31-30-131 ~]#
```

8. Test the New Container

Use:

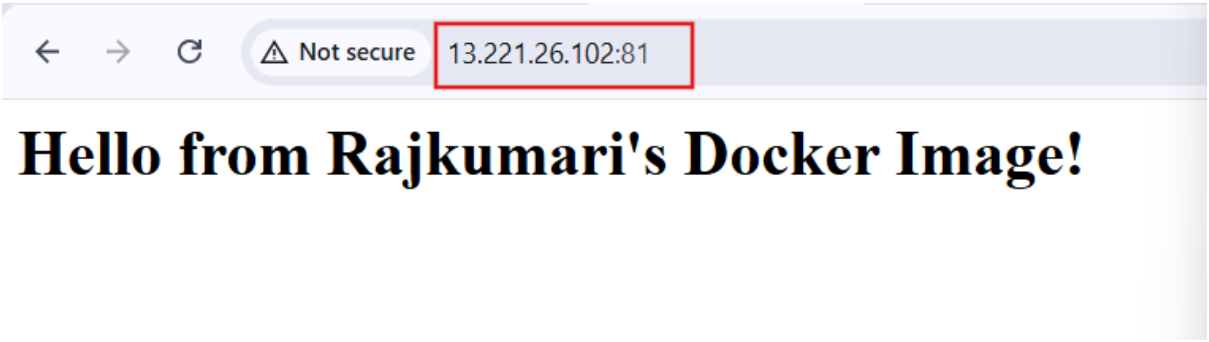
curl http://localhost:8080

You should receive the HTML content that was present in the original container when the image was committed.

You can also access it from a browser:

http://<EC2-Public-IP>:81

Make sure port **81** is allowed in the EC2 Security Group if accessing it from your computer.



9. Inspect the Created Image

You can inspect the image using:

docker inspect my-nginx-backup:v1

This displays information such as:

- Image ID
- Created time
- Architecture
- Environment variables

- Entrypoint
- Command
- Root filesystem
- Container configuration

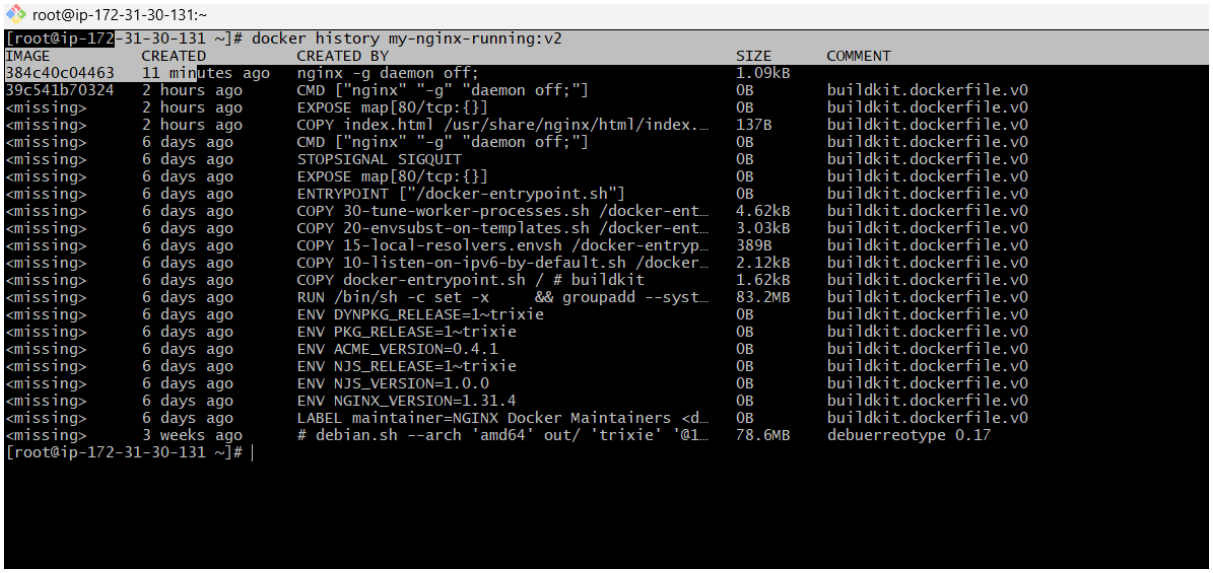
---

10. View the Image History

Use:

docker history my-nginx-running:v2

This shows the layers and history associated with the image.



2.Copy Docker image from local machine to docker server and load the image.

Local Machine

|

| docker save

↓

Docker Image (.tar file)

|

| scp

↓

Remote Docker Server

|

| docker load

↓

Docker Image available on server

2. Check the Image on the Local Machine

First, list the available Docker images:

docker images

Example:

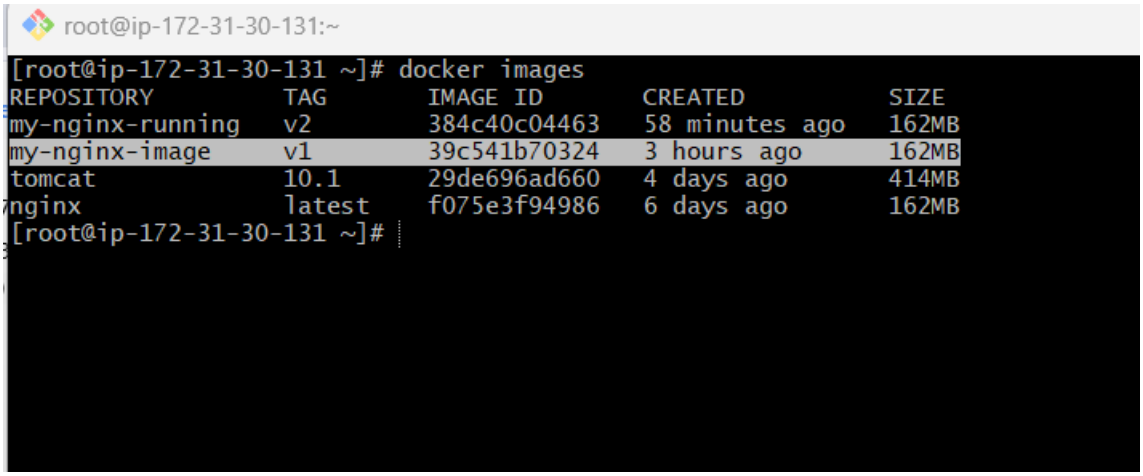
REPOSITORY TAG IMAGE ID CREATED SIZE

my-nginx-image v1 abc123456789 10 minutes ago 162MB

nginx latest def987654321 2 hours ago 162MB

Assume the image we want to transfer is:

my-nginx-image:v1



3. Save the Docker Image as a TAR File

Use the docker save command:

docker save -o my-nginx-image.tar my-nginx-image:v1

Explanation

- docker save → Saves a Docker image into a file.
- -o → Specifies the output file.
- my-nginx-image.tar → Name of the image archive.
- my-nginx-image:v1 → Docker image that we want to save.

4. Verify the TAR File

Check whether the file was created:

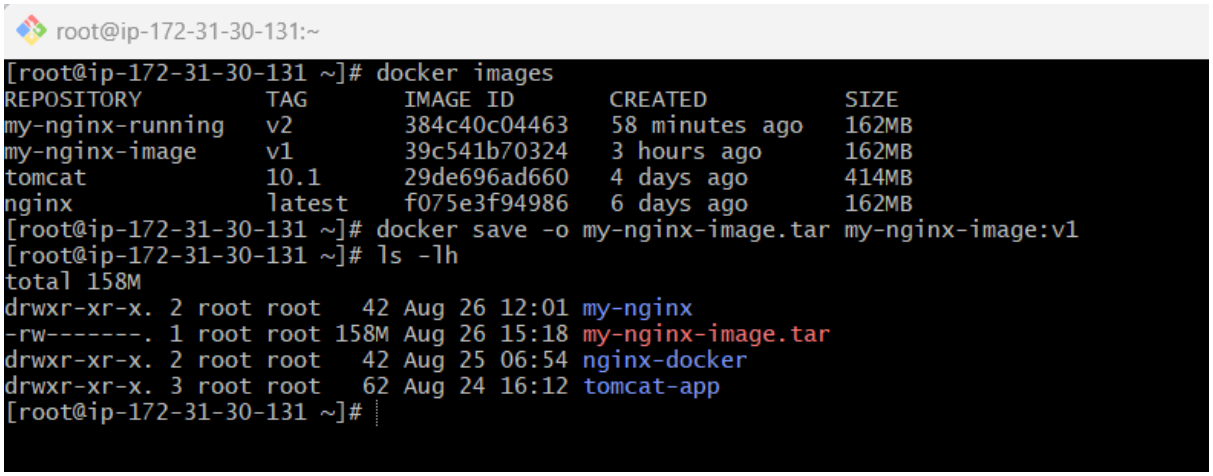
ls -lh my-nginx-image.tar

Example:

-rw-r--r-- 1 user user 162M Aug 26 20:00 my-nginx-image.tar

You can also verify the file type:

file my-nginx-image.tar



5. Copy the TAR File from Local machine to the Docker Server

If the Docker server is an AWS EC2 instance, use scp.

Syntax

scp -i <key.pem> <image.tar> <username>@<server-ip>:<destination>

scp -i server-1.pem my-nginx-image.tar ec2-user@13.221.26.102:/home/ec2-user/

Example

For an Amazon Linux EC2 server:

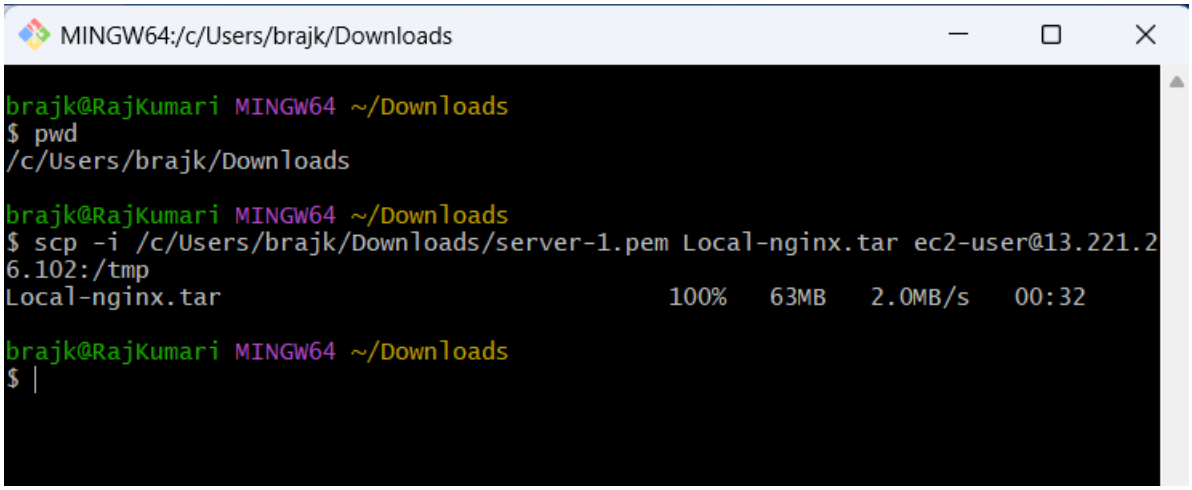
scp -i mykey.pem my-nginx-image.tar ec2-user@<EC2-PUBLIC-IP>:/home/ec2-user/

scp -i /c/Users/brajk/Downloads/server-1.pem Local-nginx.tar ec2-user@13.221.26.102:/tmp

6. SSH into the Docker Server

Connect to the remote Docker server:

ssh -i mykey.pem ec2-user@<EC2-PUBLIC-IP>

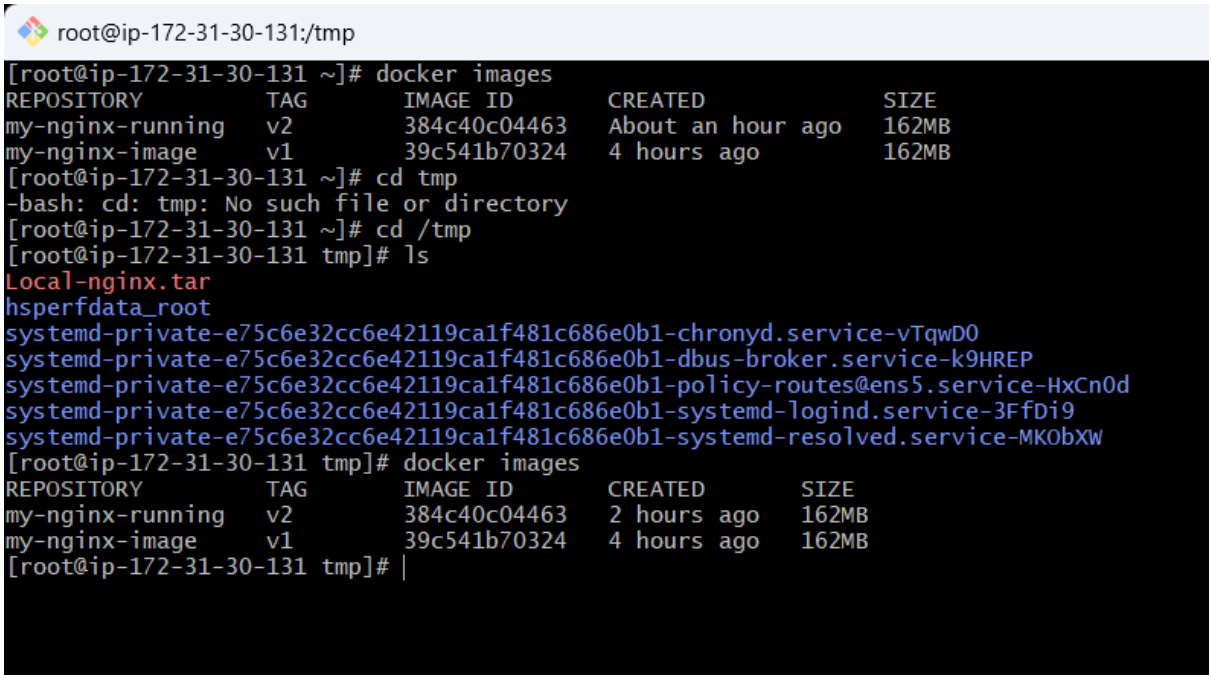


7. Verify the TAR File on the Docker Server

After logging into the server:

ls -lh

cd /tmp



8. Load the Image into Docker

Now load the image using:

docker load -i Local-nginx.tar

You should see output similar to:

Loaded image: Local-nginx:v1

This means the image was successfully loaded into Docker.

9. Verify the Image

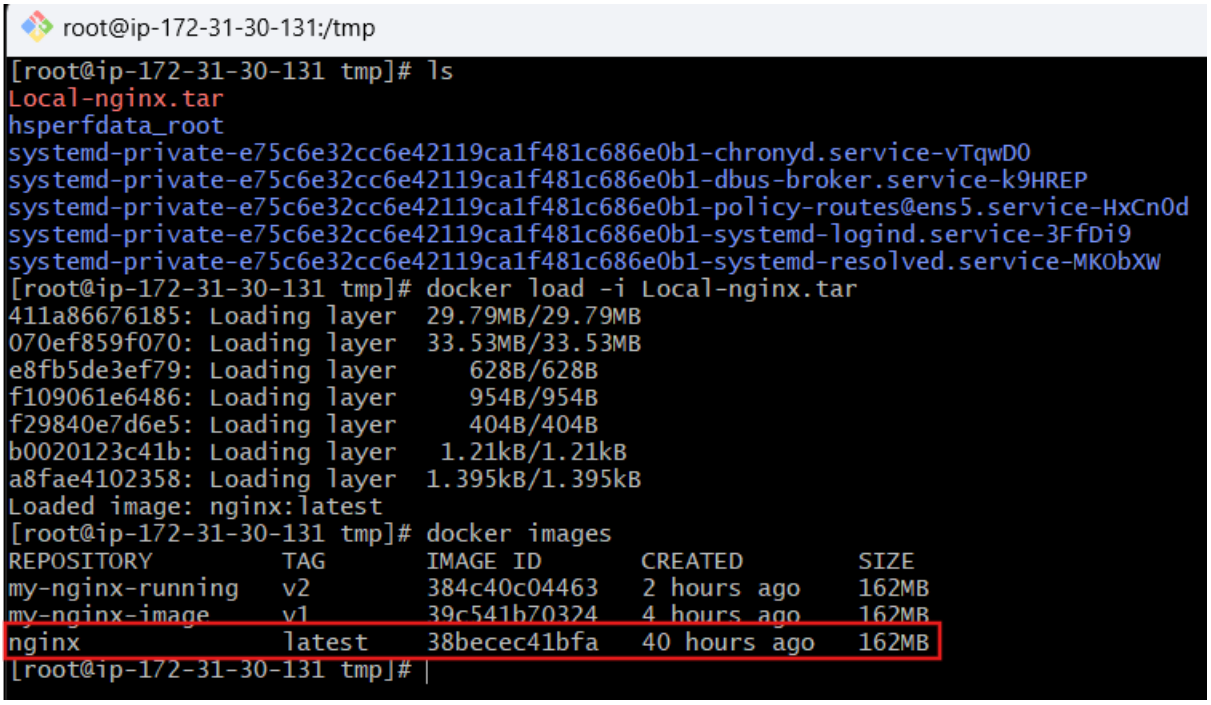
Run:

docker images

You should see:

REPOSITORY TAG IMAGE ID CREATED SIZE

my-nginx-image v1 abc123456789 10 minute



10. Run a Container from the Loaded Image

Now create a container:

docker run -d --name my-nginx-container -p 80:80 my-nginx-image:v1

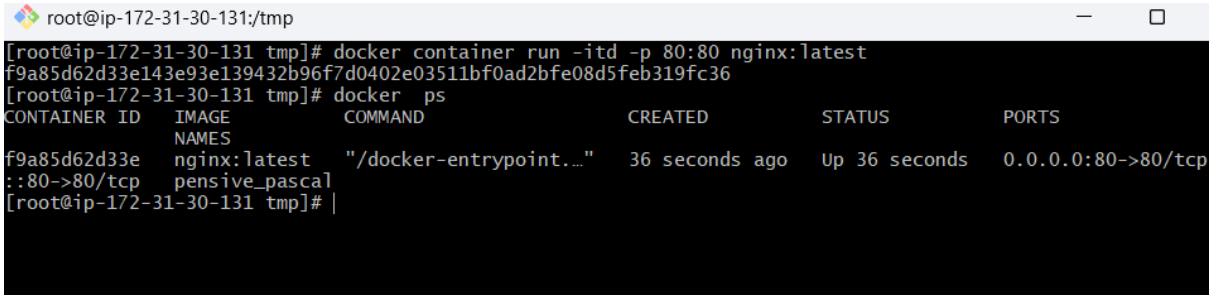
Check the running container:

docker ps

Example:

CONTAINER ID IMAGE PORTS NAMES

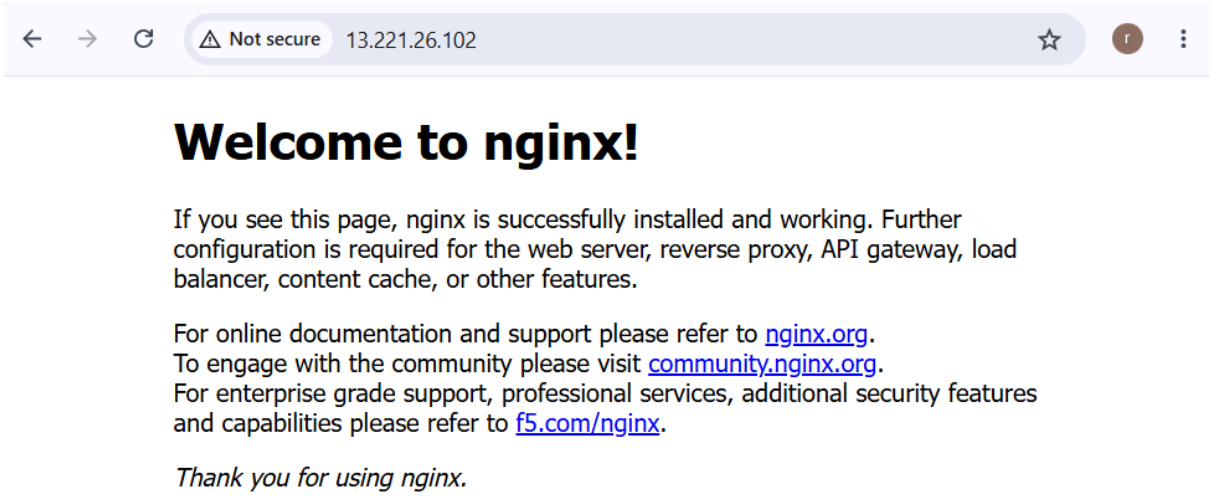
xxxxxxxxxxxx my-nginx-image:v1 0.0.0.0:80->80/tcp



11. Test the Application

From the Docker server:

curl http://localhost



3. Create Docker image using alpine and customize with tomcat.

1. Objective

Create a custom Docker image based on **Alpine Linux**, install **Java**, download and configure **Apache Tomcat**, expose Tomcat port 8080, and start Tomcat when the container launches.

The architecture is:

Alpine Linux

↓

Install Java

↓

Download Tomcat

↓

Configure Tomcat

↓

Expose Port 8080

↓

Start Tomcat

↓

Custom Tomcat Docker Image

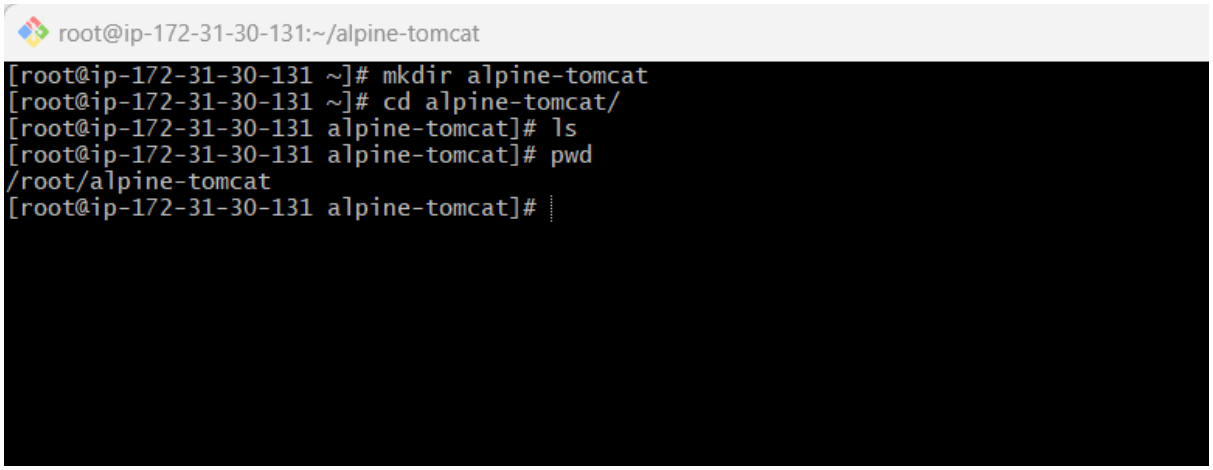
2. Create a Directory

Create a working directory on the Docker server:

mkdir alpine-tomcat

cd alpine-tomcat

Verify: pwd



3. Create the Dockerfile

Create a file named Dockerfile:

vi Dockerfile

Add the following content:

FROM alpine:latest

LABEL maintainer="Rajkumari"

RUN apk update && \

apk add --no-cache openjdk17-jre wget tar bash

ENV CATALINA_HOME=/opt/tomcat

ENV PATH=$CATALINA_HOME/bin:$PATH

RUN wget -q https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.49/bin/apache-tomcat-10.1.49.tar.gz && \

mkdir -p /opt && \

tar -xzf apache-tomcat-10.1.49.tar.gz -C /opt && \

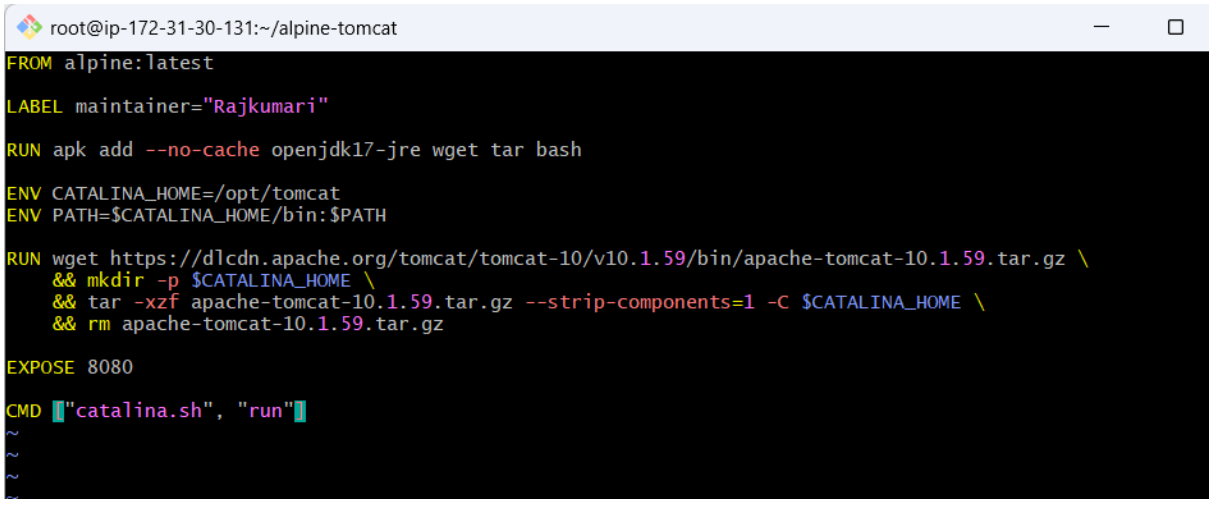
mv /opt/apache-tomcat-10.1.49 /opt/tomcat && \

rm apache-tomcat-10.1.49.tar.gz

EXPOSE 8080

CMD ["catalina.sh", "run"]

Note: Tomcat versions change over time. If that exact version is no longer available, replace the download URL and directory name with the current Tomcat 10.1 version.



4. Understand the Dockerfile

FROM

FROM alpine:latest

This uses the latest Alpine Linux image as the base image.

Alpine is a very small Linux distribution, which helps keep the Docker image relatively lightweight.

LABEL

LABEL maintainer="Rajkumari"

Adds metadata to the Docker image.

You can replace Rajkumari with your name.

Install Java and Required Packages

```
RUN apk update && \
    apk add --no-cache openjdk17-jre wget tar bash
```

Here:

- apk → Alpine Linux package manager
- openjdk17-jre → Java Runtime Environment
- wget → Downloads Tomcat
- tar → Extracts the Tomcat archive
- bash → Provides Bash shell support

Tomcat requires Java to run.

---

5. Set Tomcat Environment Variables

```
ENV CATALINA_HOME=/opt/tomcat
ENV PATH=$CATALINA_HOME/bin:$PATH
```

CATALINA_HOME defines the location where Tomcat is installed.

The PATH modification allows commands such as:

catalina.sh

to be executed directly.

---

6. Download and Install Tomcat

RUN wget -q https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.49/bin/apache-tomcat-10.1.49.tar.gz && \

mkdir -p /opt && \

tar -xzf apache-tomcat-10.1.49.tar.gz -C /opt && \

mv /opt/apache-tomcat-10.1.49 /opt/tomcat && \

rm apache-tomcat-10.1.49.tar.gz

This performs several operations:

1. Downloads Tomcat.
2. Creates /opt.
3. Extracts the Tomcat archive.
4. Renames the extracted directory to /opt/tomcat.
5. Removes the downloaded .tar.gz file.

After installation, Tomcat will be located at:

/opt/tomcat

---

7. Expose Tomcat Port

EXPOSE 8080

This documents that the application inside the container listens on port 8080.

It does **not** publish the port to the host by itself.

The port is published when using docker run -p.

8. Start Tomcat

CMD ["catalina.sh", "run"]

This starts Tomcat in the foreground.

Keeping Tomcat in the foreground is important because Docker expects the main application process to remain running.

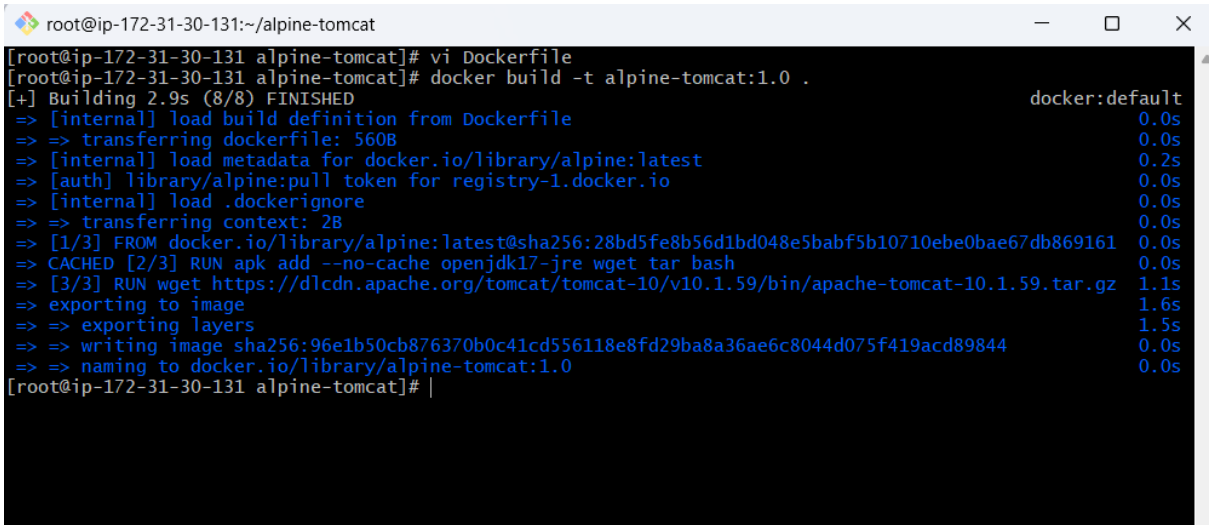
. Build the Docker Image

From the directory containing the Dockerfile:

docker build -t alpine-tomcat:1.0 .

Explanation:

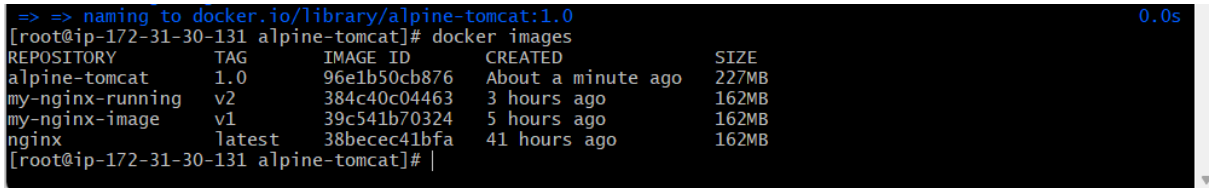
- docker build → Builds an image.
- -t alpine-tomcat:1.0 → Gives the image a name and tag.
- . → Uses the current directory as the build context.



10. Verify the Image

Run:

docker images



11. Run the Tomcat Container

Run:

```
docker run -d \
  --name alpine-tomcat-container \
  -p 8080:8080 \
  alpine-tomcat:1.0
```

Explanation:

-p 8080:8080

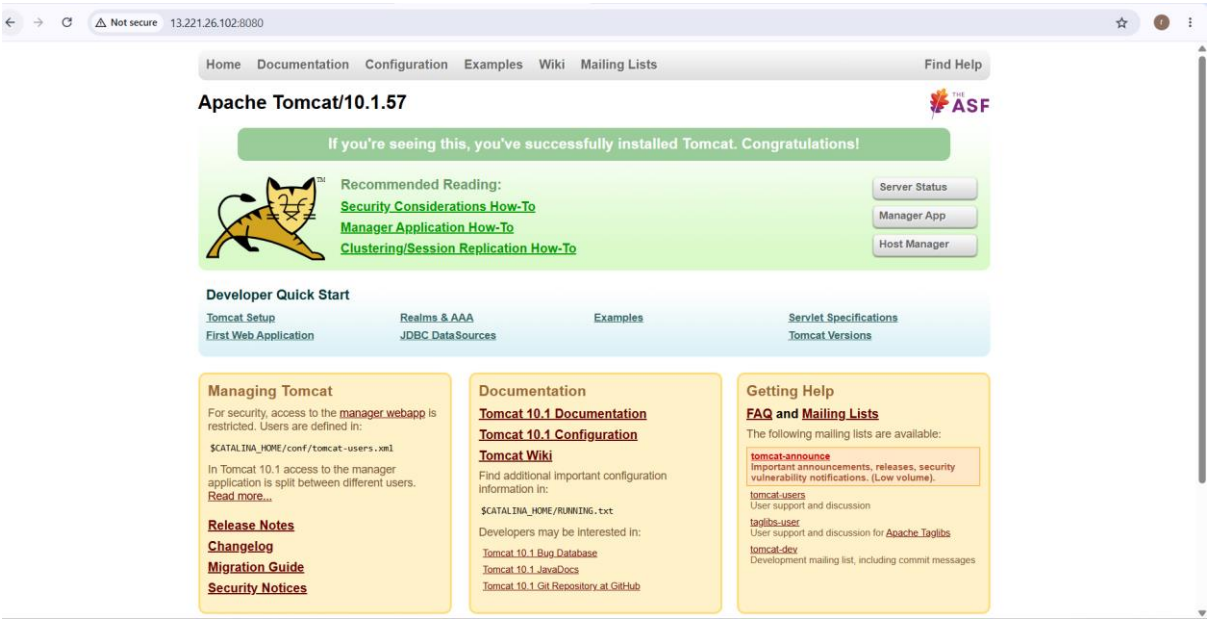
means:

Host Port 8080 → Container Port 8080

12. Test Tomcat

From the Docker server:

curl http://localhost:8080



4.Create single stage and multi stage docker file using this

source code: https://github.com/betawins/multi-stage-example.git

Single-stage Dockerfile — Maven build and runtime are in the same image.

Multi-stage Dockerfile — Maven/JDK are used only for building; the final image contains only what is required to run the application. This is the main benefit of multi-stage builds.

1. Source Code

GitHub repository:

betawins/multi-stage-example

The repository contains:

.mvn/

src/

mvnw

mvnw.cmd

pom.xml

Dockerfile

README.md

The application is a Spring Boot application and uses Maven for building.

Part 1: Clone the Source Code

2. Clone the Repository

On your Docker server:

git clone https://github.com/betawins/multi-stage-example.git

Go inside the project:

cd multi-stage-example

Check the files:

ls -la

You should see:

.mvn

src

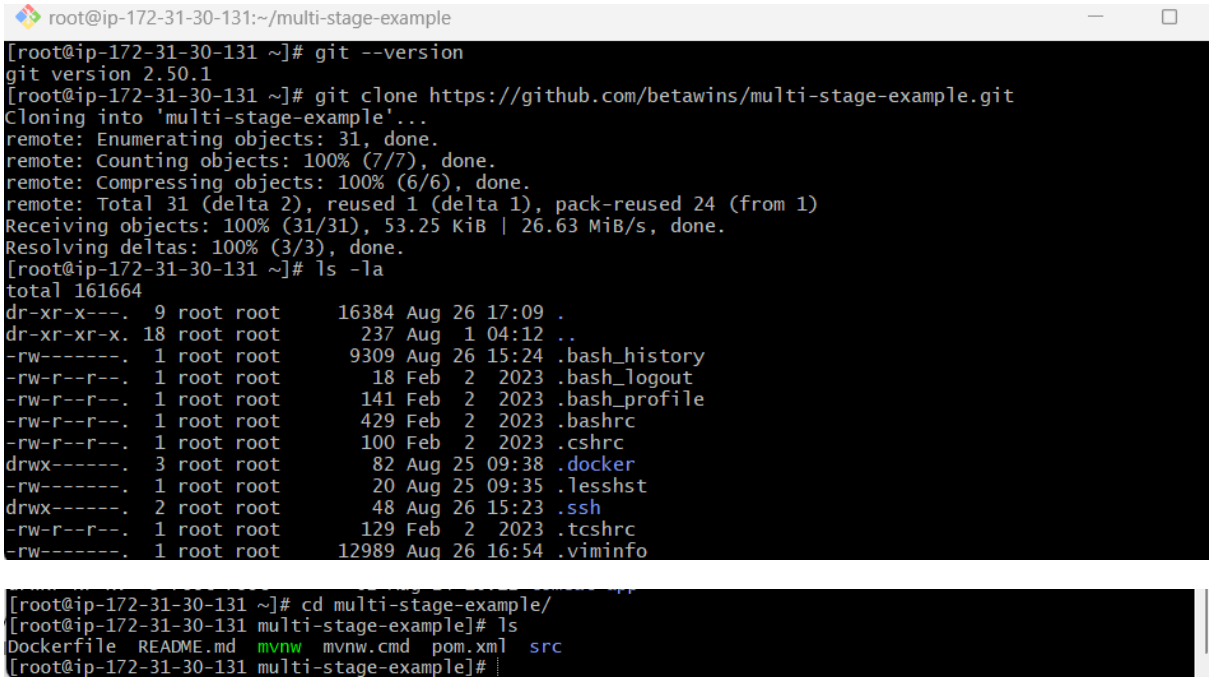
mvnw

mvnw.cmd

pom.xml

Dockerfile

README.md



Part 2: Single-Stage Dockerfile

3. What is a Single-Stage Build?

In a single-stage Dockerfile, the same image contains:

Maven

JDK

Source Code

Dependencies

Build Tools

Application JAR

Runtime

Therefore, the final image can become larger because build-time tools and files remain in the image.

---

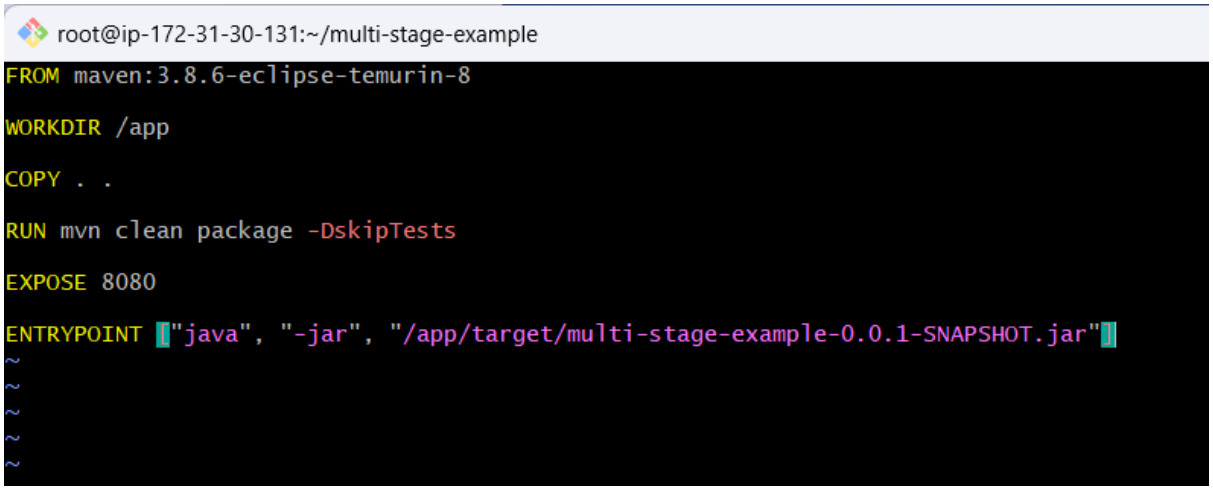
4. Create Single-Stage Dockerfile

Create:

vi Dockerfile.single

Add:

```
FROM maven:3.8.6-eclipse-temurin-8

WORKDIR /app

COPY . .

RUN ./mvnw clean package -DskipTests

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar”]
```



5. Explanation of Single-Stage Dockerfile

FROM

FROM maven:3.8.6-eclipse-temurin-8

This image provides:

- Maven
- JDK 8
- Java build environment

The application in the repository uses Spring Boot 2.2.6.RELEASE and Java 8 in its Maven configuration.

WORKDIR

WORKDIR /app

Creates /app as the working directory.

All subsequent commands execute from this directory.

COPY

COPY . .

Copies the complete project into /app.

For example:

/app

├── src

├── pom.xml

├── mvnw

└─── .mvn

Build the Application

RUN ./mvnw clean package -DskipTests

This executes Maven and creates the JAR file under:

/app/target/

The -DskipTests option skips test execution.

If you want to run tests during the image build, use:

RUN ./mvnw clean package

EXPOSE

EXPOSE 8080

The Spring Boot application listens on port 8080.

ENTRYPOINT

ENTRYPOINT ["java", "-jar", "/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar"]

This starts the Spring Boot application when the container starts.

6. Build the Single-Stage Image

Run:

docker build -f Dockerfile.single -t multi-stage-single:v1 .

Explanation:

-f Dockerfile.single

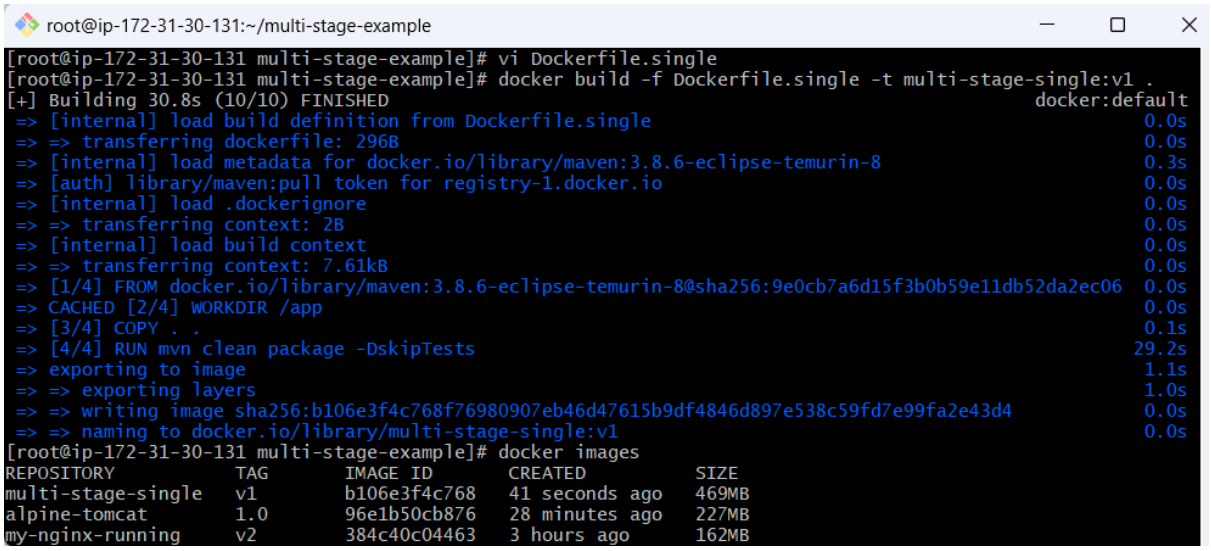
specifies the Dockerfile to use.

-t multi-stage-single:v1

assigns the image name and tag.

.

specifies the current directory as the build context.



7. Verify the Single-Stage Image

docker images

You should see:

REPOSITORY TAG IMAGE ID SIZE

multi-stage-single v1 xxxxxxxxxxx ...

---

8. Run the Single-Stage Container

```
docker run -d \
  --name multi-stage-single-container \
  -p 8080:8080 \
  multi-stage-single:v1
```

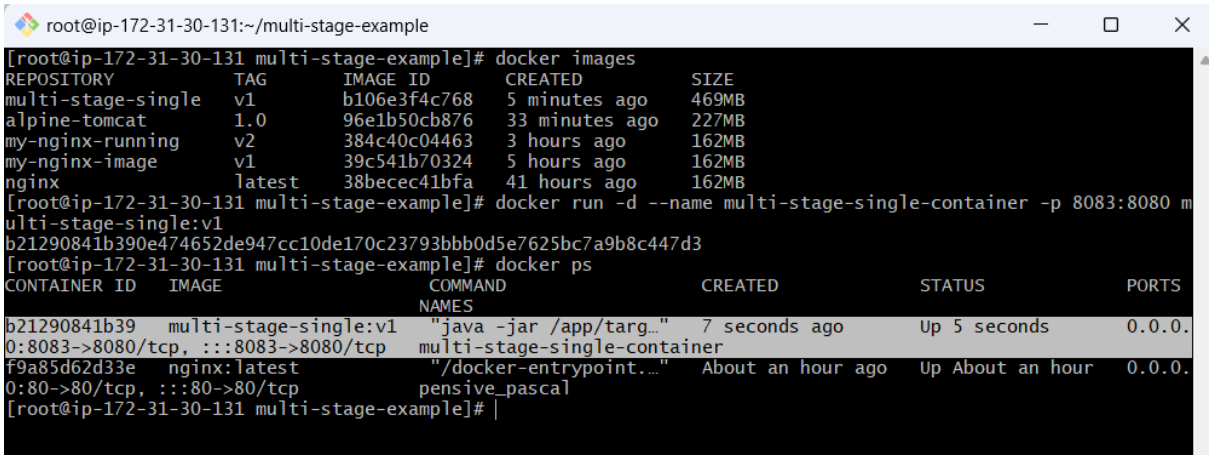
9. Verify the Container

docker ps

Expected:

CONTAINER ID IMAGE PORTS NAMES

xxxxxxxxxxxx multi-stage-single:v1 0.0.0.0:8080->808



10. Check Application Logs

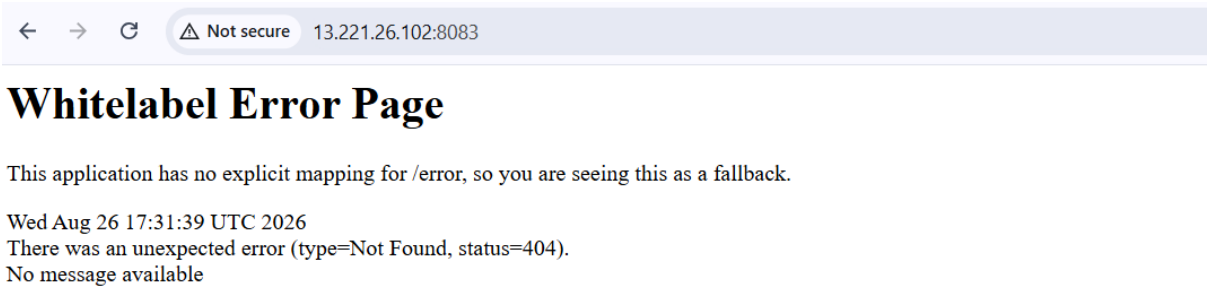
docker logs multi-stage-single-container

You should see Spring Boot startup messages.

11. Test the Application

From the Docker server:

curl http://localhost:8080



Part 3: Multi-Stage Dockerfile

12. What is a Multi-Stage Build?

A multi-stage Dockerfile uses multiple FROM instructions.

The first stage is responsible for building the application:

Source Code

↓

Maven

↓

JDK

↓

JAR

The second stage is responsible only for running the application:

JRE

+

JAR

↓

Final Image

Docker supports copying only the required build artifact from one stage to another using:

COPY --from=builder

This keeps build tools out of the final image.

14. Understand the Multi-Stage Dockerfile

Stage 1 — Builder

FROM maven:3.8.6-eclipse-temurin-8 AS builder

This stage contains:

Maven

JDK

Source Code

Dependencies

Build Tools

The name:

builder

allows us to refer to this stage later.

Copy Source Code

WORKDIR /app

COPY . .

The application source code is copied into:

/app

Build the Application

RUN ./mvnw clean package -DskipTests

This creates:

/app/target/multi-stage-example-0.0.1-SNAPSHOT.jar

---

15. Stage 2 — Runtime

FROM eclipse-temurin:8-jre

A JRE-only image is used for runtime instead of the Maven/JDK build environment.

The final image therefore doesn't need Maven or the source code.

---

16. Copy Only the JAR

This is the most important line:

COPY --from=builder /app/target/multi-stage-example-0.0.1-SNAPSHOT.jar app.jar

It means:

builder stage

|

| JAR only

↓

final stage

The Maven installation, source code, and build-time files are not copied into the final image.

This is the key advantage of multi-stage builds.

---

17. Expose Application Port

EXPOSE 8080

The Spring Boot application listens on port 8080.

---

18. Start the Application

ENTRYPOINT ["java", "-jar", "app.jar"]

This starts the Spring Boot application.

19. Build the Multi-Stage Image

Run:

docker build -f Dockerfile.multi -t multi-stage-multi:v1 .

Docker will execute:

Stage 1

↓

Maven Build

↓

JAR Created

↓

Stage 2

↓

Copy JAR

↓

Final Runtime Image

20. Verify the Image

docker images

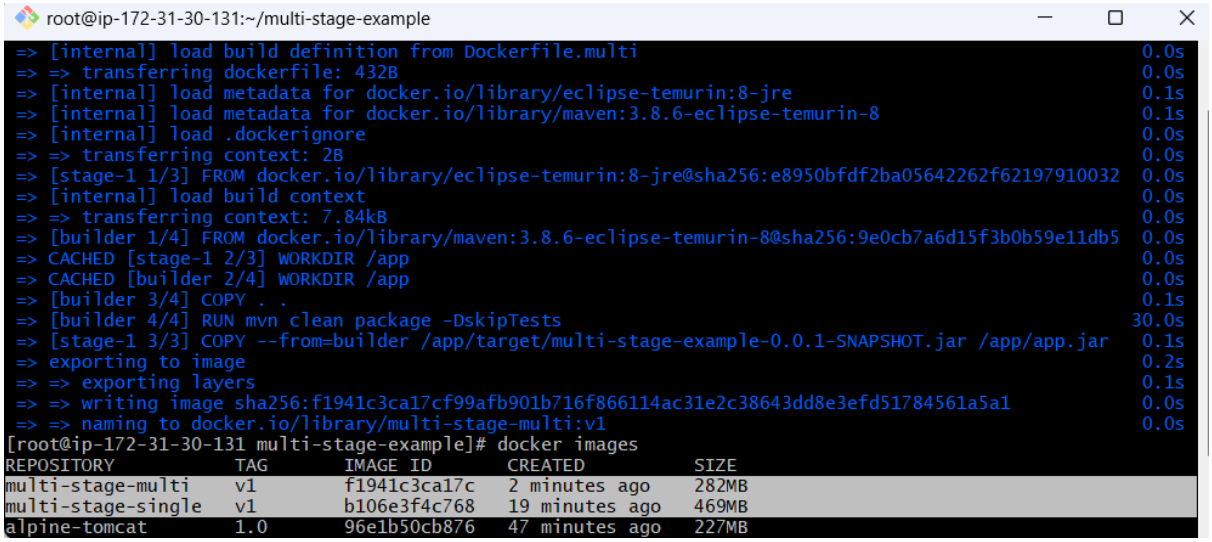
You should see:

REPOSITORY TAG IMAGE ID SIZE

multi-stage-multi v1 xxxxxxxxxxxx ...

multi-stage-single v1 xxxxxxxxxxxx ...

The multi-stage image should generally be smaller because build-time dependencies aren't included in the final image.



21. Run the Multi-Stage Container

docker run -d \

--name multi-stage-multi-container \

-p 8081:8080 \

multi-stage-multi:v1

If the single-stage container is already using port 8080, stop and remove it first:

docker stop multi-stage-single-container

docker rm multi-stage-single-container

Then run the multi-stage container.

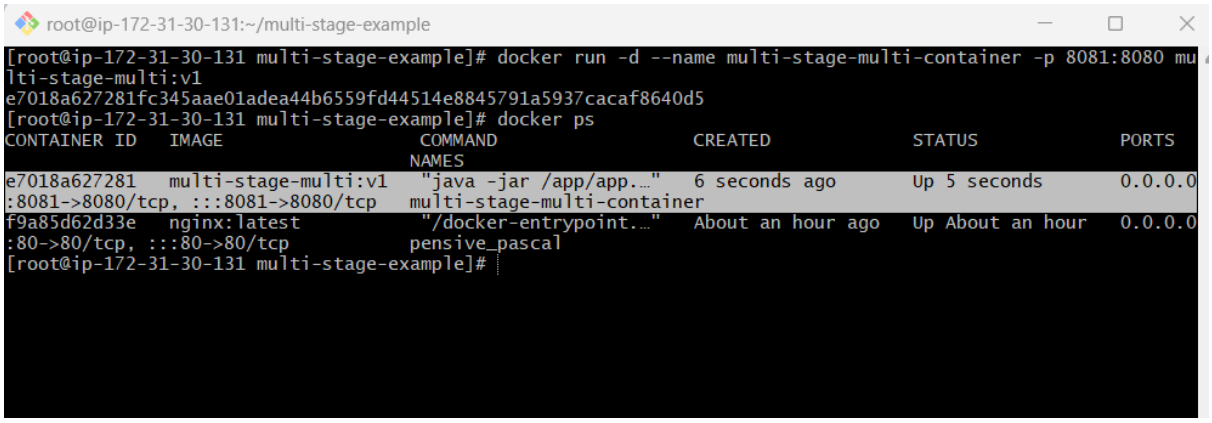
22. Verify the Container

docker ps

Example:

CONTAINER ID IMAGE PORTS NAMES

xxxxxxxxxxxx multi-stage-multi:v1 0.0.0.0:8080->8080/tcp multi-stage-multi-container



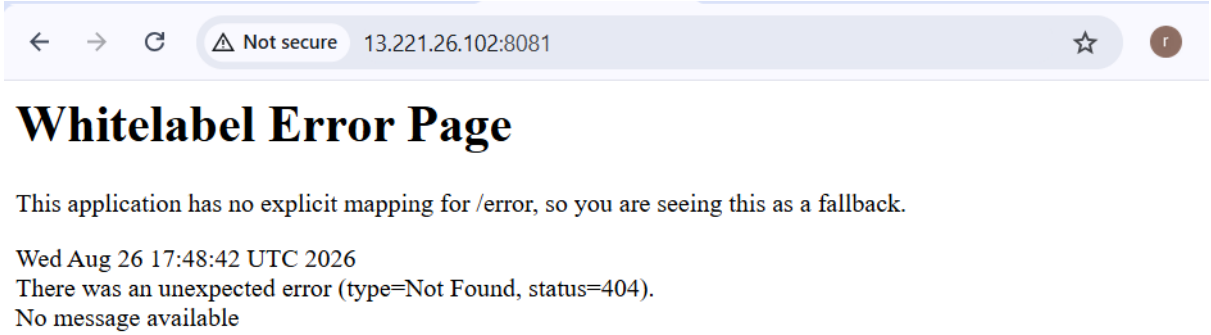
23. Check Logs

docker logs multi-stage-multi-container

---

24. Test the Application

curl http://localhost:8080



5. Install docker compose and execute sample application.

1. Objective

The objective is to:

1. Install Docker Compose on the Docker server.
2. Verify the Docker Compose installation.
3. Create a sample multi-container application.
4. Create a compose.yaml file.
5. Build and start the application using Docker Compose.

6. Verify the running containers.
7. Check application logs.
8. Access the application.
9. Stop and remove the application.

Docker Compose allows us to define multiple services in a YAML file and manage the complete application stack using commands such as docker compose up, docker compose ps, and docker compose down.

Part 1: Verify Docker Installation

Before installing Compose, verify that Docker is already installed.

docker --version

Example:

Docker version 28.x.x

Check that Docker is running:

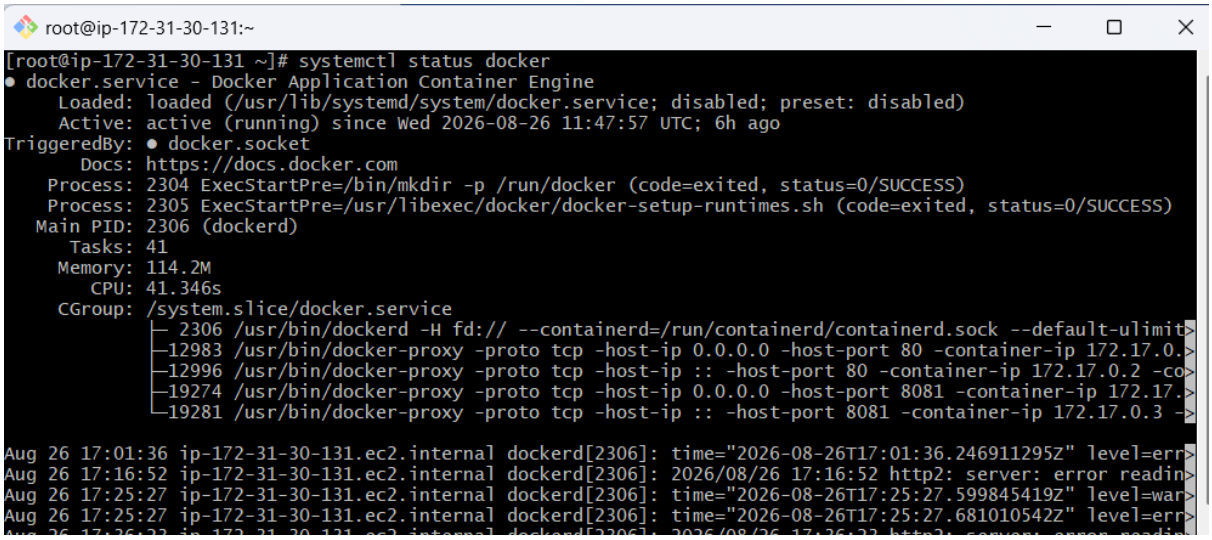
sudo systemctl status docker

If Docker is not running:

sudo systemctl start docker

Enable Docker at boot:

sudo systemctl enable docker



Part 2: Install Docker Compose

2. Install Docker Compose Plugin

For an RPM-based Linux system such as Amazon Linux/RHEL/CentOS, Docker's repository-based installation uses:

sudo yum update

sudo yum install docker-compose-plugin

Docker's official documentation recommends installing the Compose plugin through the Docker repository when possible because it is easier to maintain and update.

3. Verify Docker Compose

Run:

docker compose version

Example:

Docker Compose version v5.x.x

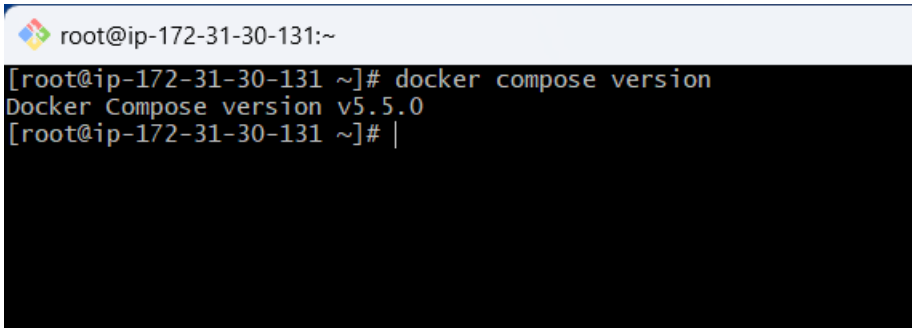
The important point is that the command is:

docker compose

not:

docker-compose

The latter refers to the older standalone/legacy installation.



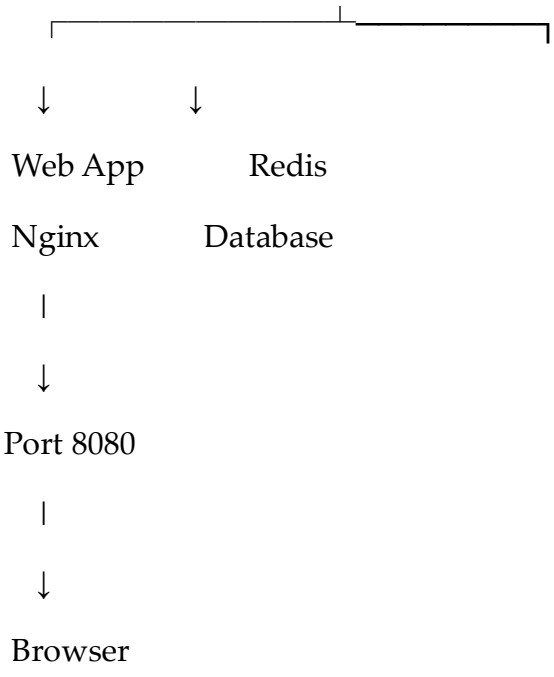
Part 3: Create a Sample Application

For this demonstration, we will create a simple **Nginx + Redis** application.

The architecture will be:

Docker Compose

|



4. Create a Project Directory

mkdir compose-demo

cd compose-demo

Verify:

pwd

5. Create an HTML Application

Create an HTML file:

vi index.html

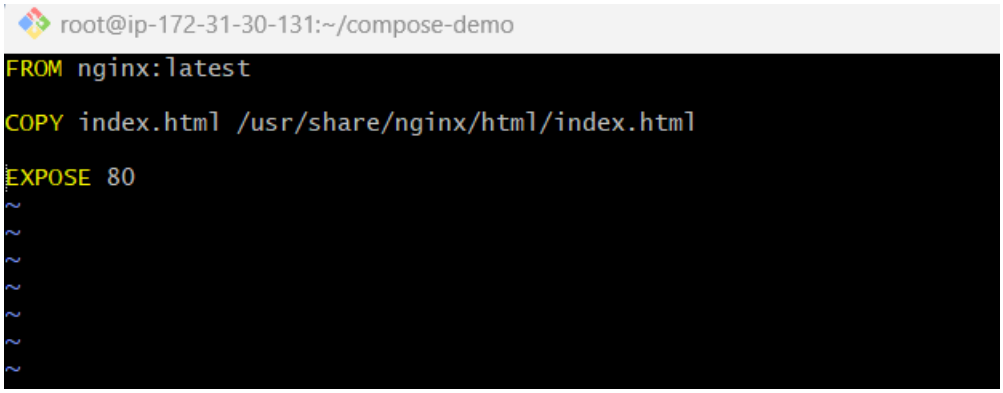
root@ip-172-31-30-131:~/compose-demo

```
<!DOCTYPE html>
<html>
<head>
        <title>Docker Compose Demo</title>
</head>
<body>
        <h1>Hello Rajkumari from Docker Compose!</h1>
        <p>This application is running using Docker Compose.</p>
</body>
</html>
```

6. Create a Dockerfile

Create:

vi Dockerfile



This creates a custom Nginx image containing our HTML application.

7. Create the Compose File

Create:

vi compose.yaml

8. Understand the Compose File

services

services:

Defines the containers/services that belong to our application.

Web Service

web:

This defines our web application service.

Build

build: .

Docker Compose builds the image using the Dockerfile in the current directory.

Container Name

container_name: compose-web

Assigns a specific name to the container.

Port Mapping

ports:

- "8080:80"

This means:

Host Port 8080

↓

Container Port 80

Therefore, Nginx listens internally on port 80, while we access it from the host using port 8080.

---

9. Redis Service

redis:

image: redis:alpine

Instead of building an image, Compose pulls the Redis Alpine image from the registry if it isn't already available locally.

Docker's Compose examples similarly demonstrate defining one application service with build and another service using an existing image.

Part 4: Validate the Compose File

Before starting the application, validate the configuration:

docker compose config

If the configuration is valid, Docker Compose displays the resolved configuration.

This is useful for detecting YAML/configuration problems before starting the application.

Part 5: Build the Application

10. Build the Images

Run:

docker compose build

Docker Compose will:

compose.yaml

|

↓

Dockerfile

|

↓

Build Nginx Image

|

↓

Redis Image

```
root@ip-172-31-30-131:~
[root@ip-172-31-30-131 ~]# cat /usr/local/lib/docker/cli-plugins/docker-buildx
Not Found[root@ip-172-31-30rm -f /usr/local/lib/docker/cli-plugins/docker-buildxer-buildx
[root@ip-172-31-30-131 ~]# curl -L https://github.com/docker/buildx/releases/download/v0.17.1/buildx-v0.1
linux-amd64 -o /usr/local/lib/docker/cli-plugins/docker-buildx
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--     0
100 55772k 100 55772k    0     0 171.8M      0  --:--:-- --:--:-- --:--:-- 303.8M
[root@ip-172-31-30-131 ~]# ls -lh /usr/local/lib/docker/cli-plugins/docker-buildx
-rw-r--r--. 1 root root 55M Aug 26 18:19 /usr/local/lib/docker/cli-plugins/docker-buildx
[root@ip-172-31-30-131 ~]# chmod +x /usr/local/lib/docker/cli-plugins/docker-buildx
[root@ip-172-31-30-131 ~]# docker buildx version
github.com/docker/buildx v0.17.1 257815a6fbaee88976808020bf04274388275ae8
[root@ip-172-31-30-131 ~]#
```

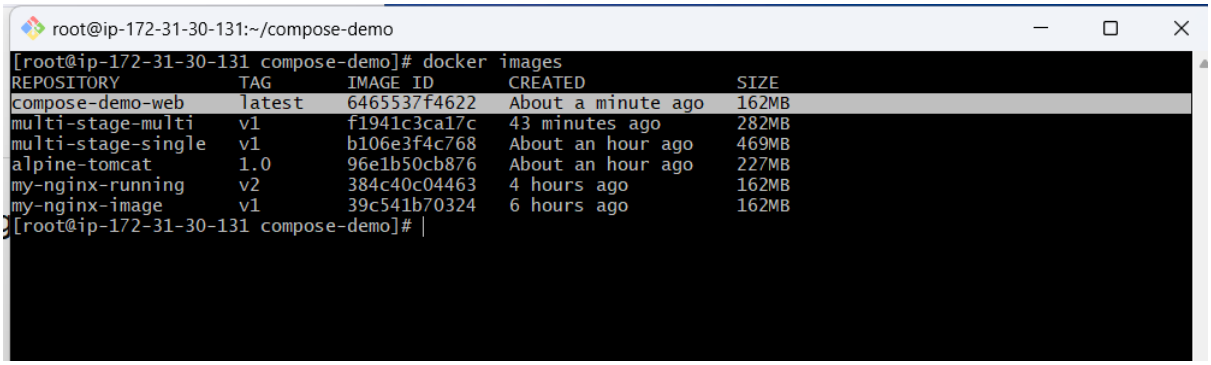
```
root@ip-172-31-30-131:~/compose-demo
[root@ip-172-31-30-131 ~]# docker compose build
no configuration file provided: not found
[root@ip-172-31-30-131 ~]# ll
total 161592
drwxr-xr-x. 2 root root        24 Aug 26 16:54 alpine-tomcat
drwxr-xr-x. 2 root root        62 Aug 26 18:15 compose-demo
drwxr-xr-x. 5 root root       186 Aug 26 17:39 multi-stage-example
drwxr-xr-x. 2 root root        42 Aug 26 12:01 my-nginx
-rw-------. 1 root root 165470208 Aug 26 15:18 my-nginx-image.tar
drwxr-xr-x. 2 root root        42 Aug 25 06:54 nginx-docker
drwxr-xr-x. 3 root root        62 Aug 24 16:12 tomcat-app
[root@ip-172-31-30-131 ~]# cd compose-demo/
[root@ip-172-31-30-131 compose-demo]# docker compose build
[+] Building 1.3s (10/10) FINISHED
 => [internal] load local bake definitions                                                0.0s
 => => reading from stdin 486B                                                            0.0s
 => [internal] load build definition from Dockerfile                                      0.1s
 => => transferring dockerfile: 178B                                                      0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                           0.5s
 => [auth] library/nginx:pull token for registry-1.docker.io                              0.0s
 => [internal] load .dockerignore                                                         0.0s
 => => transferring context: 2B                                                           0.0s
 => [internal] load build context                                                         0.0s
 => => transferring context: 309B                                                         0.0s
 => [1/2] FROM docker.io/library/nginx:latest@sha256:b34848eff6db786b6b1282d3a9c3fd0b5563dfb6d261df492  0.2s
 => => resolve docker.io/library/nginx:latest@sha256:b34848eff6db786b6b1282d3a9c3fd0b5563dfb6d261df492  0.0s
 => => sha256:38becec41bfad1ab0ac77eb1626c569fea6a8f39e59436880f61bdeed477226e 9.09kB / 9.09kB        0.0s
 => => sha256:b34848eff6db786b6b1282d3a9c3fd0b5563dfb6d261df4923378b419e0d24f0 10.23kB / 10.23kB      0.0s
 => => sha256:90c24d7f26b915f11b1f8e621ead4706b7e933de986498120310a876bf30dc1f 2.29kB / 2.29kB        0.0s
 => [2/2] COPY index.html /usr/share/nginx/html/index.html                                0.1s
 => exporting to image                                                                    0.0s
 => => exporting layers                                                                   0.0s
 => => writing image sha256:6465537f46220ba279063683fedfbd38fdd30d74e84c845d5cb6b49155f3ac70  0.0s
 => => naming to docker.io/library/compose-demo-web                                       0.0s
 => resolving provenance for metadata file                                                0.0s
[+] build 1/1
 ✔ Image compose-demo-web Built                                                           1.4s
[root@ip-172-31-30-131 compose-demo]#
```

11. Check Docker Images

Run:

docker images

You should see the Nginx-based image and Redis image.



Part 6: Start the Application

12. Start the Services

Run:

docker compose up -d

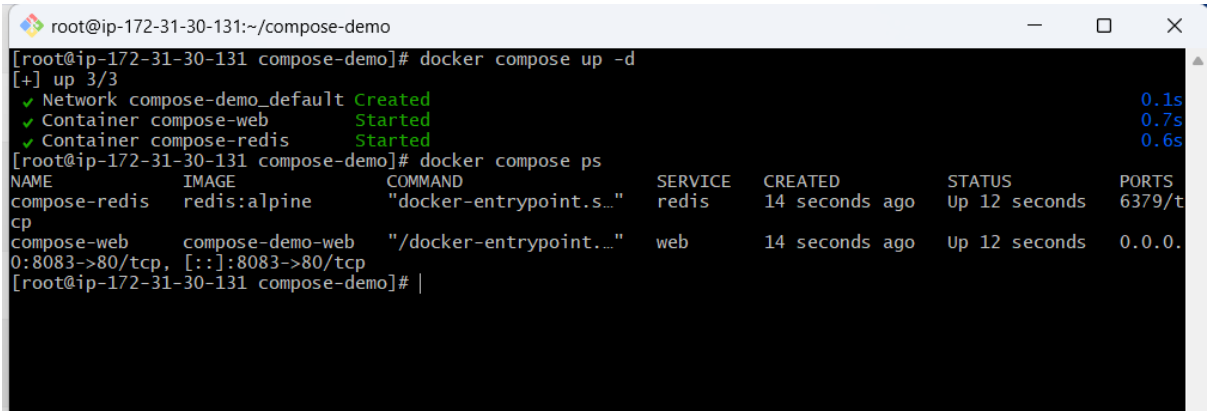
The -d option runs the containers in detached/background mode.

Docker's documentation uses docker compose up -d to start Compose applications in the background.

13. Check Running Services

Run:

docker compose ps



14. Check the Application Logs

To see logs from all services:

docker compose logs

To continuously follow the logs:

docker compose logs -f

Docker Compose provides logs, ps, up, and down as core commands for managing the application lifecycle.

Press:

Ctrl + C

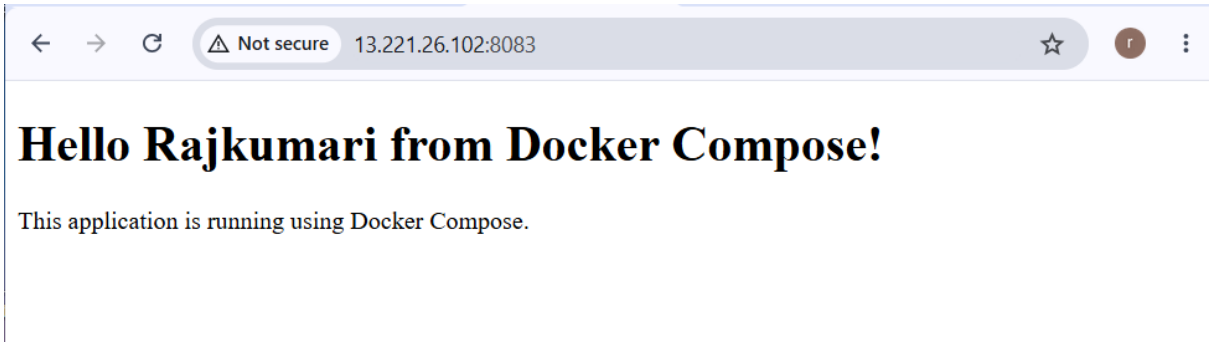
to stop following the logs.

Part 7: Test the Application

15. Test from the Docker Server

Run:

curl http://localhost:8080



Part 8: Verify Redis

17. Check Redis Container

Run:

docker compose ps

Confirm that:

compose-redis

is running.

You can also execute a Redis command inside the container:

docker exec -it compose-redis redis-cli ping

Expected:

PONG

This confirms that Redis is running successfully.

---

18. Check the Compose Network

Run:

docker network ls

You should see a network created for the Compose project.

Compose automatically creates a network for the application stack unless you configure a different networking setup.

Part 9: Useful Docker Compose Commands

Start Application

docker compose up -d

Stop Application

docker compose stop

Start Again

docker compose start

View Containers

docker compose ps

View Logs

docker compose logs

Follow Logs

docker compose logs -f

Build Images

docker compose build

Rebuild and Start

docker compose up -d --build

Execute Command Inside a Service

docker compose exec web bash

Stop and Remove Containers

docker compose down

docker compose down stops and removes the application's containers and other Compose-managed resources according to the configuration.

Part 10: Rebuild the Application

If you modify index.html, rebuild and recreate the application:

docker compose up -d --build

This is useful when you make changes to the Dockerfile or application files.

Part 11: Stop and Remove the Sample Application

When the assignment is complete:

docker compose down

Verify:

docker compose ps

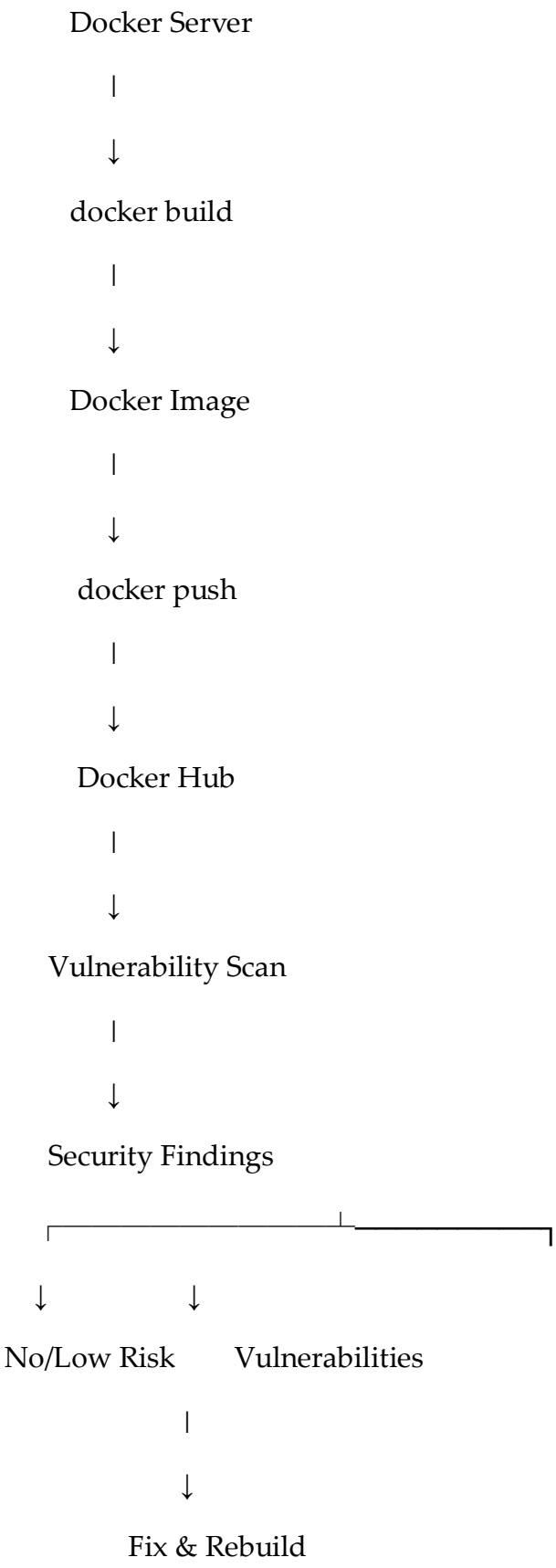
The services should no longer be running.

---

6. Implement solution to scan images when pushed to docker hub registry.

1. Objective

The objective is to configure a Docker Hub repository so that Docker images are scanned for vulnerabilities after they are pushed.



2. Prerequisites

You need:

- Docker installed
- Docker Hub account
- Docker Hub repository
- Docker server/EC2 instance
- Docker Hub username
- Permission to push to the repository

Check Docker:

docker --version

Example:

Docker version 28.x.x

Check Docker login:

docker login

Enter:

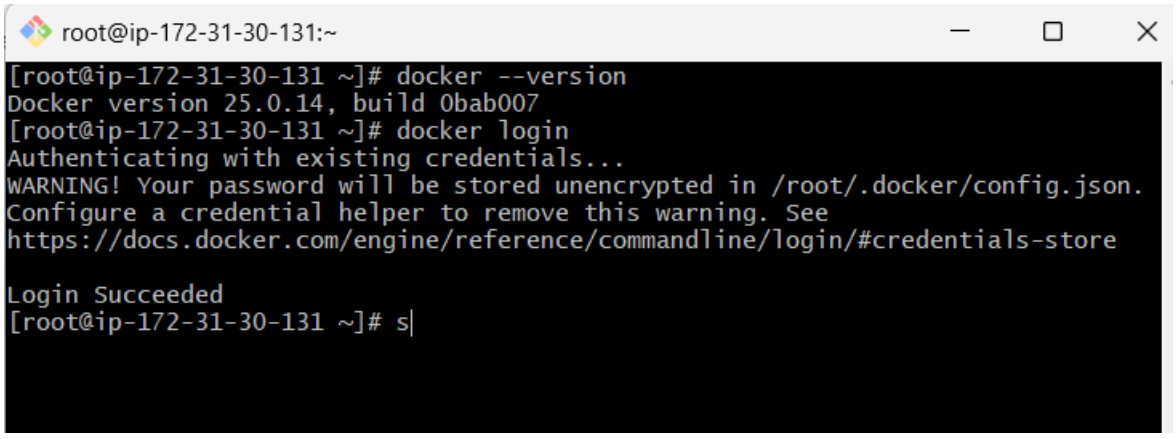
Username: <your-dockerhub-username>

Password: <your-password-or-access-token>

Expected:

Login Succeeded

For Docker Hub, using a **Personal Access Token (PAT)** instead of your Docker Hub password is the preferred authentication method.



3. Create a Docker Hub Repository

Go to Docker Hub:

Docker Hub

After logging in:

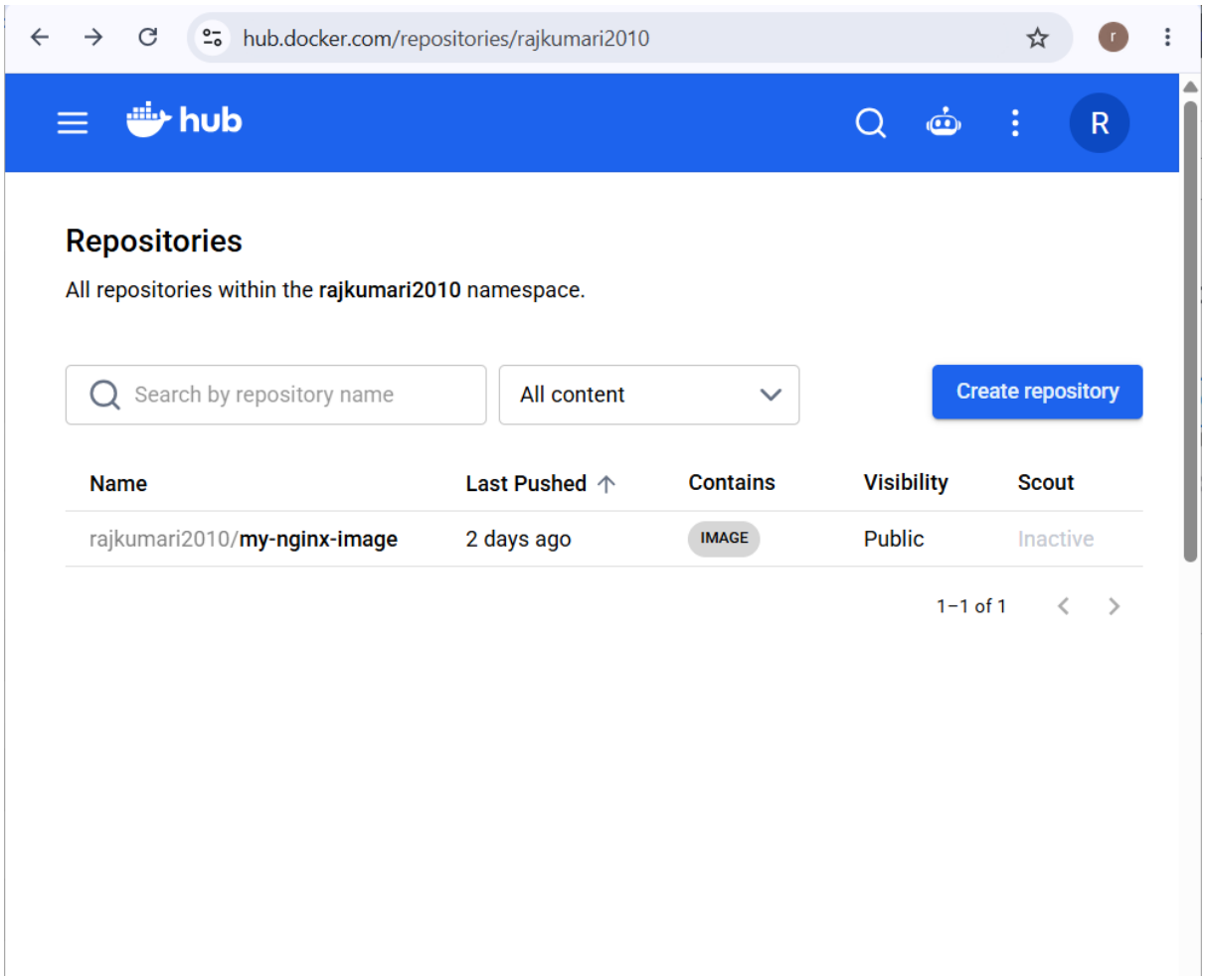
Docker Hub

↓

Repositories

↓

Create repository



Enter a repository name, for example:

docker-image-scan-demo

Choose the visibility:

Public

or:

Private

Then click:

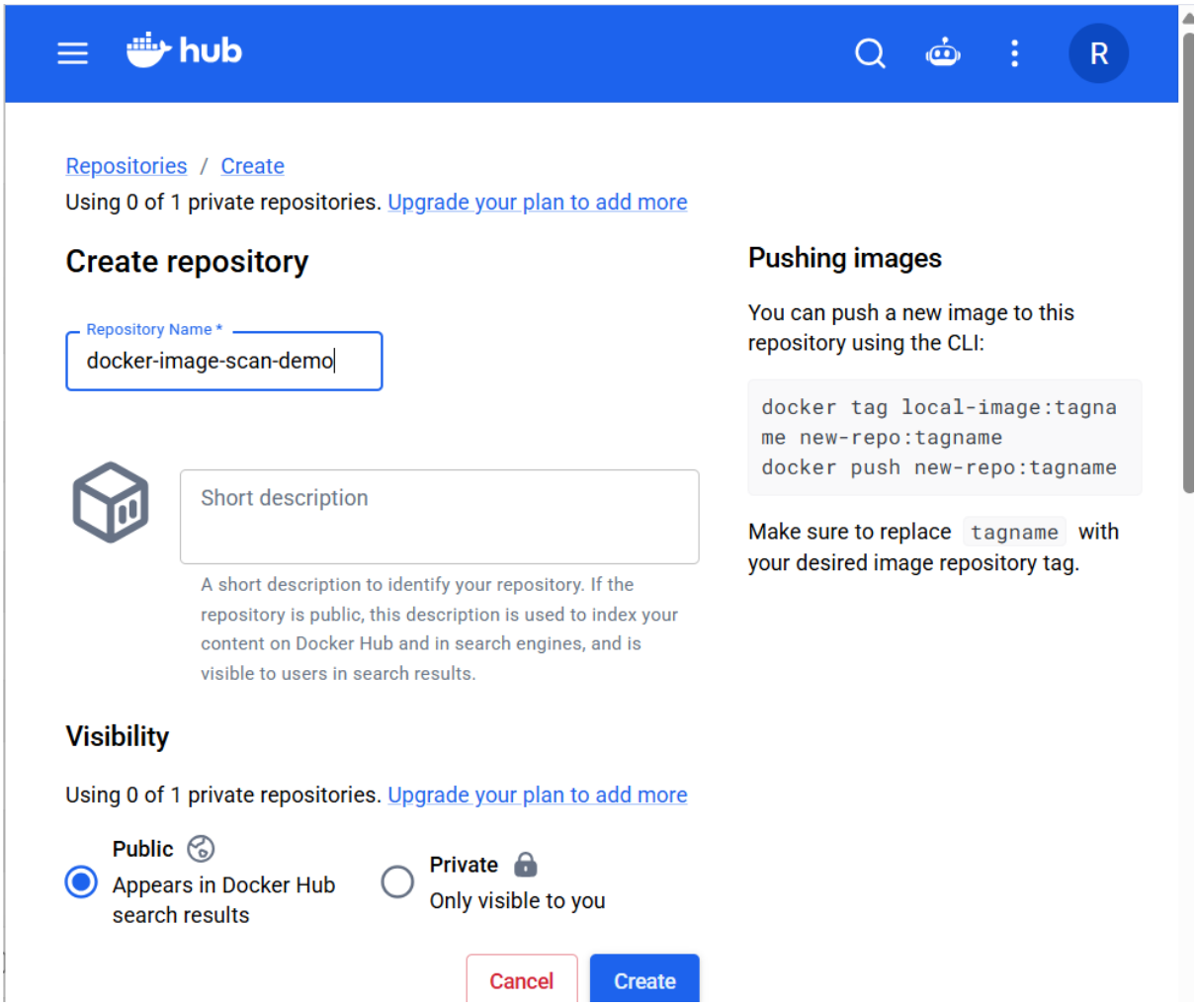
Create

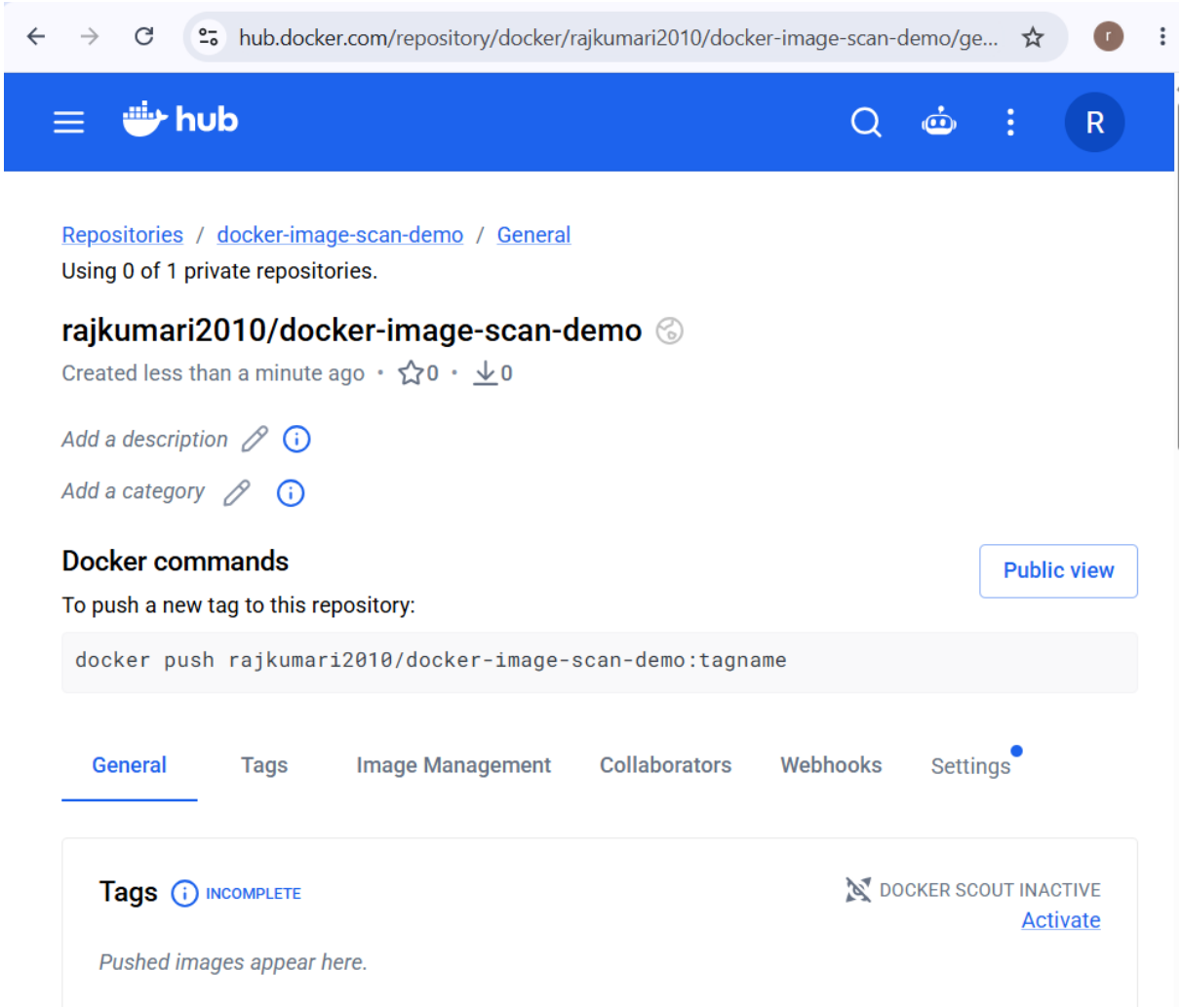
Your repository will look like:

<dockerhub-username>/docker-image-scan-demo

For example:

rajkumari/docker-image-scan-demo





4. Create a Sample Docker Image

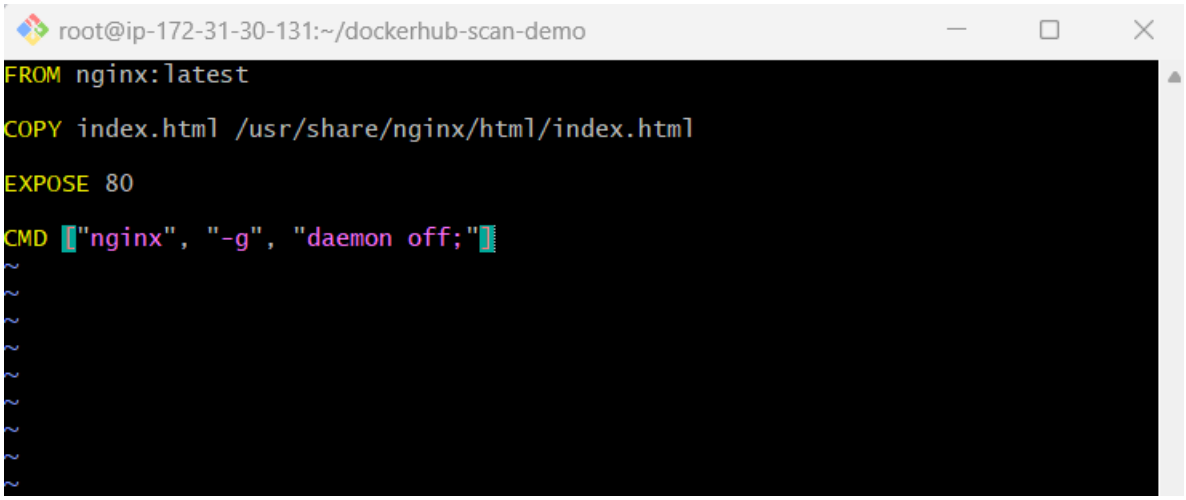
On your Docker server:

mkdir dockerhub-scan-demo

cd dockerhub-scan-demo

Create a Dockerfile:

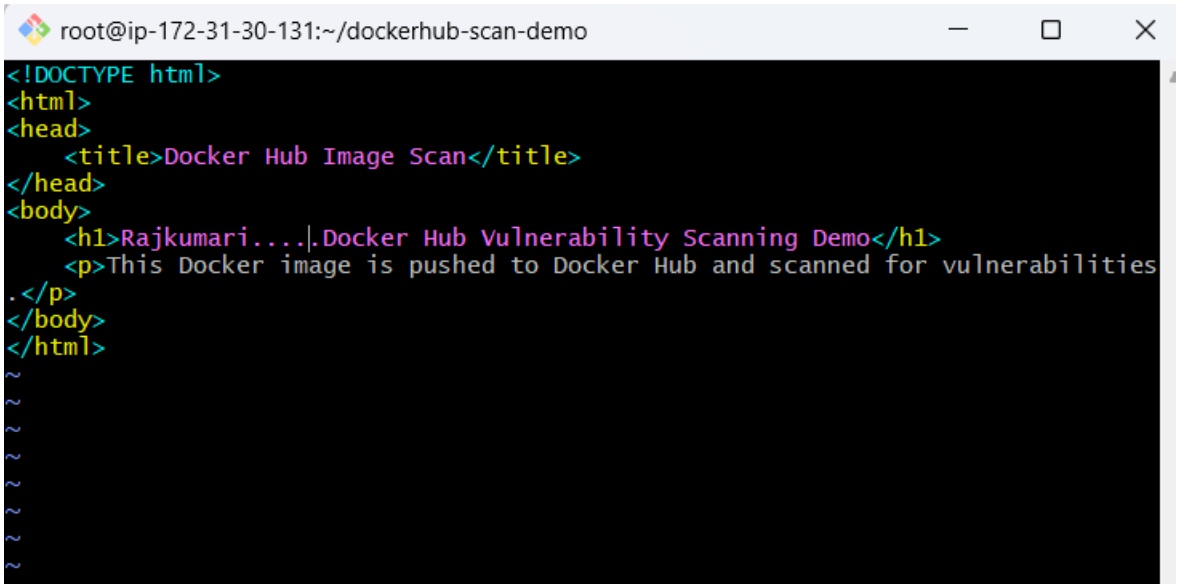
vi Dockerfile



5. Create Sample HTML File

Create:

vi index.html



6. Build the Docker Image

Run:

docker build -t dockerhub-scan-demo:v1 .

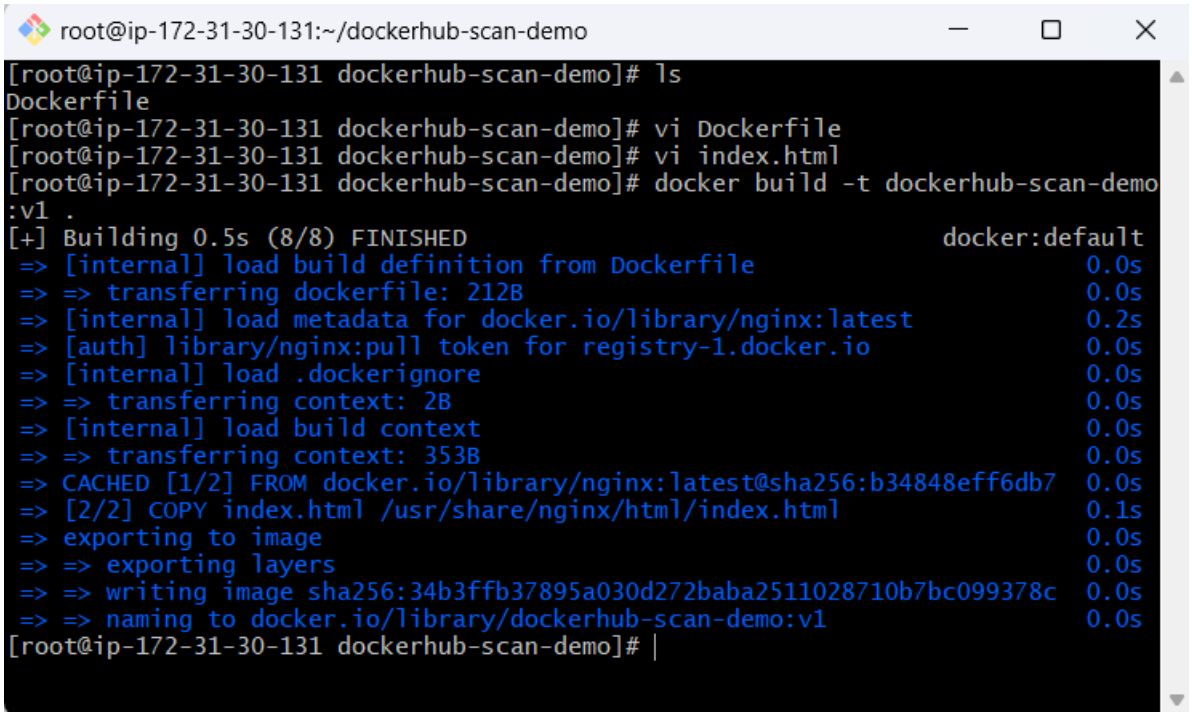
Verify:

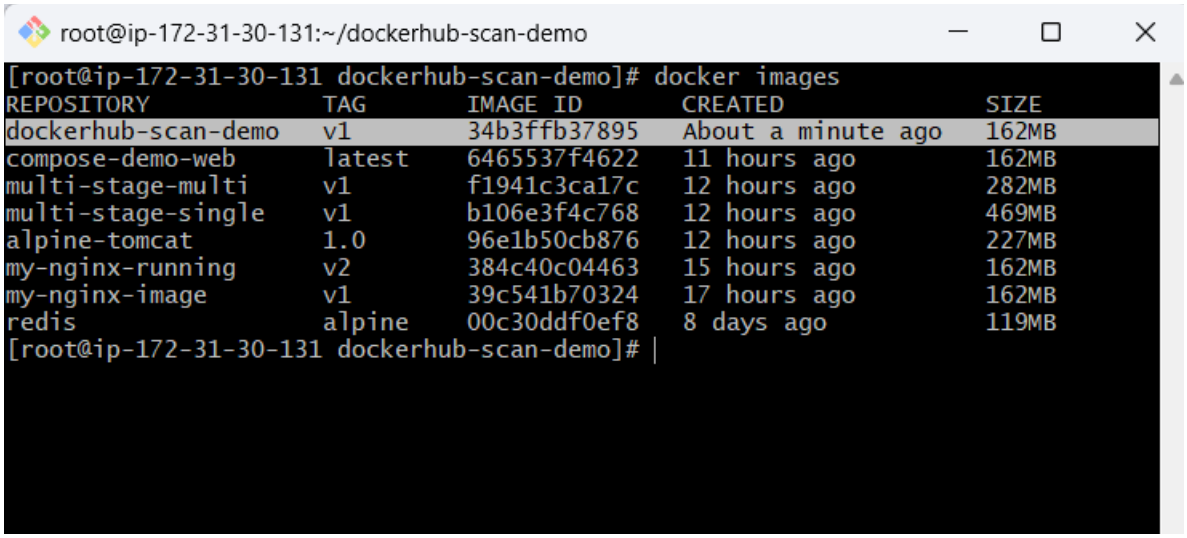
docker images

You should see:

REPOSITORY TAG IMAGE ID CREATED SIZE

dockerhub-scan-demo v1 xxxxxxxxxxx Few seconds ago ...



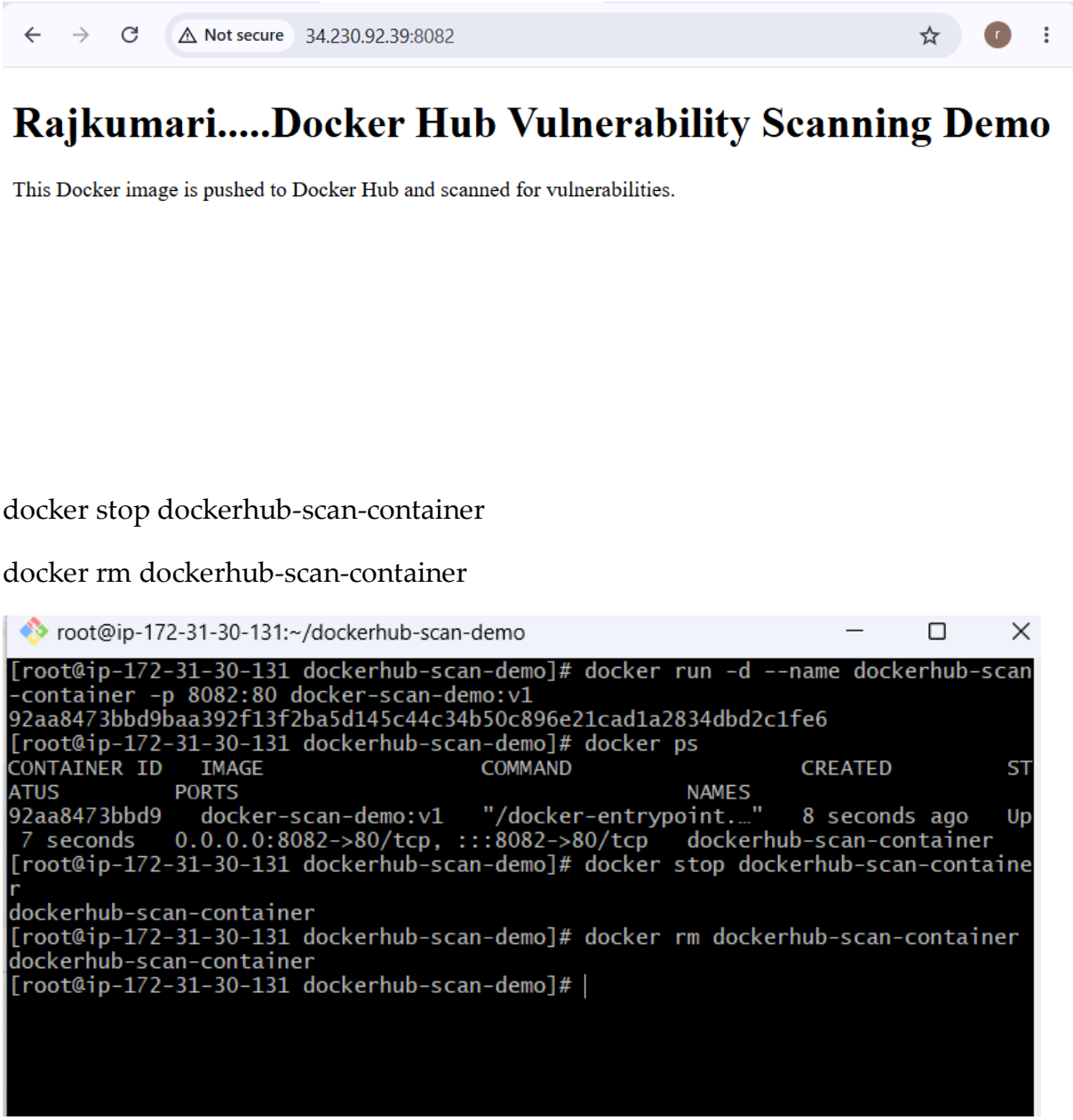


7. Test the Image Locally

Before pushing the image, test it.

Run:

```
docker run -d \
  --name dockerhub-scan-container \
  -p 8080:80 \
  dockerhub-scan-demo:v1
```



docker stop dockerhub-scan-container

docker rm dockerhub-scan-container

8. Login to Docker Hub

Run:

docker login

You will be prompted:

Username:

Password:

Enter your Docker Hub username and Personal Access Token.

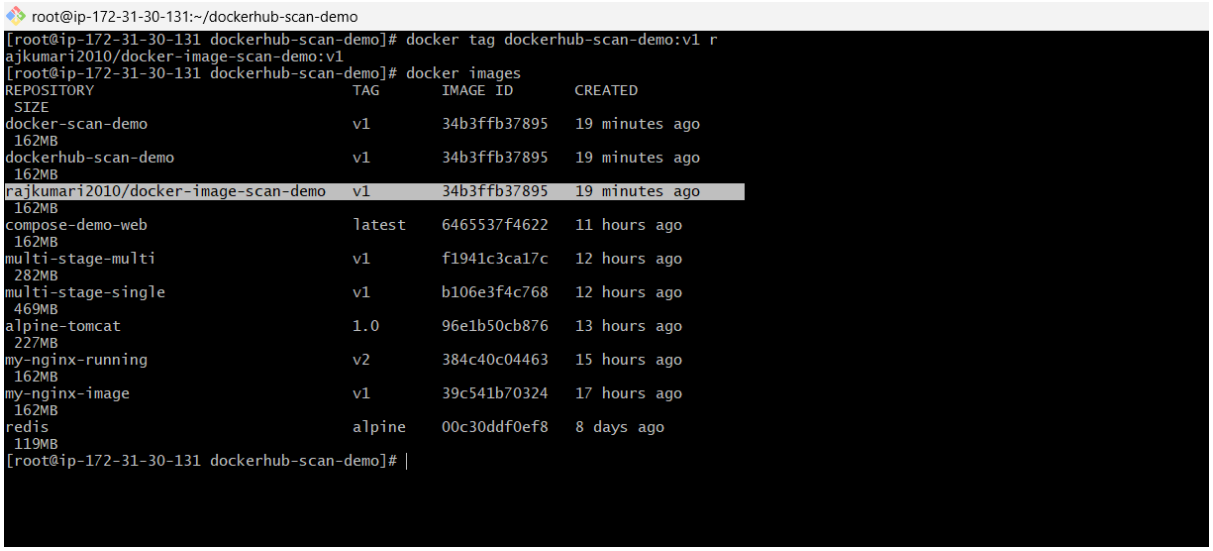
Expected:

Login Succeeded

You can verify Docker's stored login configuration:

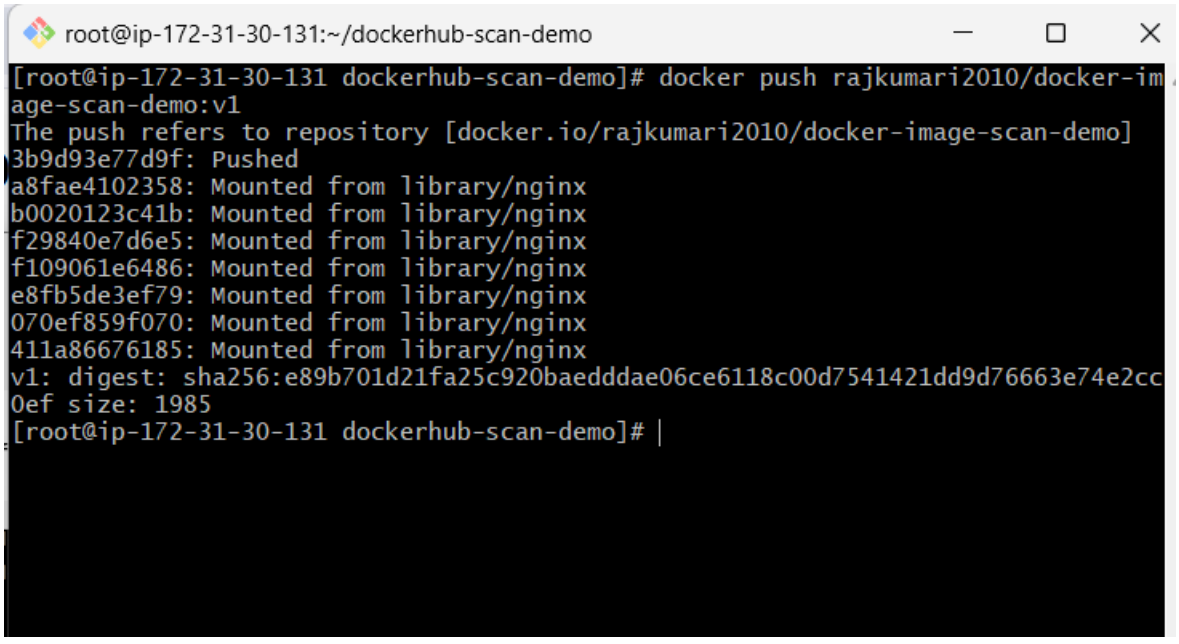
cat ~/.docker/config.json

Be careful not to share this file because it contains Docker authentication information.



10. Push the Image to Docker Hub

Run: docker push rajkumari2010/docker-image-scan-demo:v1



11. Verify the Image in Docker Hub

Open:

Docker Hub Repositories

Navigate to:

Repositories

↓

docker-image-scan-demo

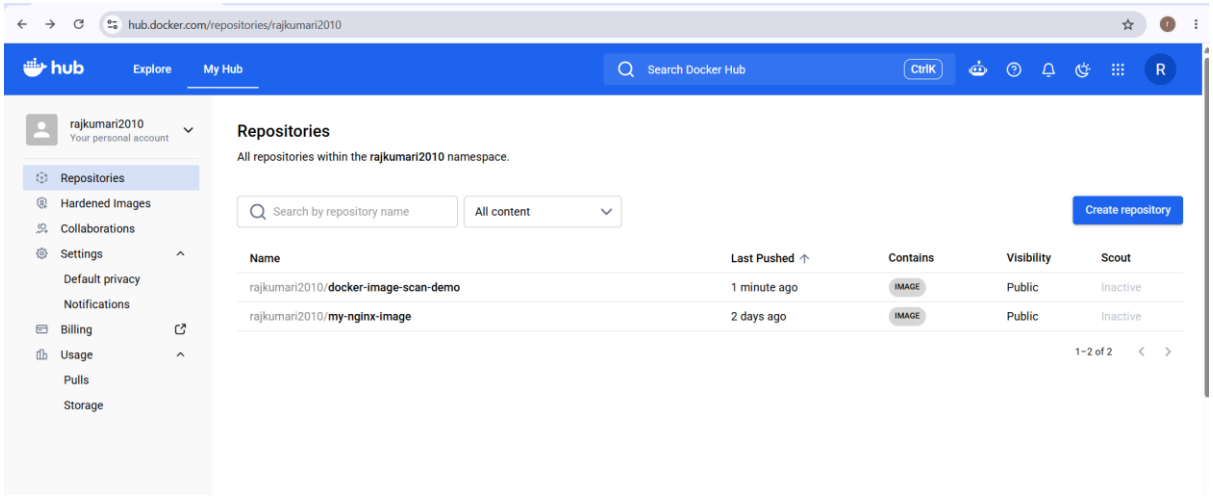
↓

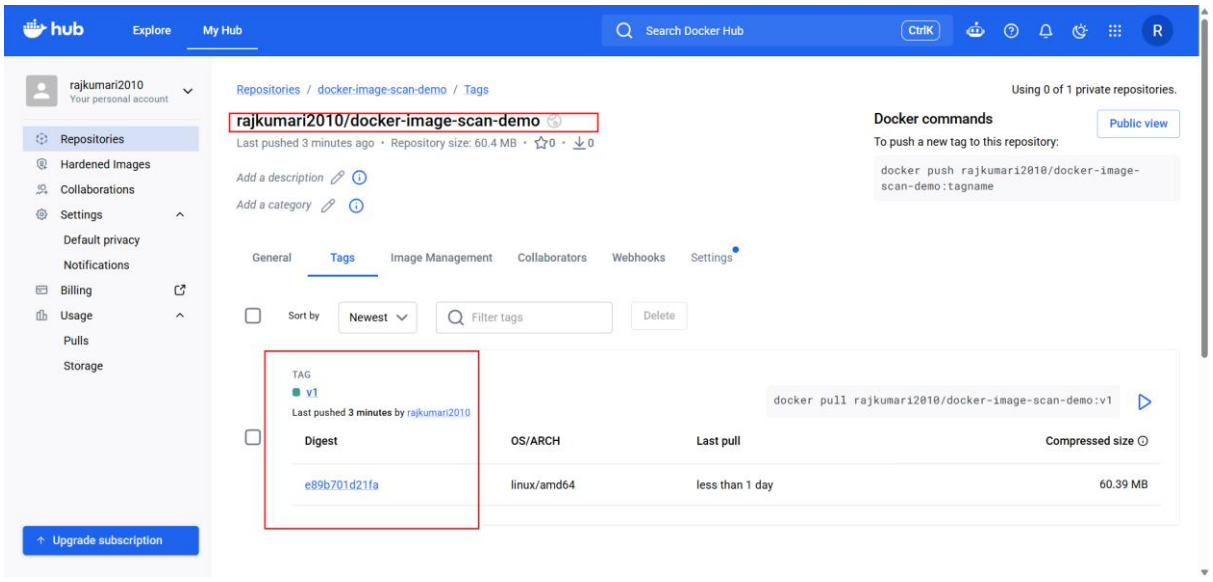
Tags

↓

v1

You should see the pushed image.





12. Enable/Check Vulnerability Scanning

Open the repository in Docker Hub.

Look for a section such as:

Vulnerabilities

or:

Scan

Depending on your Docker Hub plan/UI, vulnerability scanning may appear at the repository or image/tag level.

The expected workflow is:

Docker Hub Repository

|

↓

Image Tag

|

↓

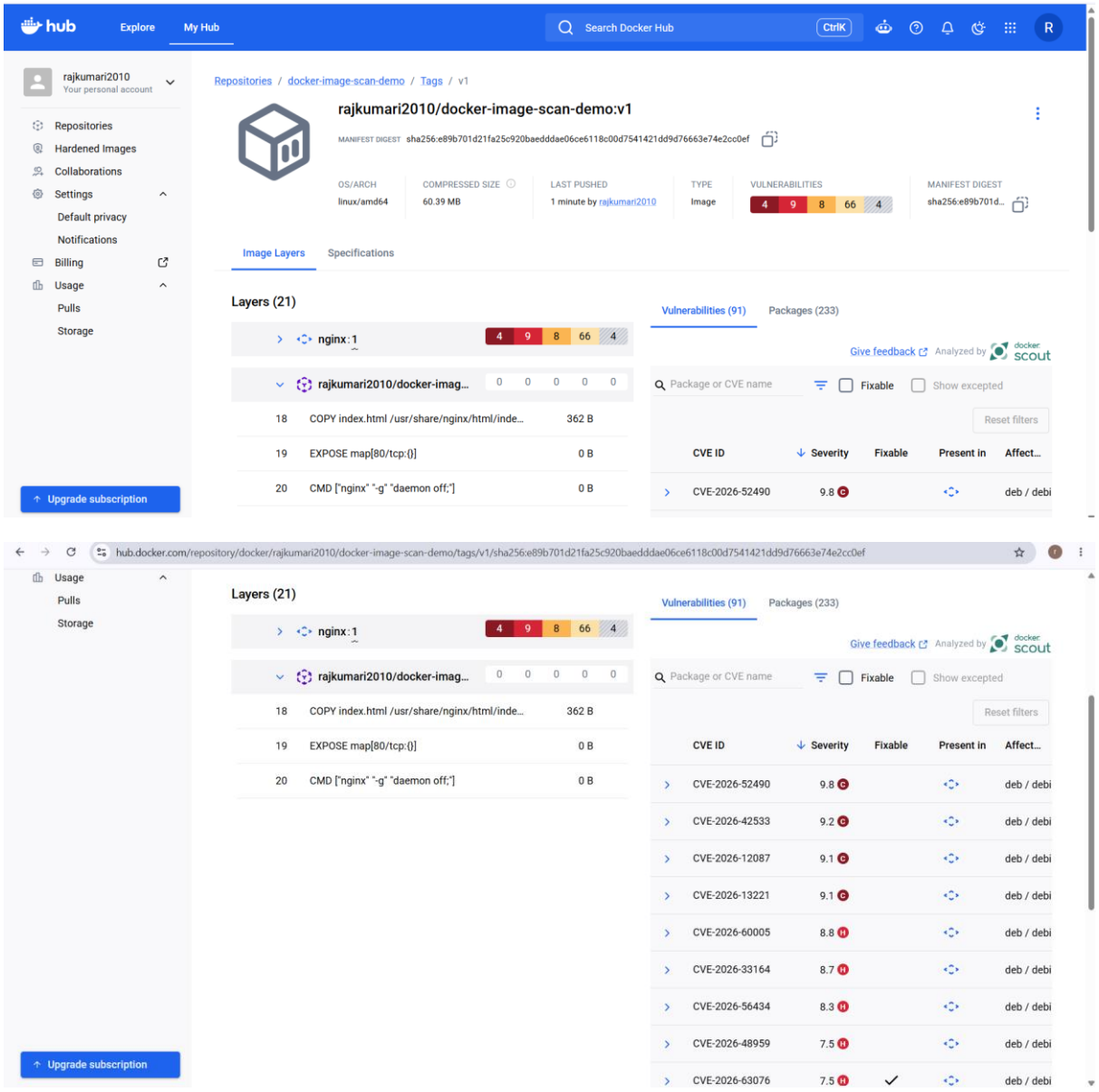
Vulnerabilities / Scan

|

↓

Security Findings

If scanning is available for your repository, Docker Hub will analyze the image after it has been pushed.



7. Implement solution to scan images when pushed to AWS ECR.

The complete flow is:

Docker Server / EC2

|

| docker build

↓

Docker Image

|

| docker push

↓

AWS ECR

|

| Scan on Push

↓

Vulnerability Scan

|

↓

View CVE Findings

1. Prerequisites

Make sure Docker and AWS CLI are installed.

Check Docker

docker --version

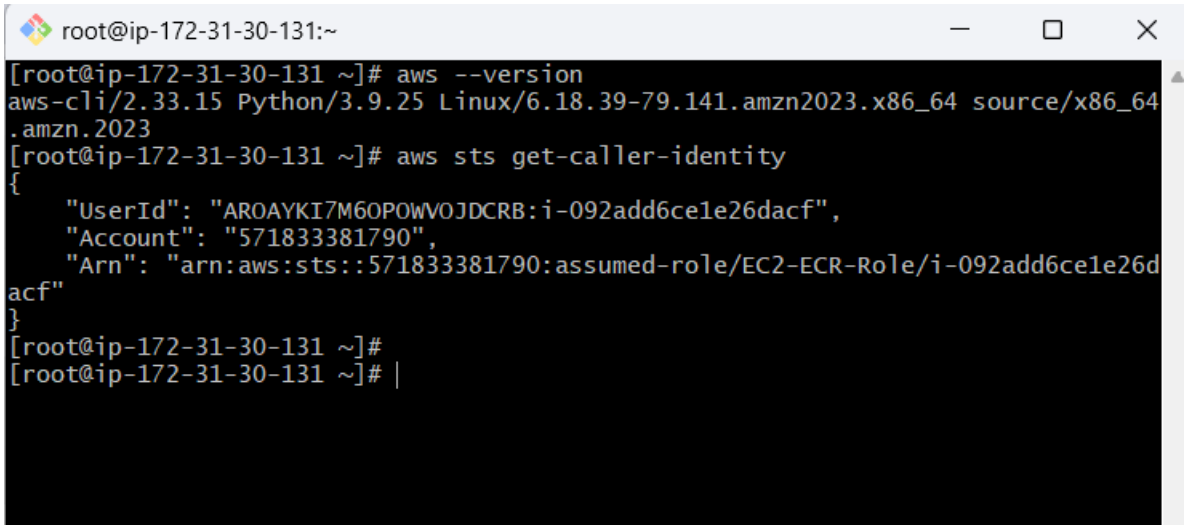
Check AWS CLI

aws --version

Verify AWS credentials

aws sts get-caller-identity

You should see your AWS Account ID and IAM role/user



2. Create an ECR Repository

We will create a repository called:

docker-scan-demo

Run:

```
aws ecr create-repository \
  --repository-name docker-scan-demo \
  --region us-east-1
```

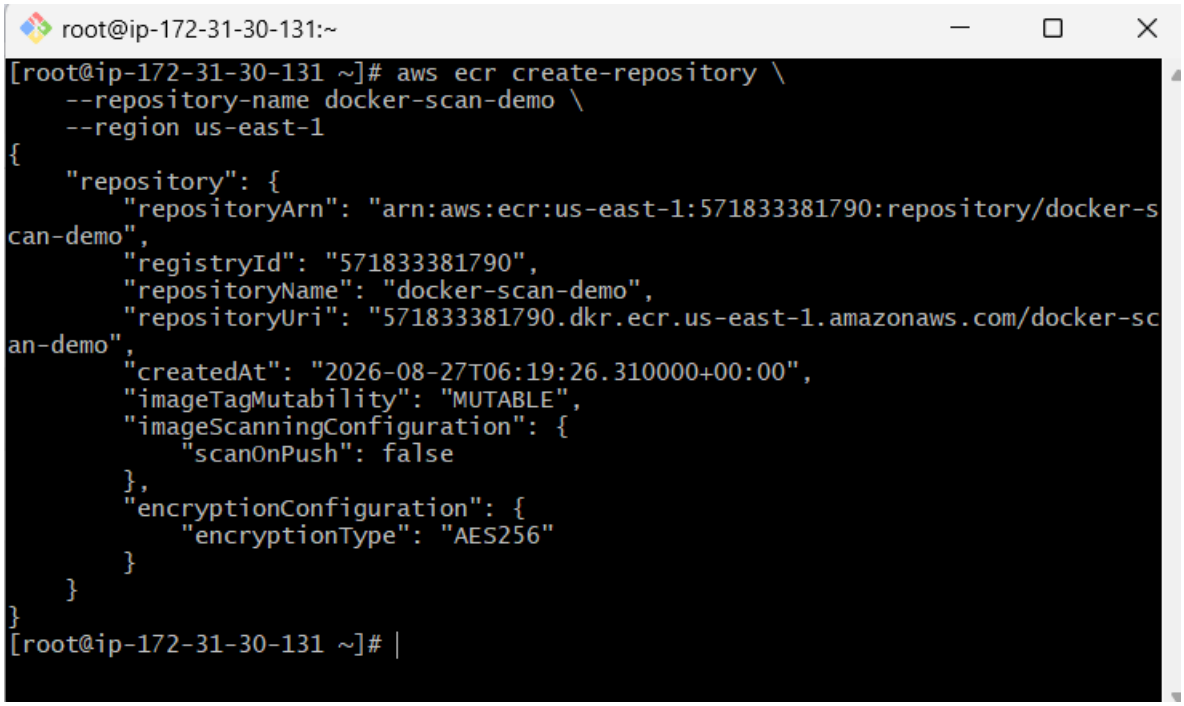
Verify:

```
aws ecr describe-repositories \
  --repository-names docker-scan-demo \
  --region us-east-1
```

Your repository URL will look like:

123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo

Replace 123456789012 with your AWS Account ID.

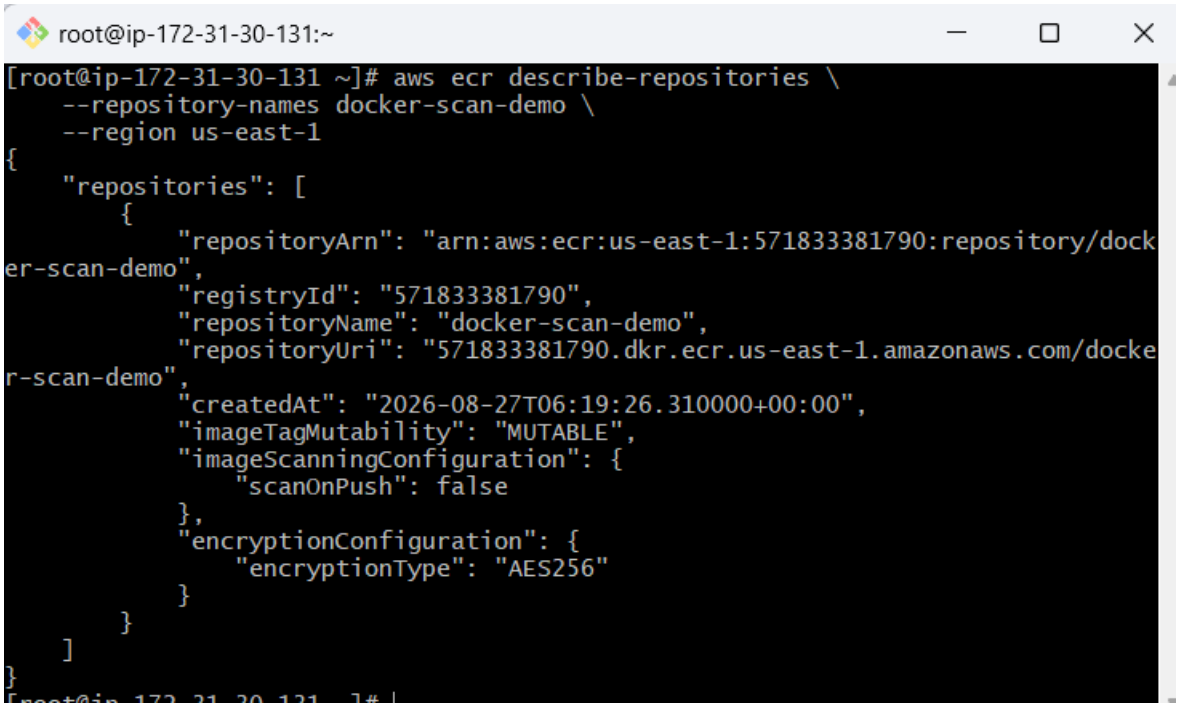


Verify:

aws ecr describe-repositories \

--repository-names docker-scan-demo \

--region us-east-1



3. Enable ECR Scan on Push

You can enable scanning from the AWS Console.

Go to:

AWS Console

↓

Amazon ECR

↓

Private registry

↓

Scanning

Select:

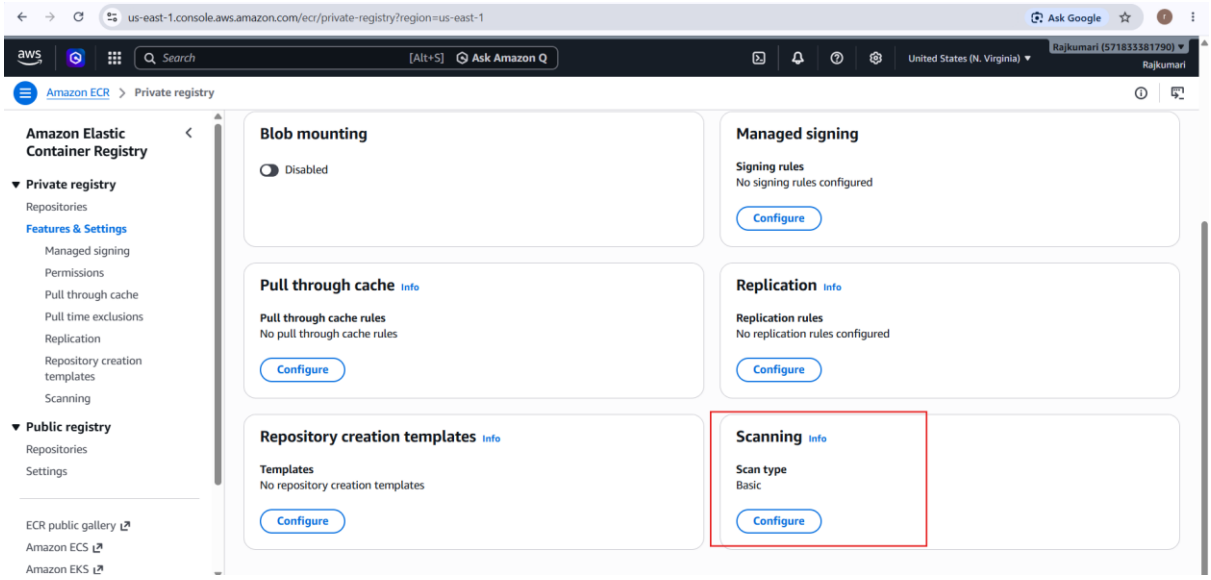
Basic scanning

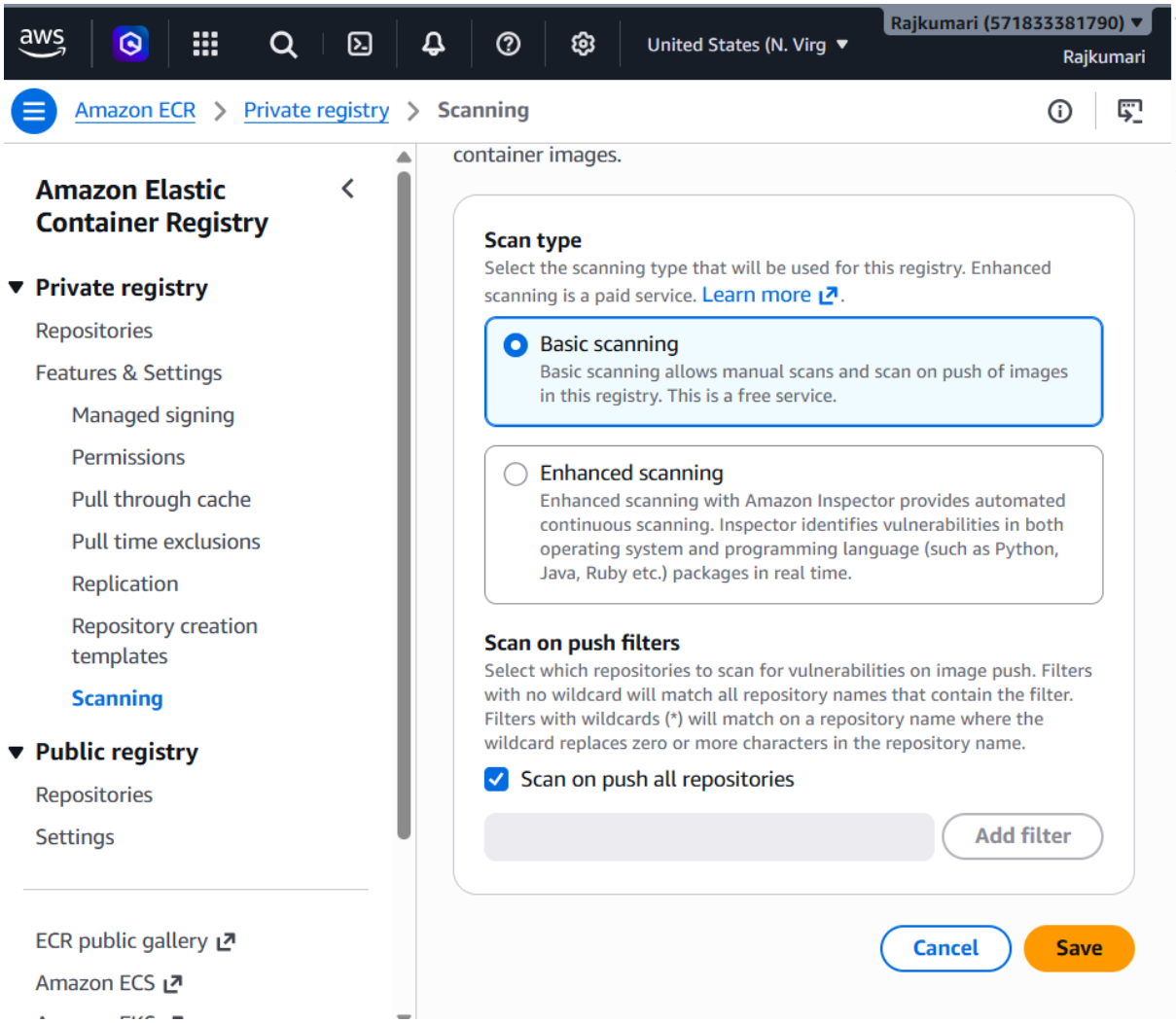
Configure:

Scan on push

You can apply it to all repositories or configure a repository filter.

AWS ECR Basic Scanning supports automatic scanning when images are pushed to the registry.





4. Enable Scan on Push Using AWS CLI

You can also configure it from the command line.

Verify:

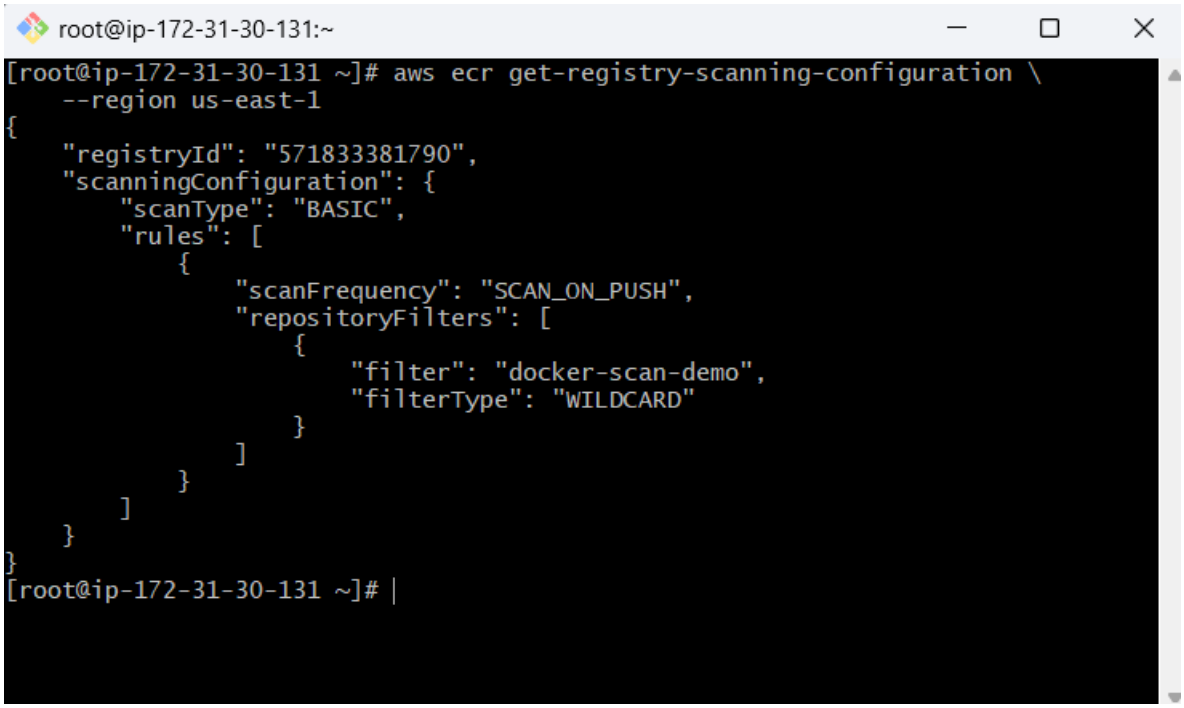
aws ecr get-registry-scanning-configuration \

--region us-east-1

You should see:

scanType: BASIC

scanFrequency: SCAN_ON_PUSH



5. Create a Sample Docker Image

Create a directory:

mkdir docker-scan-demo

cd docker-scan-demo

Create the Dockerfile:

vi Dockerfile

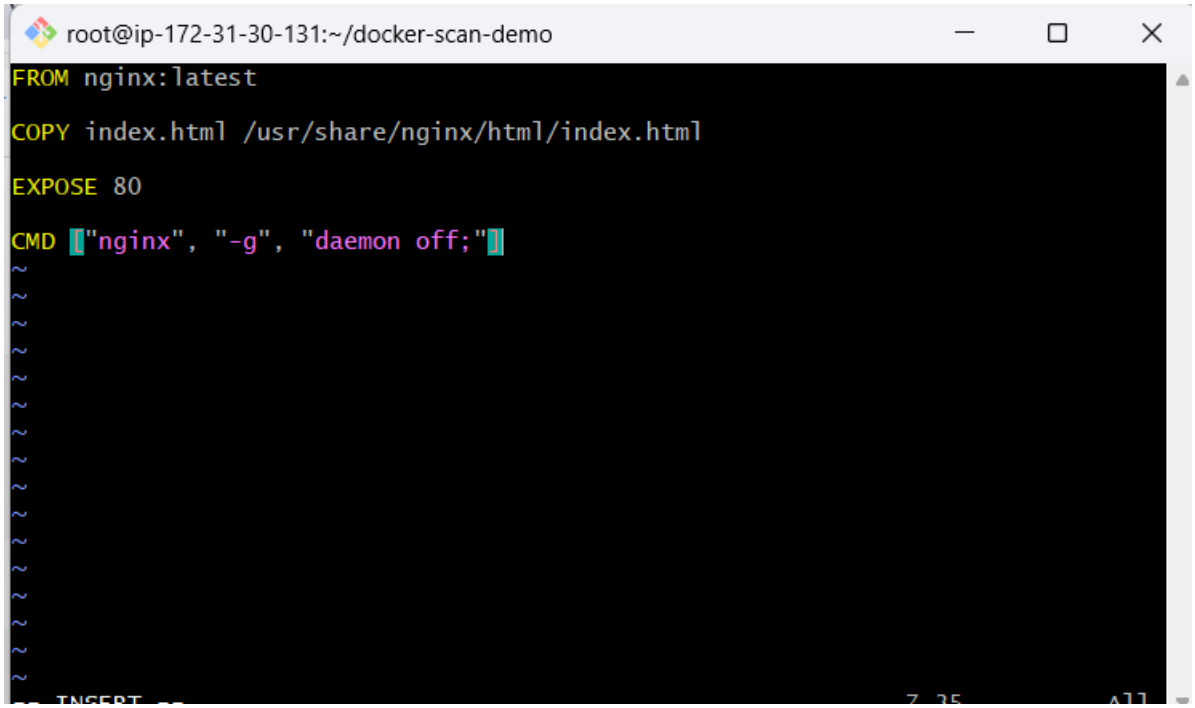
Add:

FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

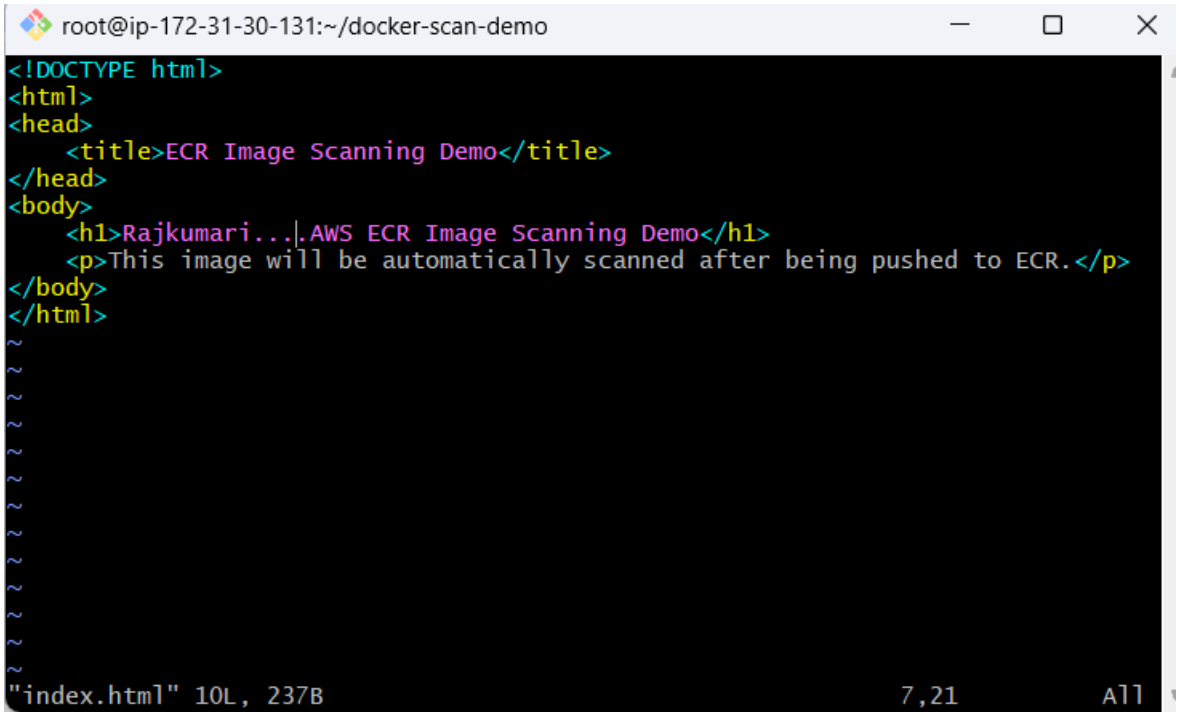
EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]



6. Create index.html

vi index.html



7. Build the Docker Image

Run:

docker build -t docker-scan-demo:v1 .

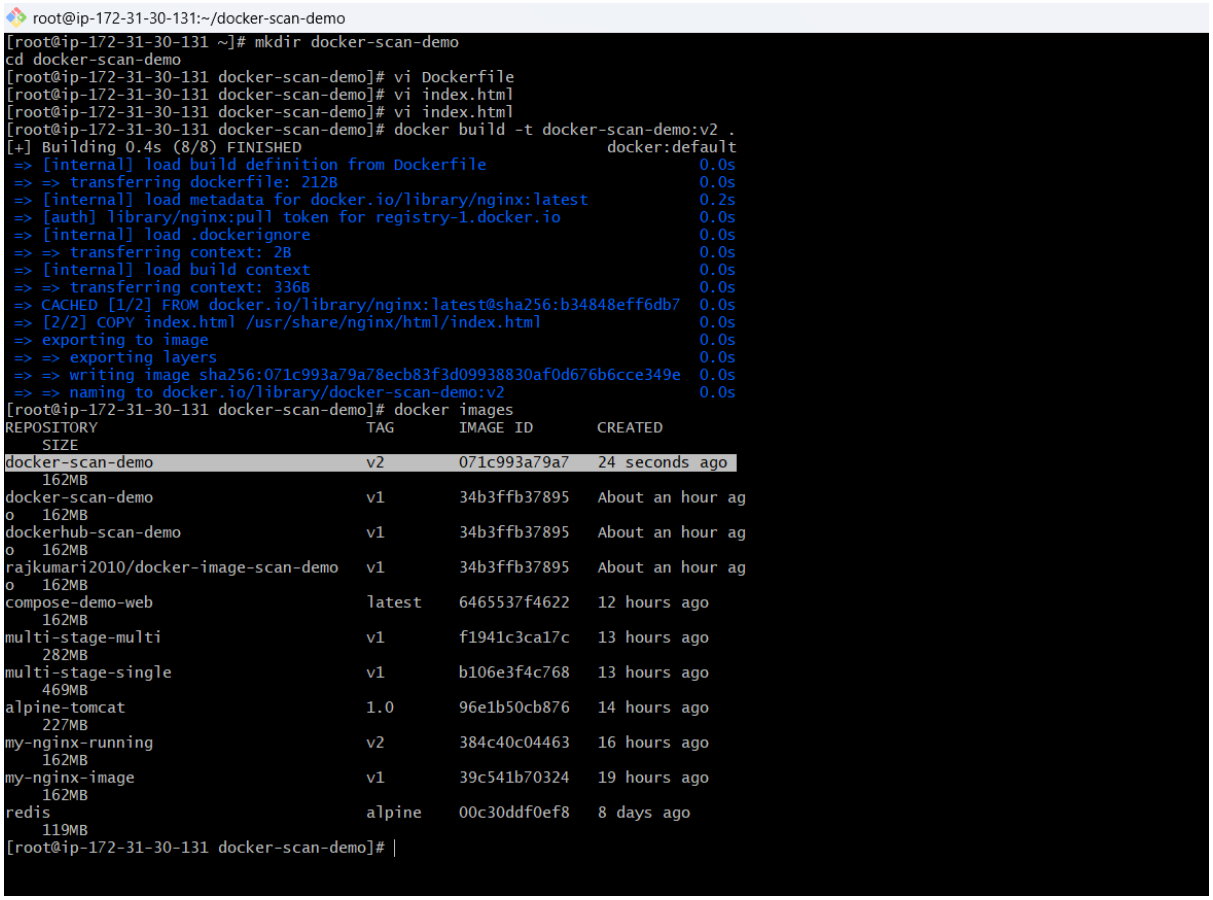
Verify:

docker images

You should see:

REPOSITORY TAG IMAGE ID CREATED SIZE

docker-scan-demo v1 xxxxxxxxxxxx Few seconds ago ...



8. Test the Image Locally

Run:

docker run -d \

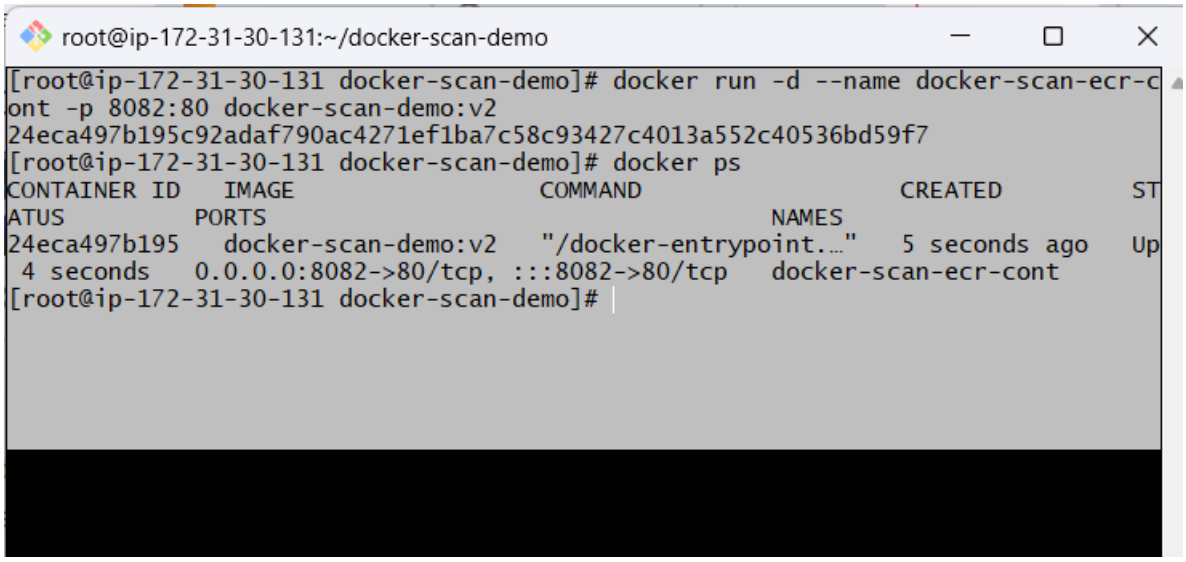
--name docker-scan-container \

-p 8080:80 \

docker-scan-demo:v1

Check:

docker ps



Rajkumari....AWS ECR Image Scanning Demo

This image will be automatically scanned after being pushed to ECR.

You should receive the HTML page.

After testing:

docker stop docker-scan-container

docker rm docker-scan-container

9. Authenticate Docker with ECR

Run: **9. Authenticate Docker with ECR**

Run:

aws ecr get-login-password --region us-east-1 | \

docker login \

--username AWS \

--password-stdin \

123456789012.dkr.ecr.us-east-1.amazonaws.com

Replace with your account ID

10. Tag the Image

Tag the image with your ECR repository URL.

docker tag docker-scan-demo:v1 \

123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo:v1

Verify:

docker images

You should now see:

docker-scan-demo

v1

123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo

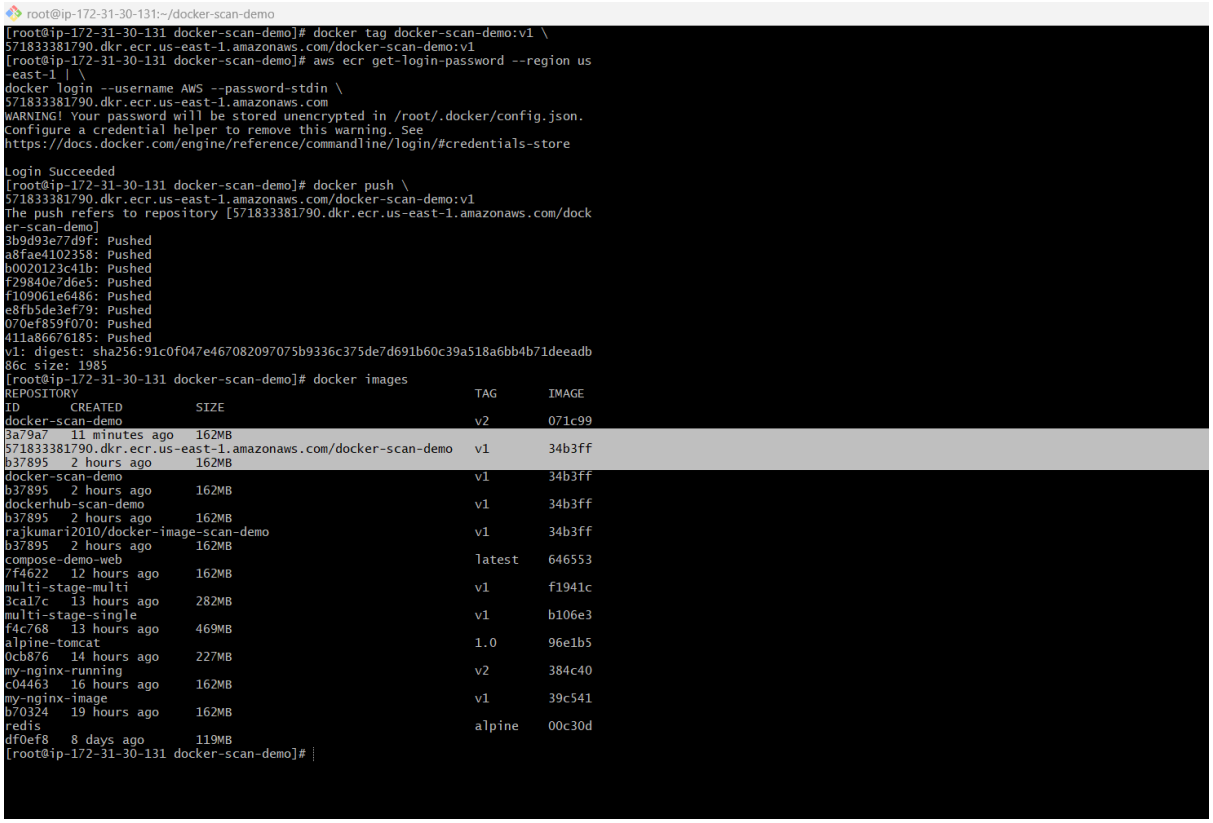
v1

11. Push the Image to ECR

Run:

docker push \

123456789012.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo:v1



12. Check the Image in ECR

Go to:

AWS Console

↓

Amazon ECR

↓

Repositories

↓

docker-scan-demo

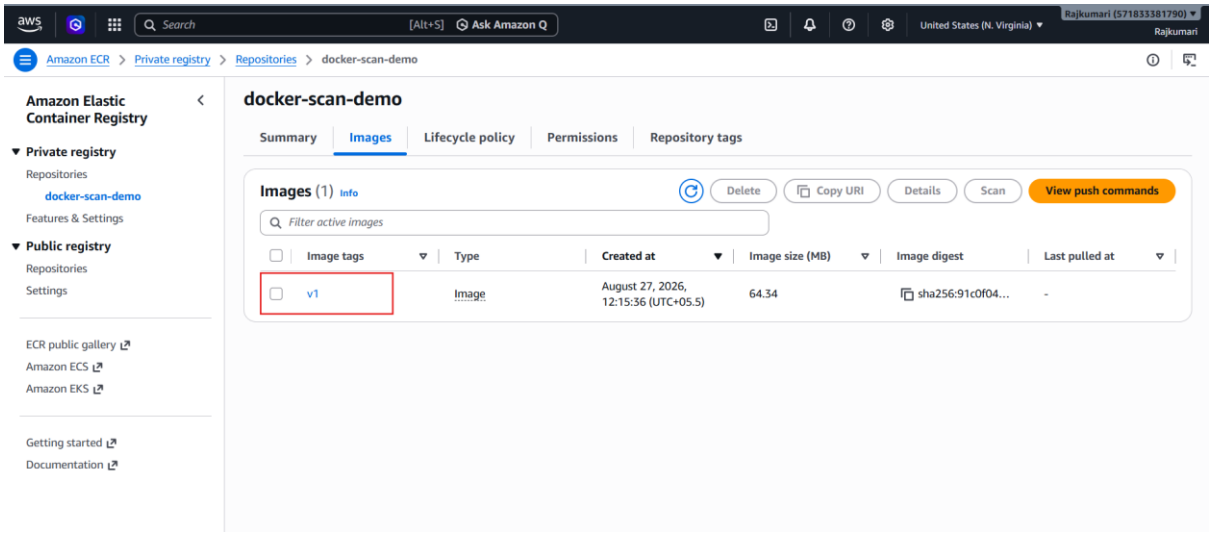
You should see:

Image tag: v1

Click/select the image.

Look for the vulnerability/security information.

The scan may initially be in progress.



13. View Vulnerability Findings

After the scan completes, ECR will show vulnerability findings.

You may see:

CRITICAL

HIGH

MEDIUM

LOW

For example:

CRITICAL 0

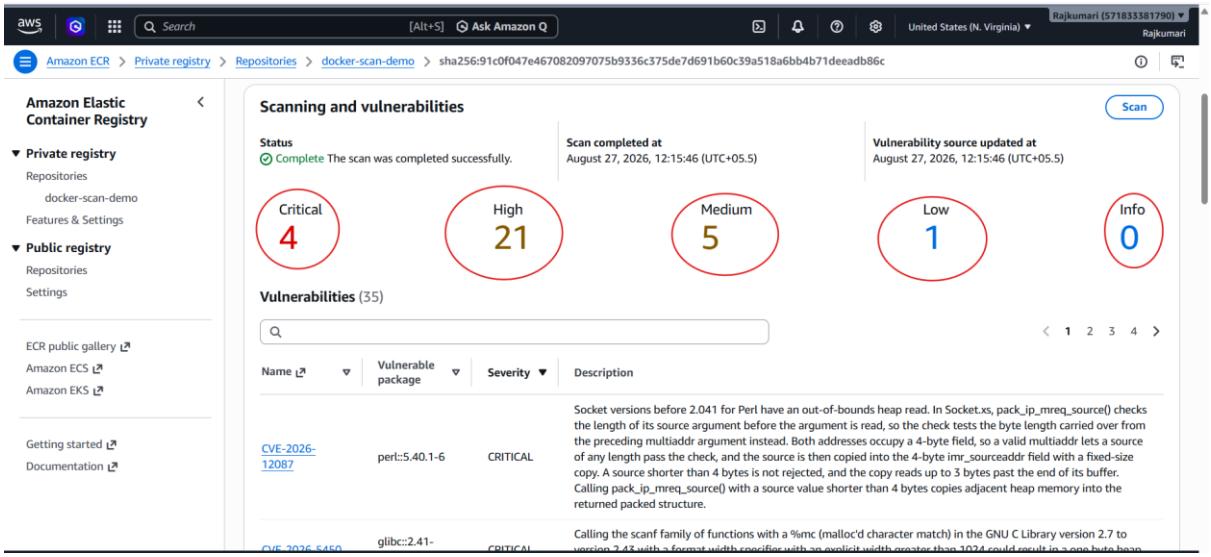
HIGH 2

MEDIUM 5

LOW 3

The actual numbers will depend on the image and current vulnerability database.

Basic ECR scanning primarily identifies vulnerabilities in the operating-system packages contained in the image.



8. Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

This is a very common **CI/CD practical task**. Since you are working with Jenkins and Docker, I'll show you a complete setup you can use for your documentation and practice.

Architecture

GitHub

|

| git clone

↓

Jenkins

|

↓

Docker Build

|

↓

Docker Image

|

↓

Docker Hub Login

|

↓

Docker Push

|

↓

Docker Hub

---

1. Prerequisites

Make sure you have:

- Jenkins installed
- Docker installed on the Jenkins server/agent
- Git installed
- A GitHub repository containing a Dockerfile
- Docker Hub account
- Docker Hub Personal Access Token
- Jenkins Docker Pipeline support/plugins

Check Docker on the Jenkins node:

docker --version

Check Git:

git --version

Check Jenkins user can execute Docker:

sudo -u jenkins docker ps

If you get:

permission denied

add Jenkins to the Docker group:

sudo usermod -aG docker jenkins

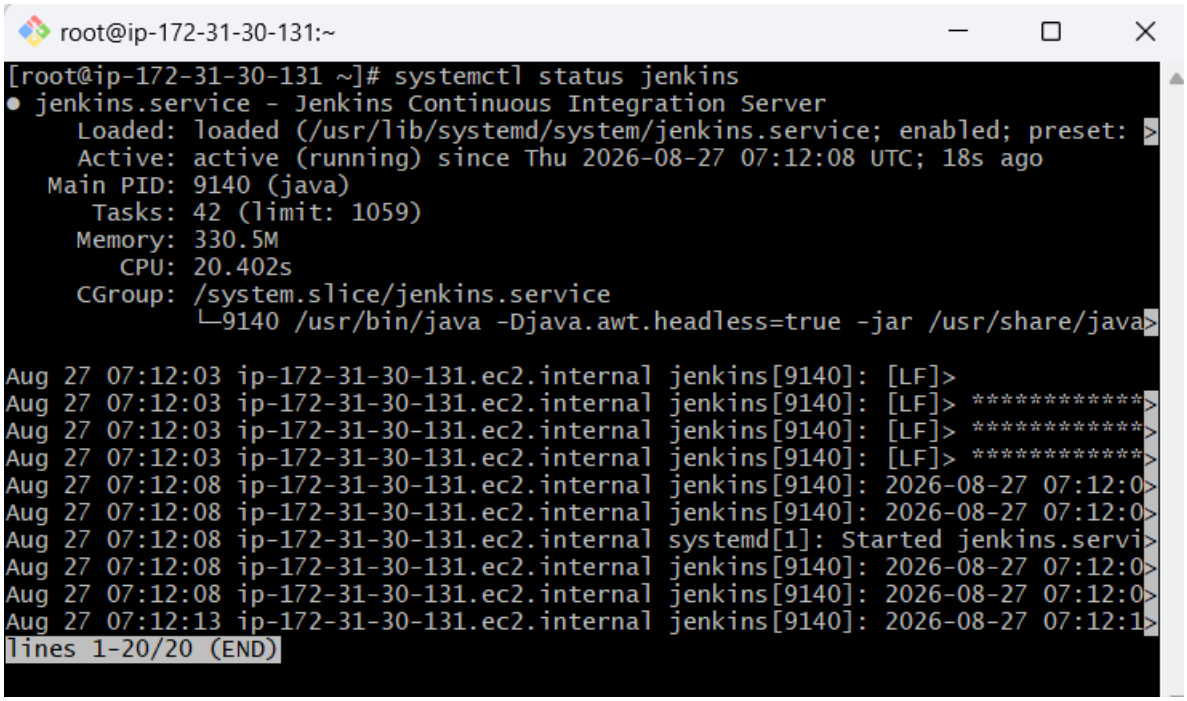
Then restart Jenkins:

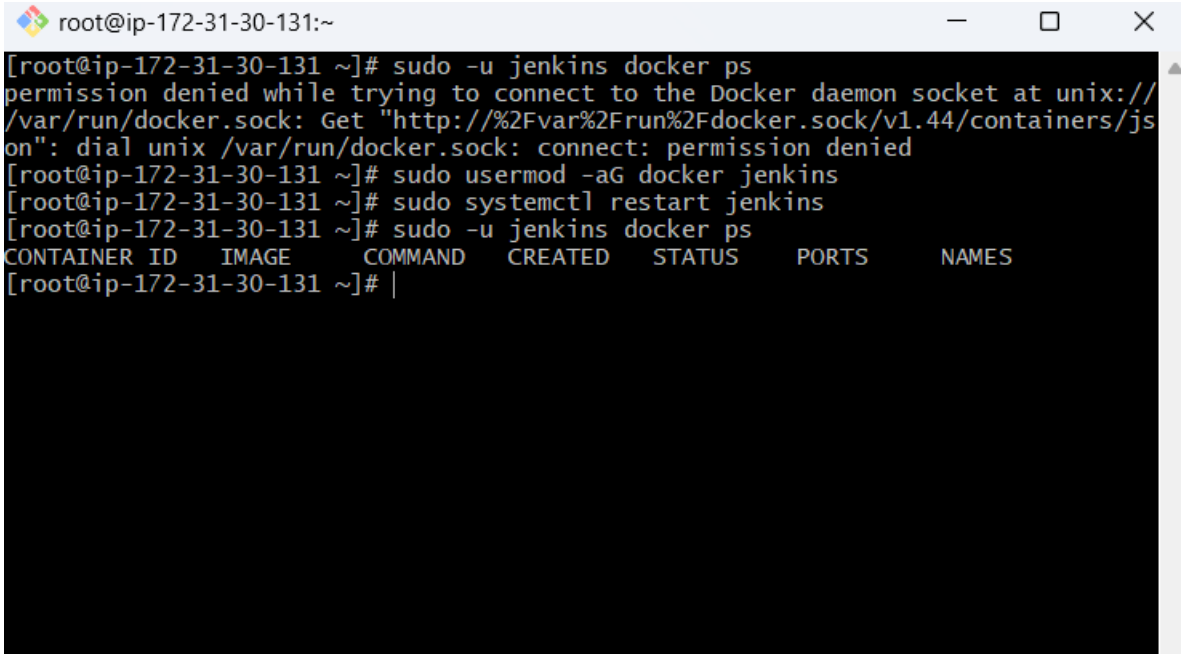
sudo systemctl restart jenkins

You may need to reconnect/restart the Jenkins agent as well.

Verify again:

sudo -u jenkins docker ps





2. Create Docker Hub Repository

Go to Docker Hub:

Docker Hub

Create a repository.

For example:

Repository name:

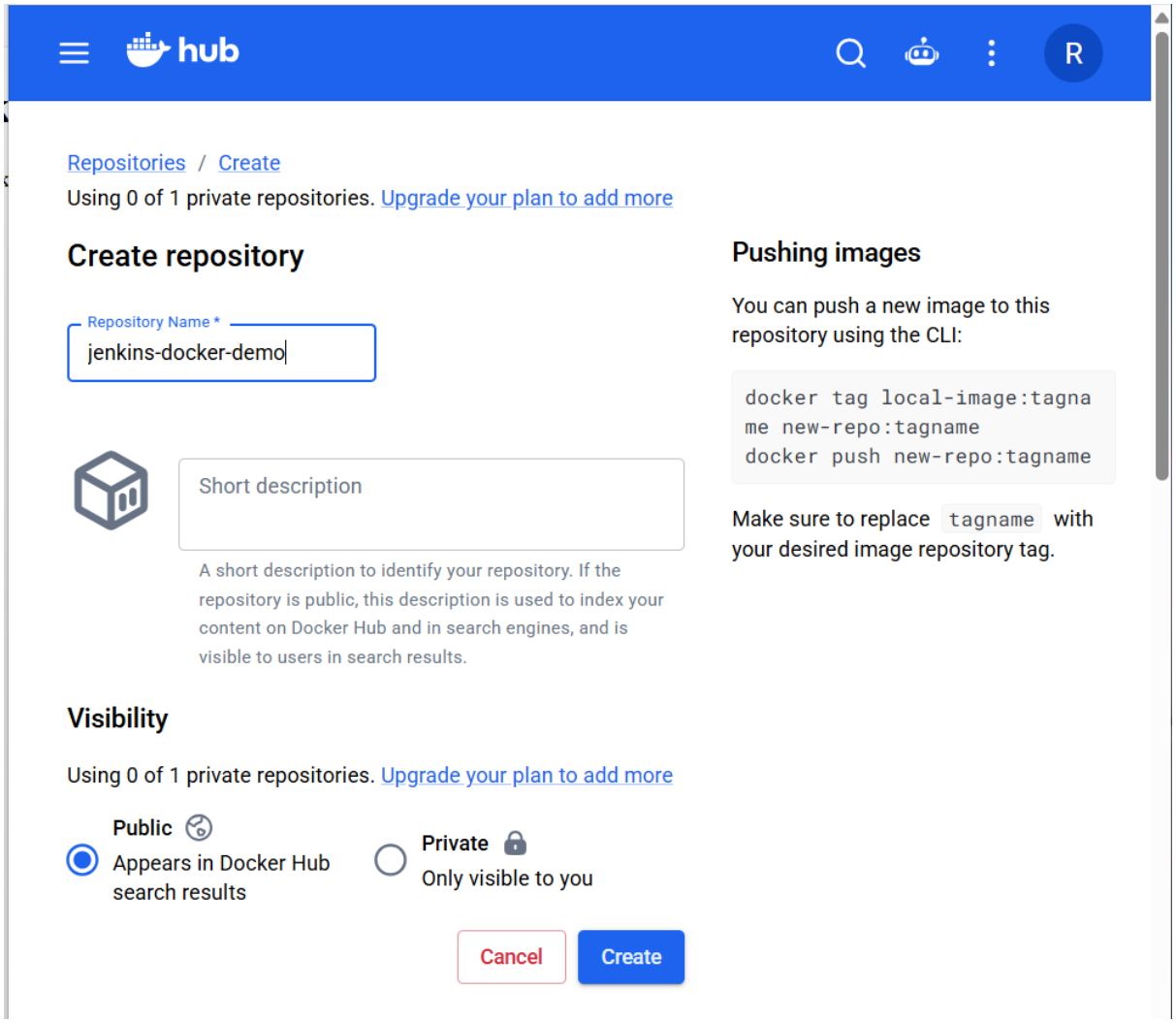
jenkins-docker-demo

Your image name will be:

<dockerhub-username>/jenkins-docker-demo

For example:

Rajkumari2010/jenkins-docker-demo



3. Create Docker Hub Access Token

Do **not** put your Docker Hub password directly inside the Jenkinsfile.

Use a Docker Hub Personal Access Token.

In Docker Hub, go to your account security settings and create an access token.

Example:

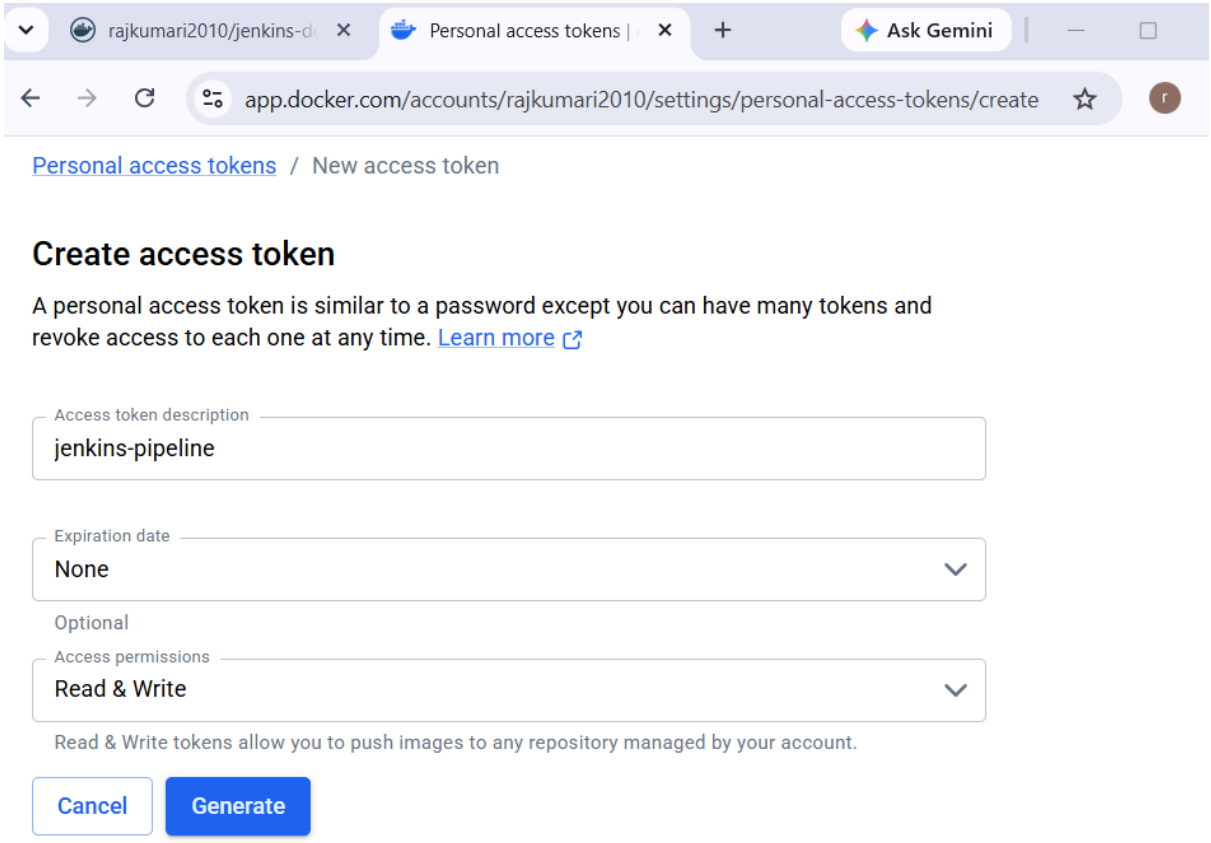
Username:

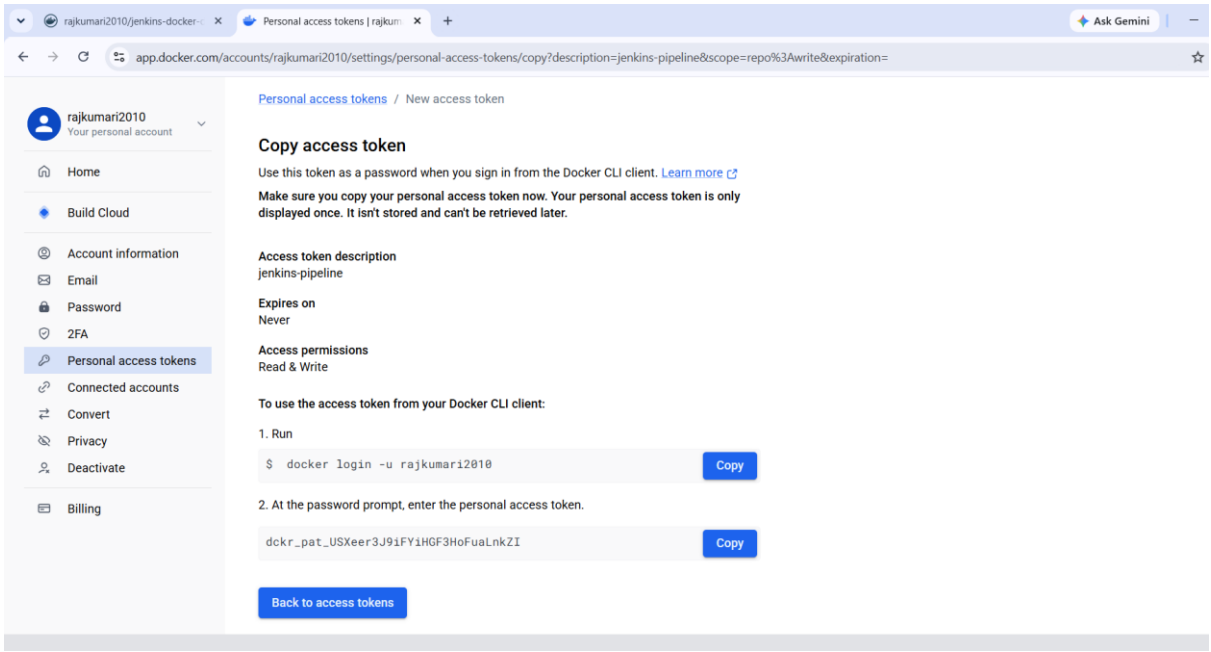
rajkumari

Token:

dckr_pat_xxxxxxxxxxxx

Keep the token secret.





. Add Docker Hub Credentials to Jenkins

Open Jenkins:

Jenkins

↓

Manage Jenkins

↓

Credentials

↓

System

↓

Global credentials

↓

Add Credentials

Choose:

Kind:

Username with password

Enter:

Username:

your Docker Hub username

For example:

rajkumari

For the password field, enter the **Docker Hub access token**, not your normal password.

Then:

ID:

dockerhub-credentials

Description:

Docker Hub Credentials

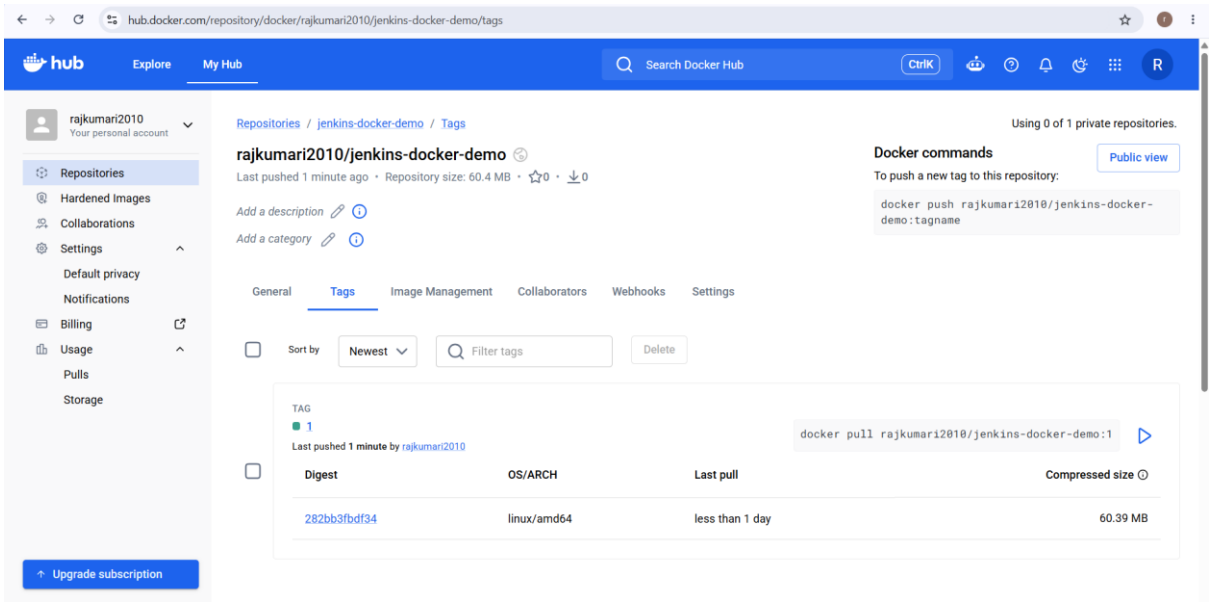
Click:

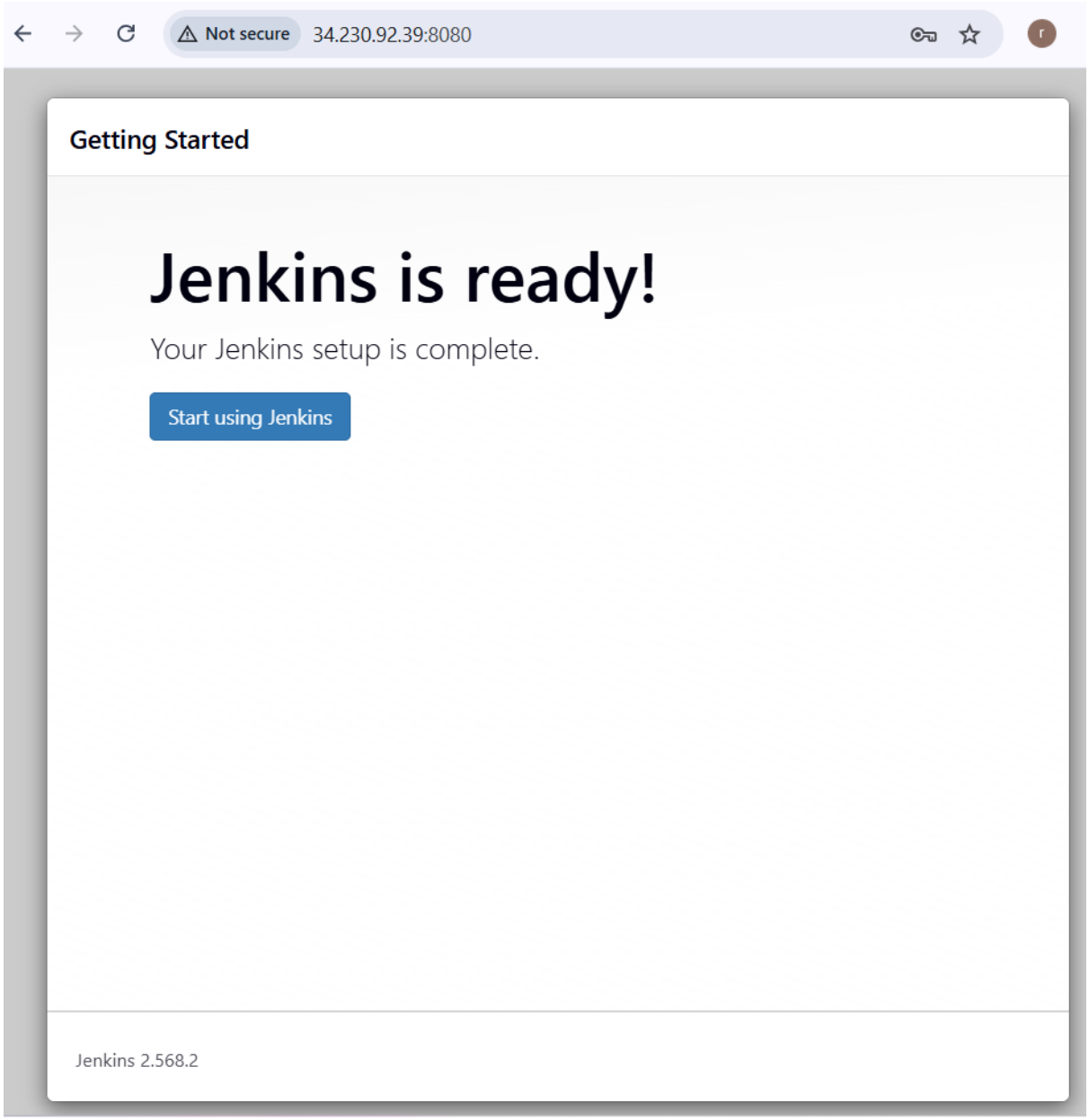
Create

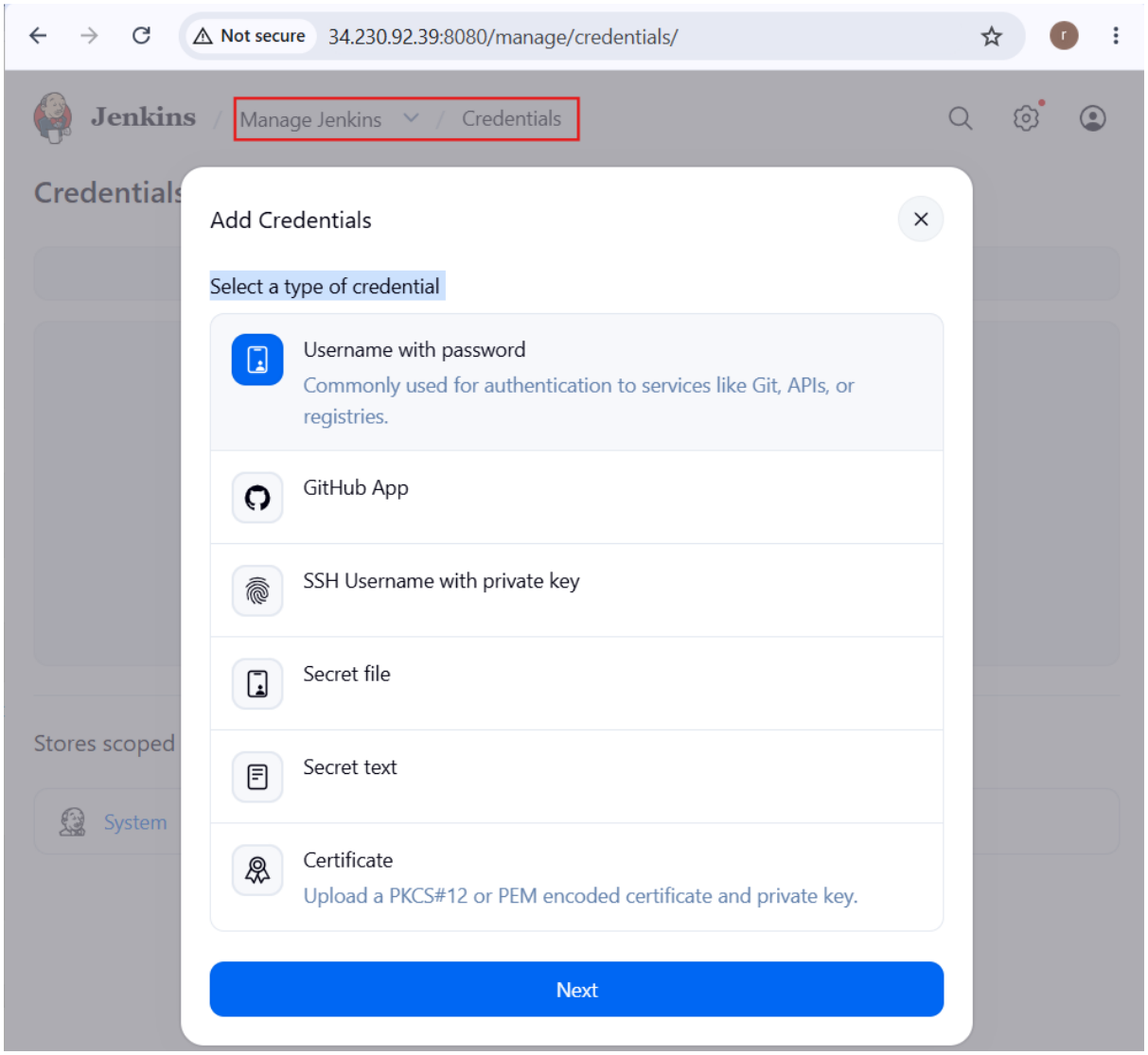
Your Jenkins credential ID is now:

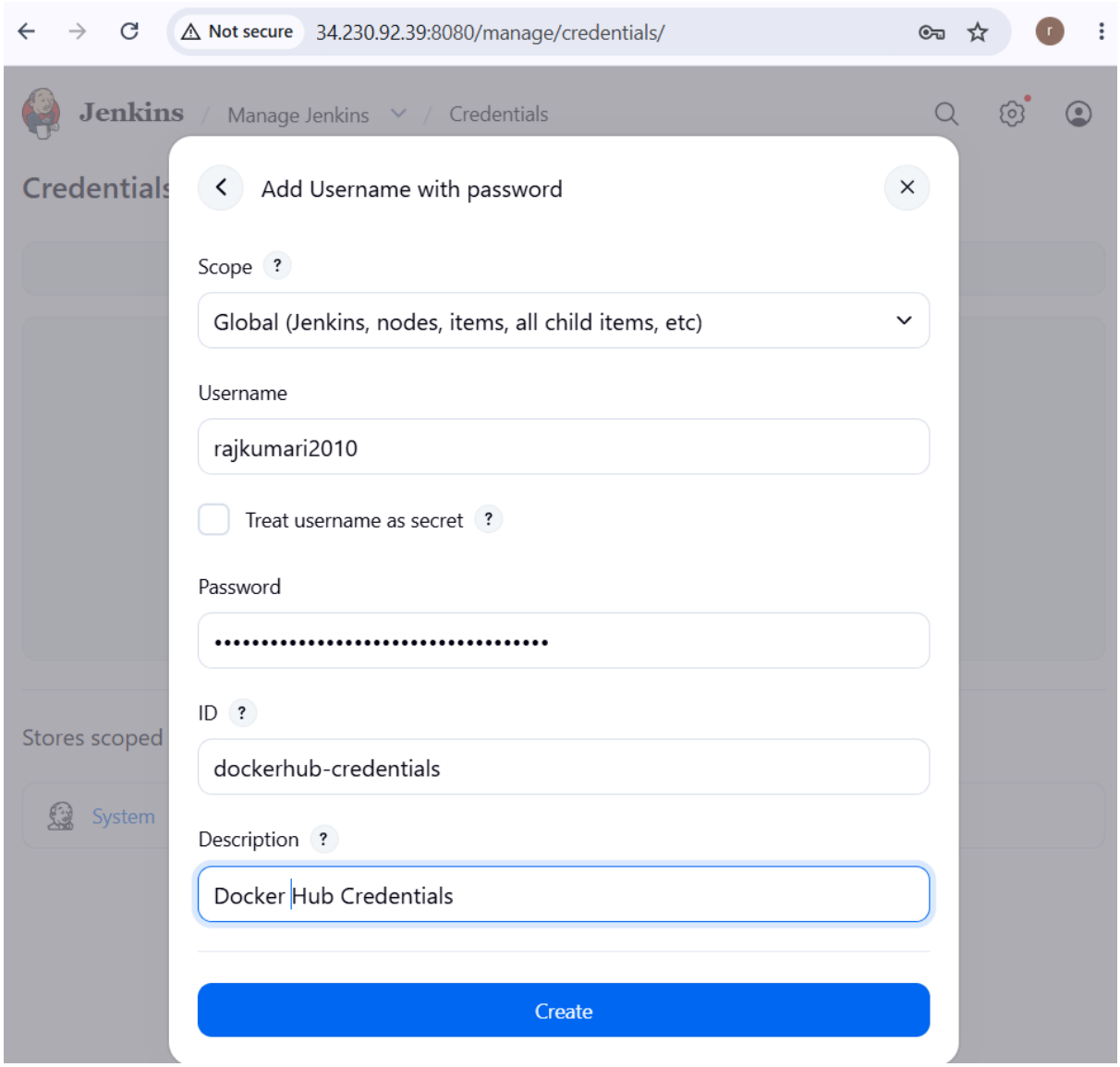
dockerhub-credentials

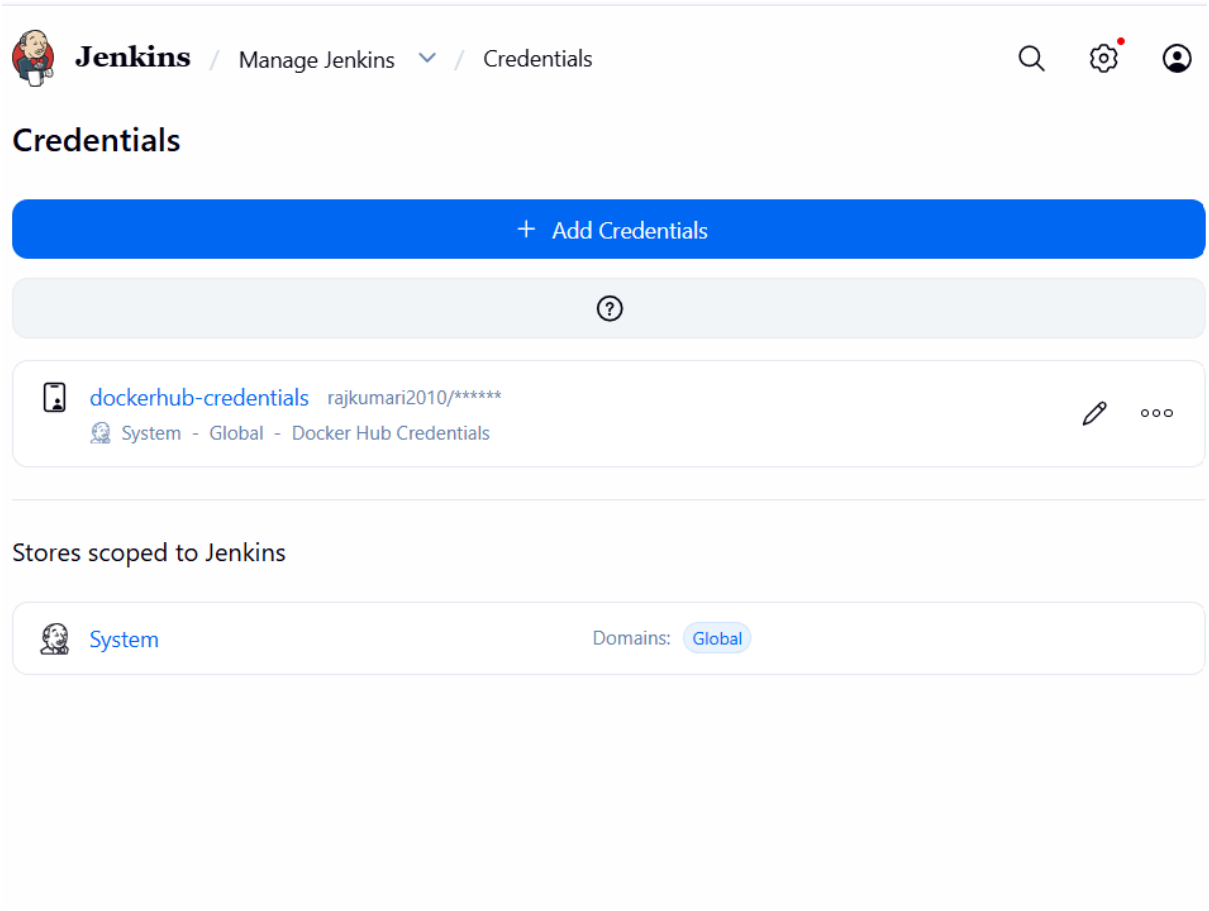
We will use this ID in the Jenkinsfile.




Not secure
34.230.92.39:8080
Getting Started
Jenkins is ready!
Your Jenkins setup is complete.
Start using Jenkins
Jenkins 2.568.2


Not secure
34.230.92.39:8080/manage/credentials/
Jenkins
Manage Jenkins
Credentials
Credentials
Add Credentials
Select a type of credential
Username with password
Commonly used for authentication to services like Git, APIs, or registries.
GitHub App
SSH Username with private key
Secret file
Secret text
Certificate
Upload a PKCS#12 or PEM encoded certificate and private key.
Next
Stores scoped
System


Not secure
34.230.92.39:8080/manage/credentials/
Jenkins
Manage Jenkins
Credentials
Credentials
Add Username with password
Scope
Global (Jenkins, nodes, items, all child items, etc)
Username
rajkumari2010
Treat username as secret
Password
ID
dockerhub-credentials
Description
Docker Hub Credentials
Create
Stores scoped
System



5. Create a Sample Docker Project

For example, create:

jenkins-docker-demo

Project structure:

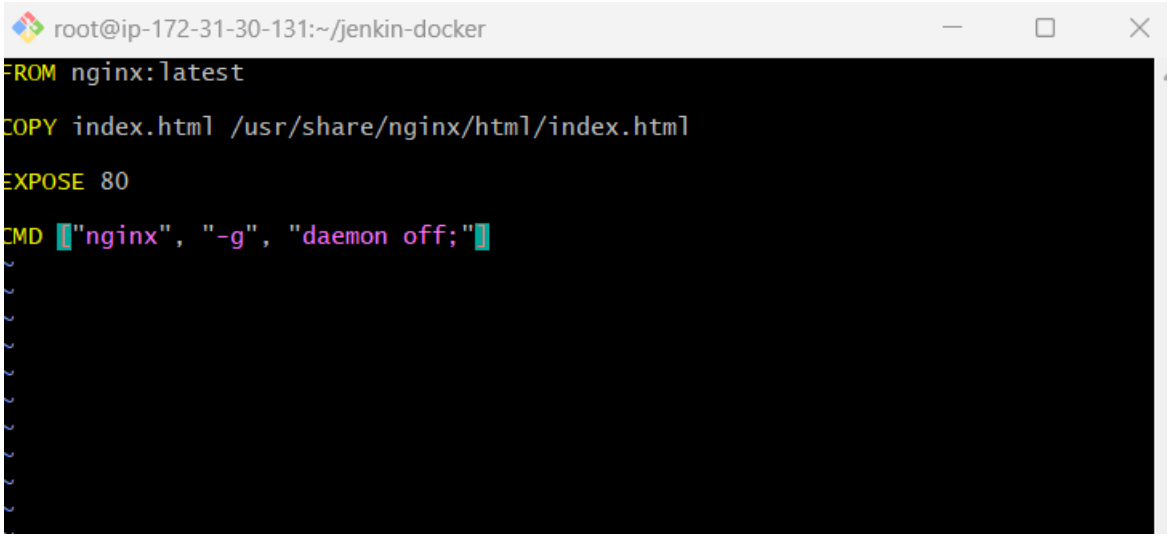
jenkins-docker-demo/

```
|
├── Dockerfile
├── index.html
└──── Jenkinsfile
```

6. Create Dockerfile

Create:

vi Dockerfile



7. Create index.html

8. Test Docker Build Manually

Before involving Jenkins, test your Dockerfile:

docker build -t jenkins-docker-demo:v1 .

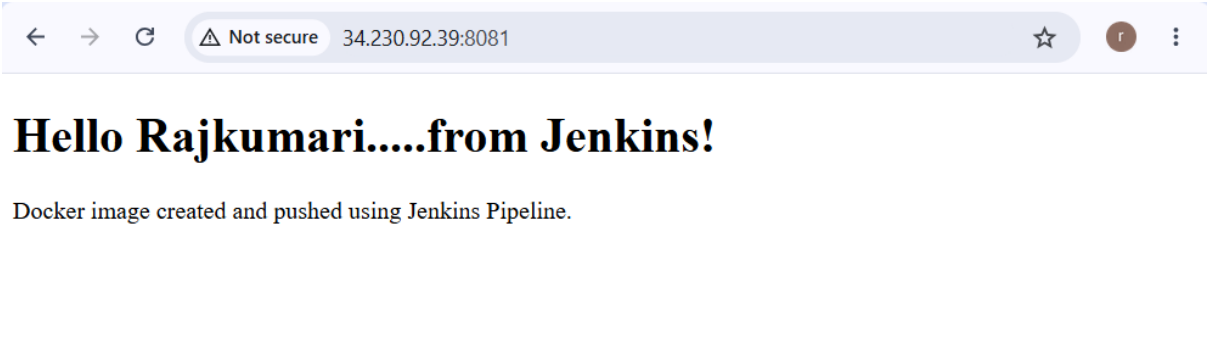
Check:

docker images

```
root@ip-172-31-30-131:~/jenkin-docker
[root@ip-172-31-30-131 jenkin-docker]# vi Dockerfile
[root@ip-172-31-30-131 jenkin-docker]# vi index.html
[root@ip-172-31-30-131 jenkin-docker]# docker build -t jenkins-docker-demo:v1 .
[+] Building 0.8s (8/8) FINISHED                                docker:default
 => [internal] load build definition from Dockerfile                      0.1s
 => => transferring dockerfile: 212B                                      0.0s
 => [internal] load metadata for docker.io/library/nginx:latest           0.4s
 => [auth] library/nginx:pull token for registry-1.docker.io              0.0s
 => [internal] load .dockerignore                                         0.0s
 => => transferring context: 2B                                           0.0s
 => [internal] load build context                                         0.0s
 => => transferring context: 313B                                         0.0s
 => CACHED [1/2] FROM docker.io/library/nginx:latest@sha256:b34848eff6db7  0.0s
 => [2/2] COPY index.html /usr/share/nginx/html/index.html                0.1s
 => exporting to image                                                    0.0s
 => => exporting layers                                                   0.0s
 => => writing image sha256:04f7a31a23ca206f5d8b49dd699c17ded53908d43b6fc  0.0s
 => => naming to docker.io/library/jenkins-docker-demo:v1                 0.0s
[root@ip-172-31-30-131 jenkin-docker]# docker imahes
docker: 'imahes' is not a docker command.
See 'docker --help'
[root@ip-172-31-30-131 jenkin-docker]# docker images
REPOSITORY                                                      TAG       IMAGE
ID       CREATED              SIZE
jenkins-docker-demo                                             v1        04f7a3
1a23ca   18 seconds ago       162MB
docker-scan-demo                                                v2        071c99
3a79a7   About an hour ago    162MB
571833381790.dkr.ecr.us-east-1.amazonaws.com/docker-scan-demo   v1        34b3ff
b37895   3 hours ago          162MB
docker-scan-demo                                                v1        34b3ff
b37895   3 hours ago          162MB
dockerhub-scan-demo                                             v1        34b3ff
b37895   3 hours ago          162MB
rajkumari2010/docker-image-scan-demo                            v1        34b3ff
b37895   3 hours ago          162MB
compose-demo-web                                                latest    646553
7f4622   13 hours ago         162MB
multi-stage-multi                                               v1        f1941c
3ca17c   14 hours ago         282MB
multi-stage-single                                              v1        b106e3
f4c768   14 hours ago         469MB
alpine-tomcat                                                   1.0       96e1b5
0cb876   15 hours ago         227MB
my-nginx-running                                                v2        384c40
c04463   17 hours ago         162MB
my-nginx-image                                                  v1        39c541
b70324   20 hours ago         162MB
redis                                                           alpine    00c30d
df0ef8   8 days ago           119MB
[root@ip-172-31-30-131 jenkin-docker]#
```

Create a container and run on port no 8081

```
root@ip-172-31-30-131:~/jenkin-docker
[root@ip-172-31-30-131 jenkin-docker]# docker run -d --name jenkins-docker-cont
-p 8081:80 jenkins-docker-demo:v1
32f6c9c2c568abd62ac68b4370d08429724bb010988d000440d9f1622aabaccc
[root@ip-172-31-30-131 jenkin-docker]# docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED
 STATUS          PORTS                                   NAMES
32f6c9c2c568   jenkins-docker-demo:v1   "/docker-entrypoint.…"   7 seconds ago
 Up 5 seconds   0.0.0.0:8081->80/tcp, :::8081->80/tcp   jenkins-docker-cont
[root@ip-172-31-30-131 jenkin-docker]#
```



Test:

curl http://localhost:8080

If everything works:

docker stop jenkins-docker-demo

docker rm jenkins-docker-demo

9. Push Project to GitHub

Create a GitHub repository, for example:

jenkins-docker-demo

Then:

git init

Add files:

git add .

Commit:

git commit -m "Initial Docker Jenkins pipeline"

Add remote:

git remote add origin https://github.com/<username>/jenkins-docker-demo.git

Push:

git branch -M main

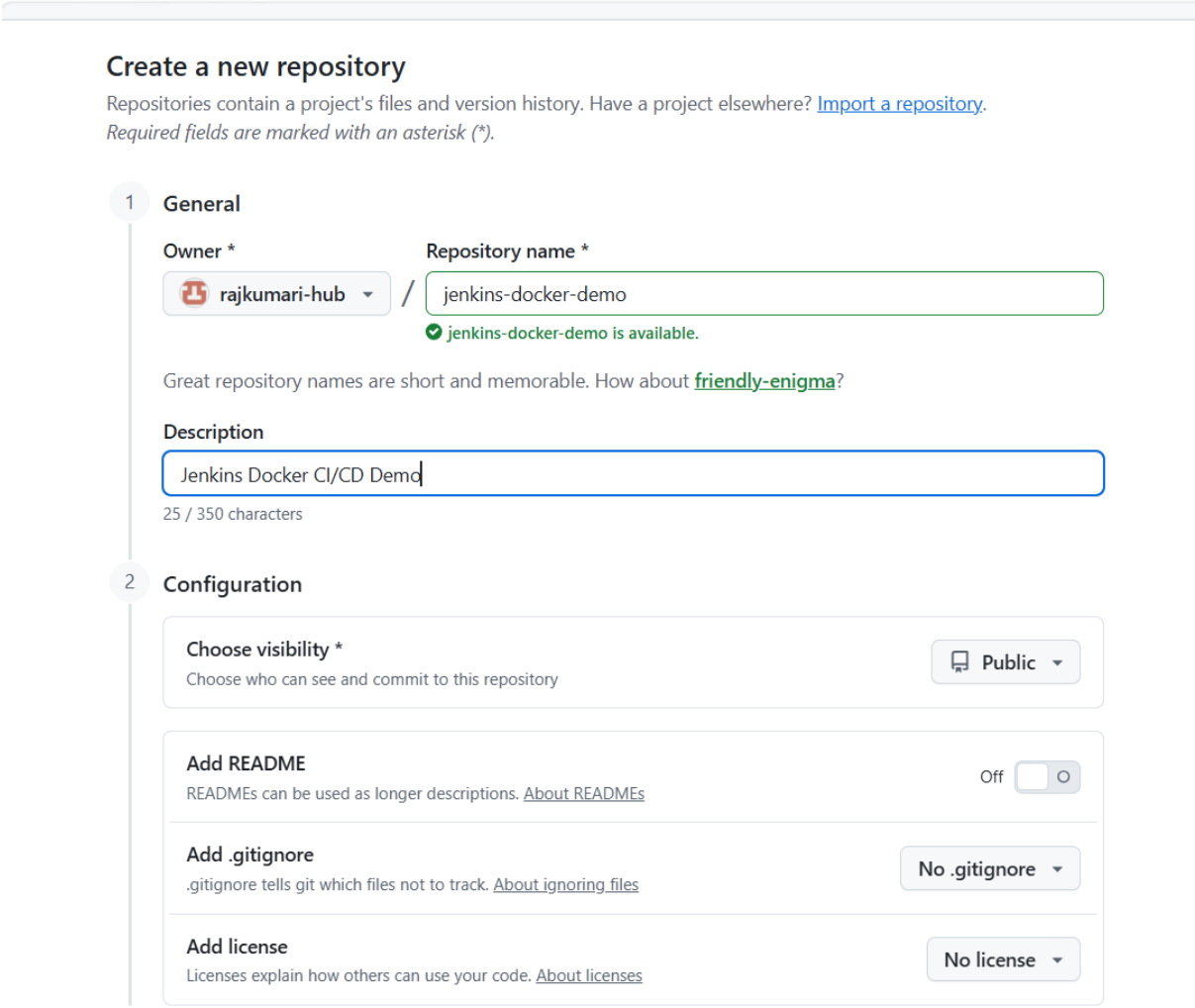
git push -u origin main

Your GitHub repository should contain:

Dockerfile

index.html

Jenkinsfile



9. Push Project to GitHub

Create a GitHub repository, for example:

jenkins-docker-demo

Then:

git init

Add files:

git add .

Commit:

git commit -m "Initial Docker Jenkins pipeline"

Add remote:

git remote add origin https://github.com/<username>/jenkins-docker-demo.git

Push:

git branch -M main

git push -u origin main

Your GitHub repository should contain:

Dockerfile

index.html

Jenkinsfile

10. Create Jenkinsfile

This is the most important part.

Create:

vi Jenkinsfile

root@ip-172-31-30-131:~/jenkin-docker

```
pipeline {

    agent any

    environment {
        DOCKER_IMAGE = 'rajkumari2010/jenkins-docker-demo'
        DOCKER_TAG   = "${BUILD_NUMBER}"
    }

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/rajkumari-hub/jenkins-docker-demo.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh '''
                    docker build \
                    -t ${DOCKER_IMAGE}:${DOCKER_TAG} .
                '''
            }
        }

        stage('Docker Login') {
            steps {
                withCredentials([
                    usernamePassword(
                        credentialsId: 'dockerhub-credentials',
                        usernameVariable: 'DOCKER_USERNAME',
                        passwordVariable: 'DOCKER_PASSWORD'
                    )
                ]) {
                    sh '''
                        echo "$DOCKER_PASSWORD" | \
                        docker login \
                        -u "$DOCKER_USERNAME" \
                        --password-stdin
                    '''
                }
            }
        }

        stage('Push Docker Image') {
            steps {
                sh '''
                    docker push ${DOCKER_IMAGE}:${DOCKER_TAG}
                '''
            }
        }
    }

    post {
        success {
-- INSERT --
```

11. Understand the Jenkinsfile

Pipeline

pipeline {

This indicates that we are using a **Declarative Jenkins Pipeline**.

Agent

agent any

Jenkins can execute the pipeline on any available agent.

If you specifically want to use your Docker agent, you could use a label:

agent {

label 'devops-node'

}

For your setup, if the Docker installation is on your devops-node, this is often better.

---

12. Environment Variables

```
environment {
   DOCKER_IMAGE = 'rajkumari/jenkins-docker-demo'
   DOCKER_TAG   = "${BUILD_NUMBER}"
}
```

This defines:

DOCKER_IMAGE

DOCKER_TAG

BUILD_NUMBER is a Jenkins built-in variable.

Suppose Jenkins executes build:

Build #15

Then:

DOCKER_TAG = 15

The resulting image is:

rajkumari/jenkins-docker-demo:15

This is better than always using latest, because each Jenkins build gets its own image version.

---

13. Git Clone Stage

```
stage('Git Clone') {
   steps {
      git branch: 'main',
         url: 'https://github.com/<github-username>/jenkins-docker-demo.git'
```

```
    }
}
```

Jenkins downloads the source code from GitHub.

The workspace will contain:

Dockerfile

index.html

Jenkinsfile

---

14. Build Docker Image Stage

```
stage('Build Docker Image') {
    steps {
        sh '''
            docker build \
            -t ${DOCKER_IMAGE}:${DOCKER_TAG} .
        '''
    }
}
```

Jenkins executes:

```
docker build -t rajkumari/jenkins-docker-demo:15 .
```

The image is created locally on the Jenkins agent.

---

15. Docker Login Stage

```
withCredentials([
    usernamePassword(
        credentialsId: 'dockerhub-credentials',
        usernameVariable: 'DOCKER_USERNAME',
```

```
        passwordVariable: 'DOCKER_PASSWORD'
    )
])
```

Jenkins retrieves the Docker Hub credentials securely.

Then:

```
echo "$DOCKER_PASSWORD" | docker login \
-u "$DOCKER_USERNAME" \
--password-stdin
```

This is preferable to putting the username/password directly into the Jenkinsfile.

---

16. Push Docker Image Stage

```
stage('Push Docker Image') {
    steps {
        sh '''
            docker push ${DOCKER_IMAGE}:${DOCKER_TAG}
        '''
    }
}
```

For build number 15:

```
docker push rajkumari/jenkins-docker-demo:15
```

The image goes to:

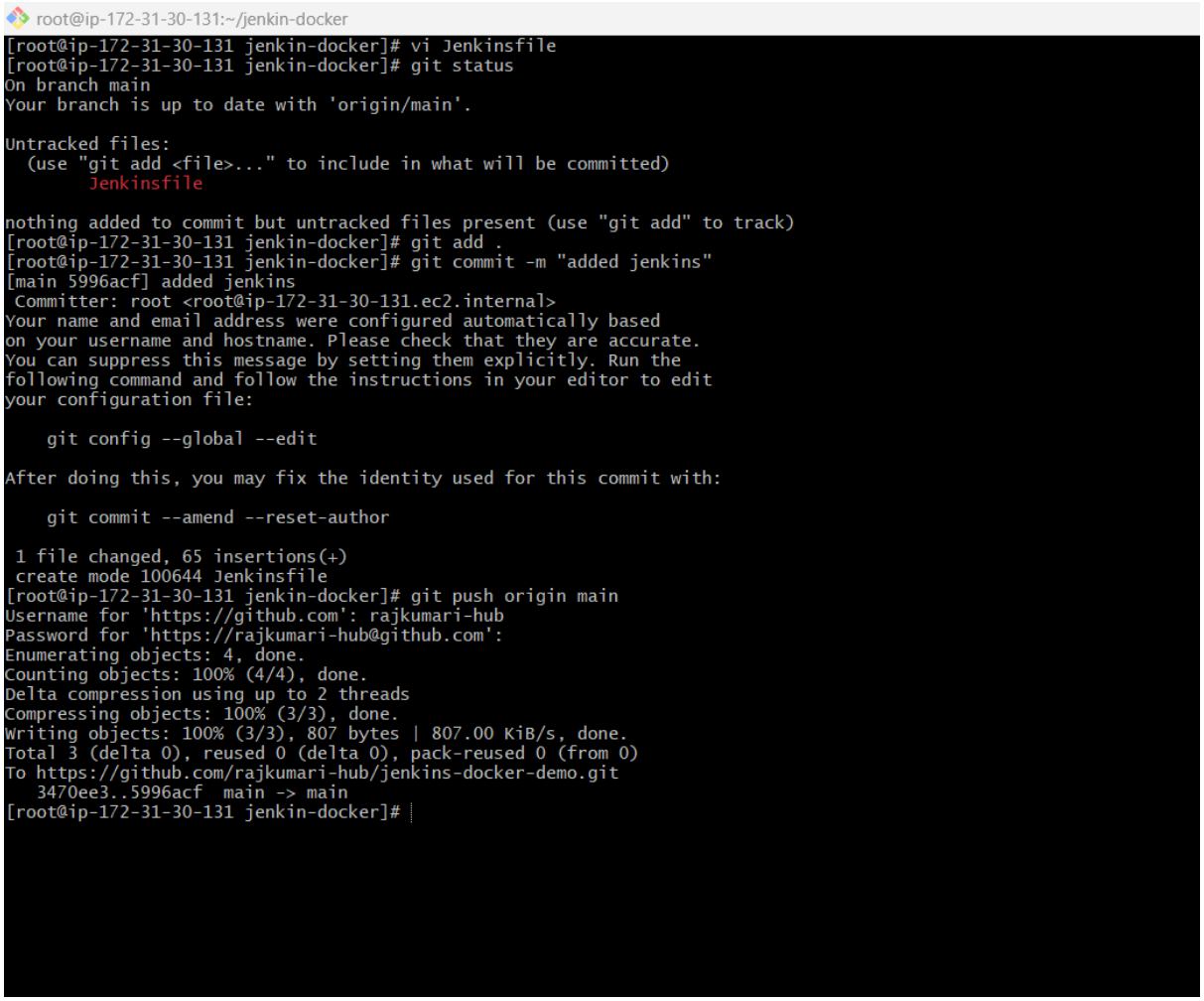
Docker Hub

↓

rajkumari/jenkins-docker-demo

↓

15



17. Create Jenkins Pipeline Job

Open Jenkins:

Jenkins Dashboard

↓

New Item

Enter:

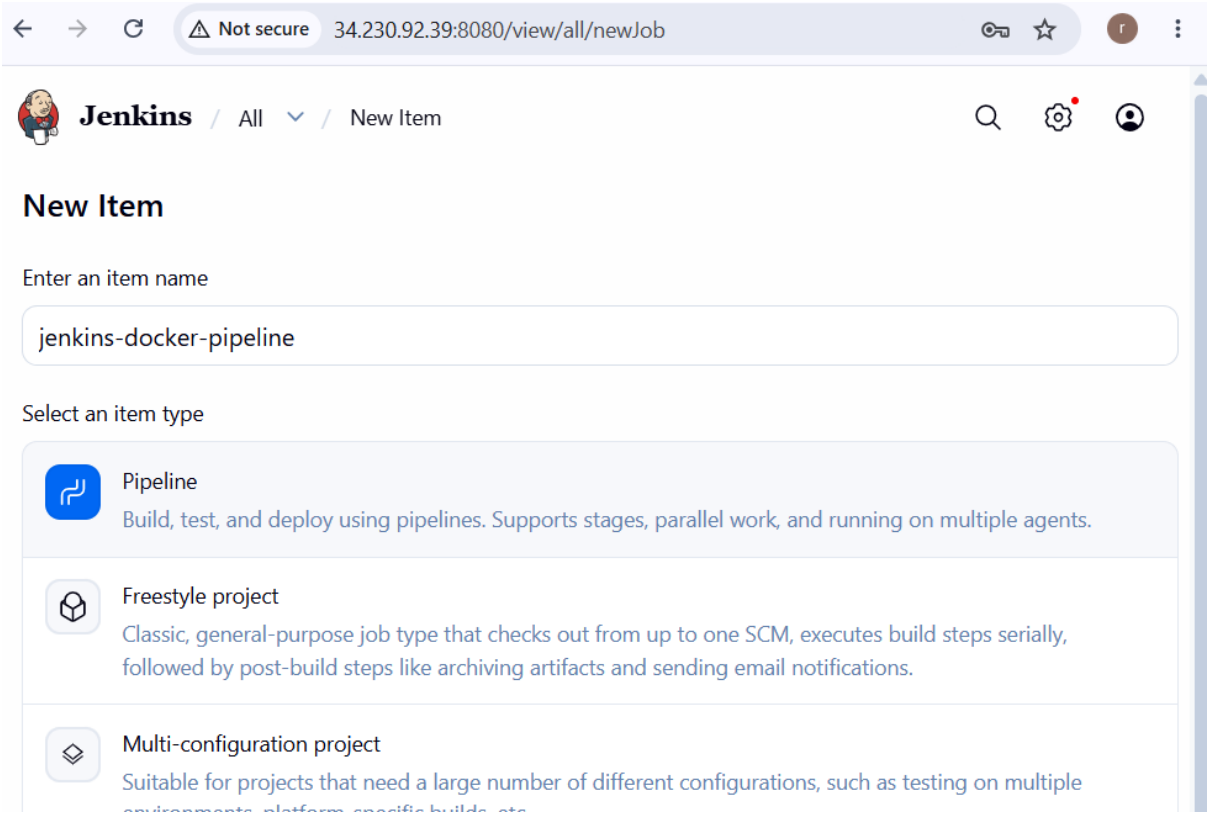
jenkins-docker-pipeline

Select:

Pipeline

Click:

OK



18. Configure Pipeline

Scroll to:

Pipeline

Select:

Definition:

Pipeline script from SCM

Select:

SCM:

Git

Repository URL:

https://github.com/<github-username>/jenkins-docker-demo.git

Branch:

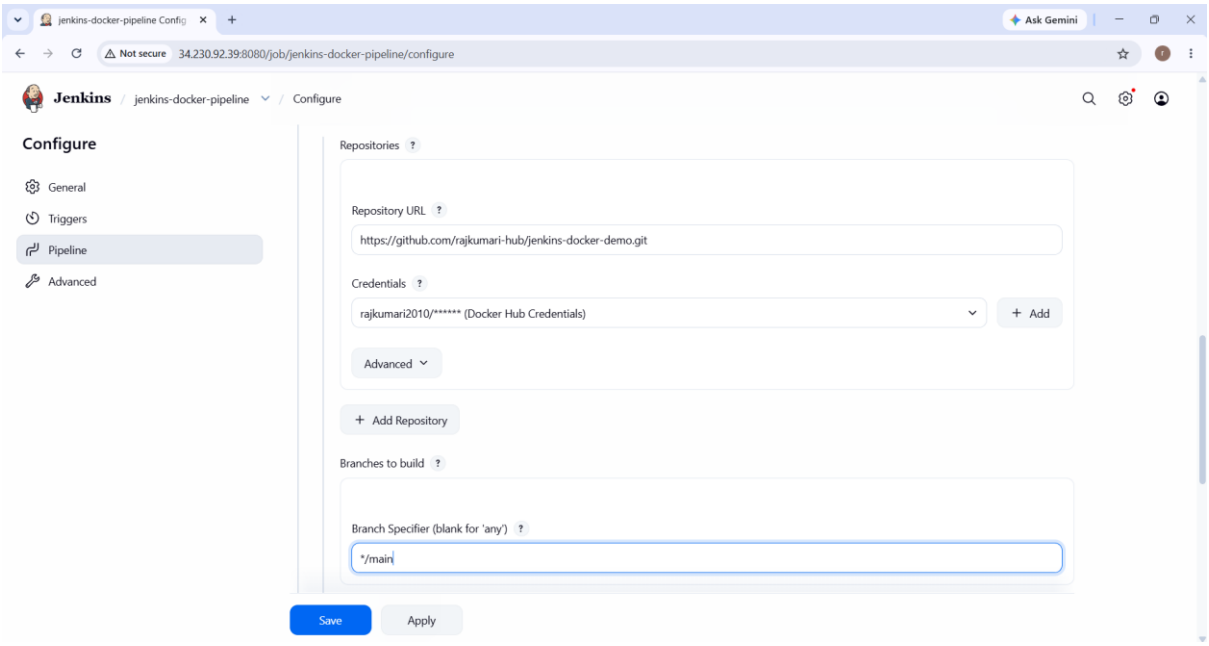
*/main

Script Path:

Jenkinsfile

Click:

Save



19. Run the Pipeline

Click:

Build Now

Jenkins will execute:

Git Clone

↓

Build Docker Image

↓

Docker Login

↓

Push Docker Image

You should see:

[Pipeline] stage

[Pipeline] { (Git Clone)

[Pipeline] stage

[Pipeline] { (Build Docker Image)

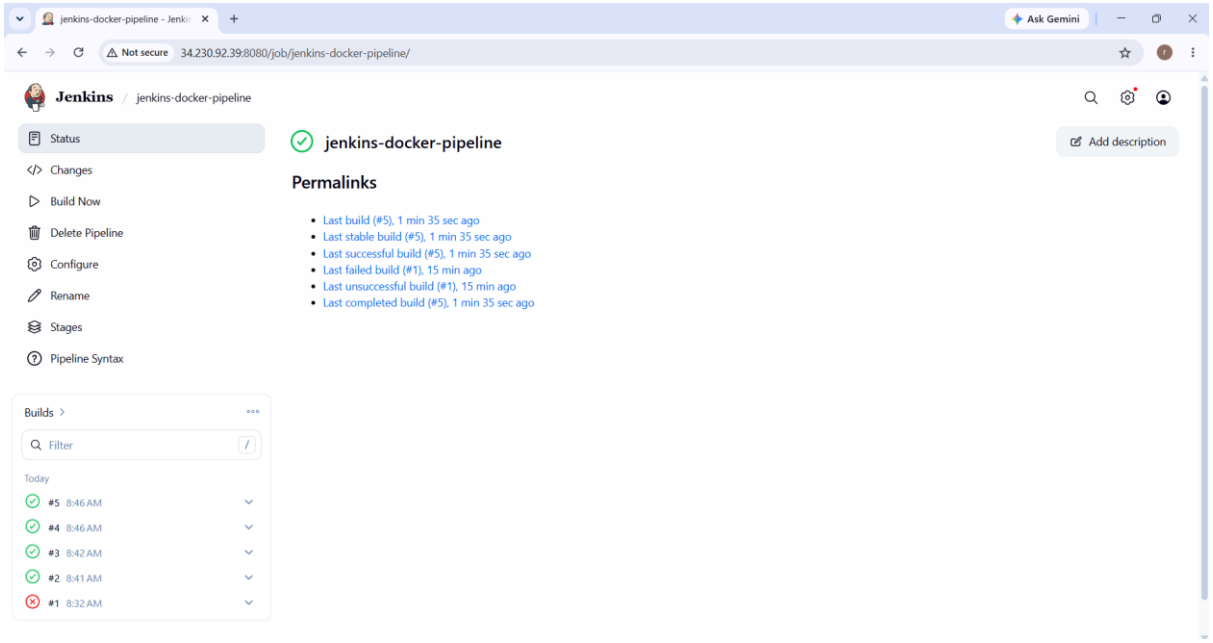
[Pipeline] stage

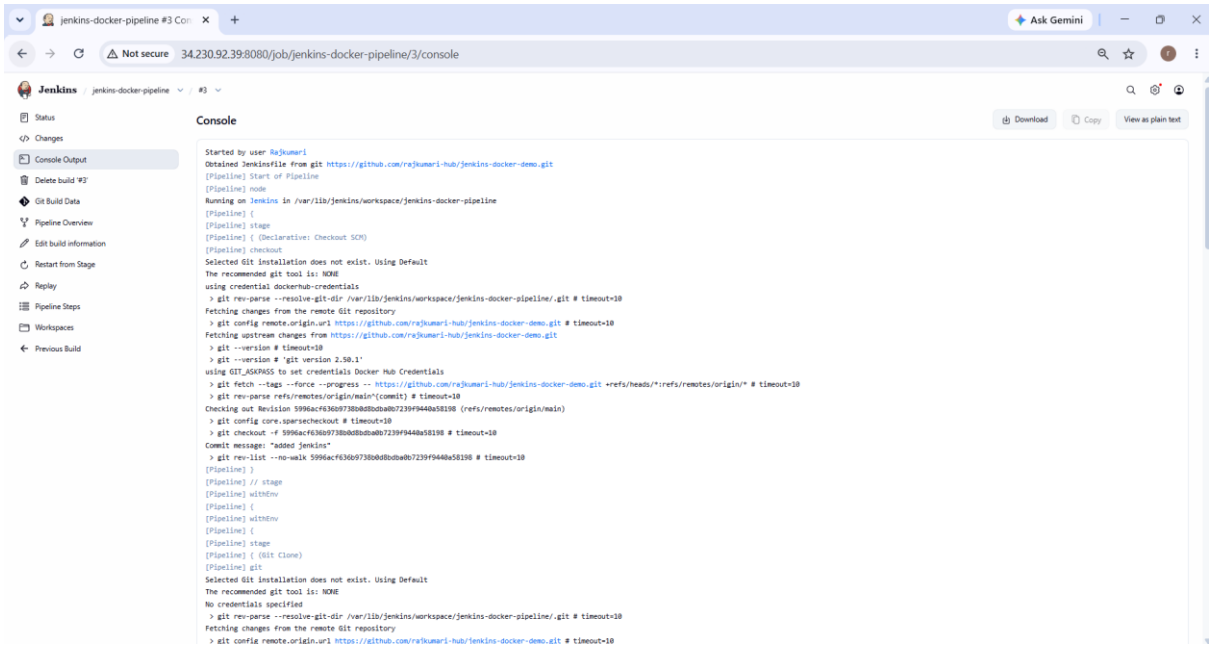
[Pipeline] { (Docker Login)

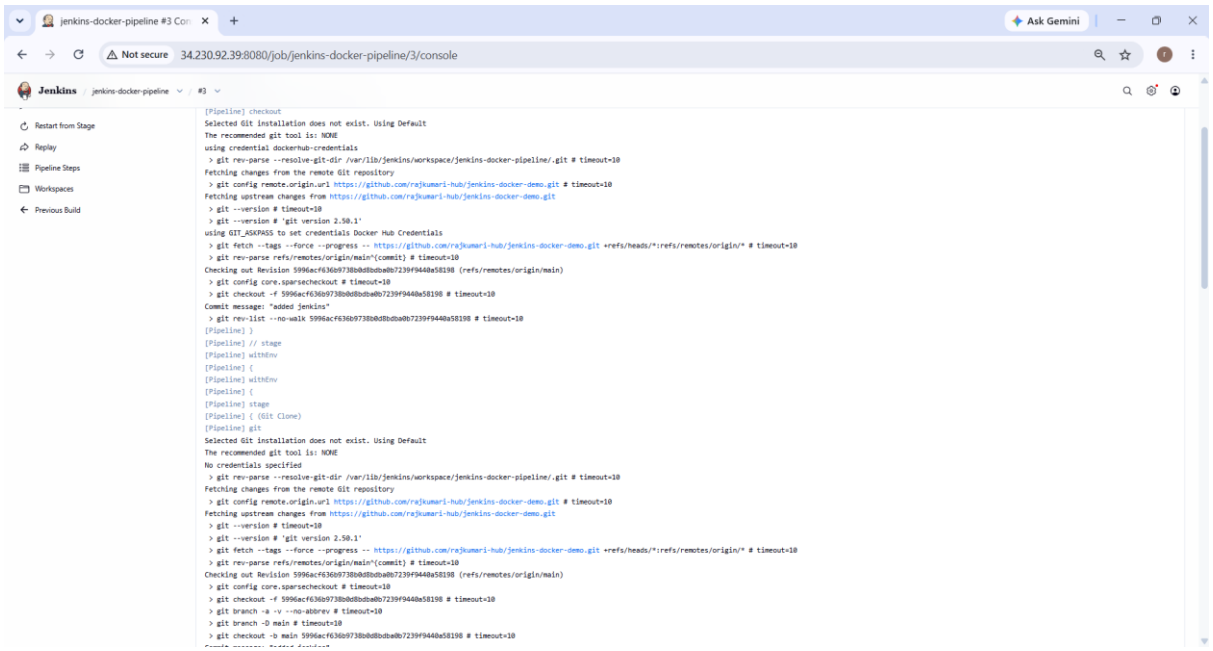
[Pipeline] stage

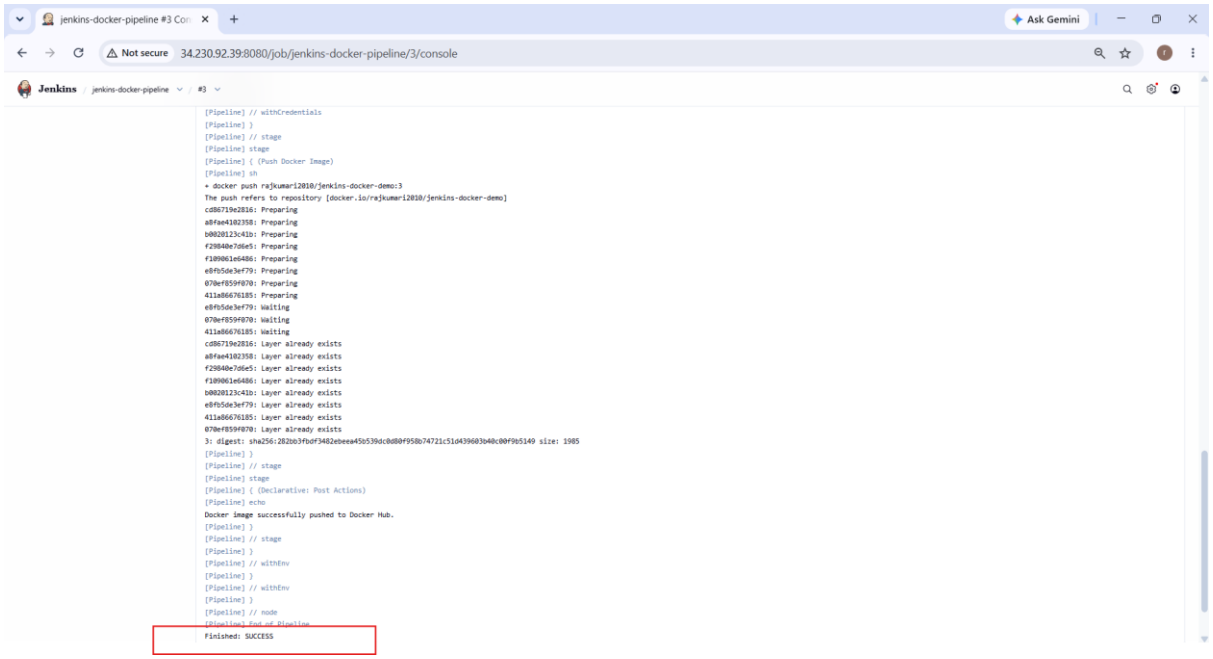
[Pipeline] { (Push Docker Image)

Finished: SUCCESS




jenkins-docker-pipeline #3 Con
Ask Gemini
Not secure
34.230.92.39:8080/job/jenkins-docker-pipeline/3/console
Jenkins
jenkins-docker-pipeline
#3
Status
Changes
Console Output
Delete build '#3'
Git Build Data
Pipeline Overview
Edit build information
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
Console
Download
Copy
View as plain text
Started by user Rajkumari
Obtained Jenkinsfile from git https://github.com/rajkumari-hub/jenkins-docker-demo.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/jenkins-docker-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential dockerhub-credentials
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumari-hub/jenkins-docker-demo.git # timeout=10
Fetching upstream changes from https://github.com/rajkumari-hub/jenkins-docker-demo.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials Docker Hub Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumari-hub/jenkins-docker-demo.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5996acf636b9738b0d8bdba0b7239f9440a58198 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5996acf636b9738b0d8bdba0b7239f9440a58198 # timeout=10
Commit message: "added jenkins"
> git rev-list --no-walk 5996acf636b9738b0d8bdba0b7239f9440a58198 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumari-hub/jenkins-docker-demo.git # timeout=10

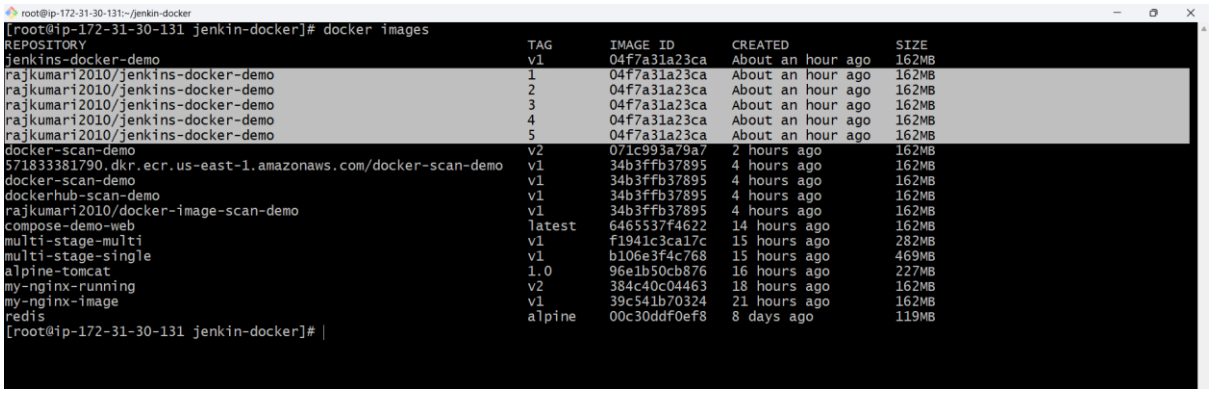

jenkins-docker-pipeline #3 Con
Ask Gemini
Not secure
34.230.92.39:8080/job/jenkins-docker-pipeline/3/console
Jenkins
jenkins-docker-pipeline
#3
Restart from Stage
Replay
Pipeline Steps
Workspaces
Previous Build
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential dockerhub-credentials
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumari-hub/jenkins-docker-demo.git # timeout=10
Fetching upstream changes from https://github.com/rajkumari-hub/jenkins-docker-demo.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials Docker Hub Credentials
> git fetch --tags --force --progress -- https://github.com/rajkumari-hub/jenkins-docker-demo.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5996acf636b9738b0d8bdba0b7239f9440a58198 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5996acf636b9738b0d8bdba0b7239f9440a58198 # timeout=10
Commit message: "added jenkins"
> git rev-list --no-walk 5996acf636b9738b0d8bdba0b7239f9440a58198 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Git Clone)
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins-docker-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/rajkumari-hub/jenkins-docker-demo.git # timeout=10
Fetching upstream changes from https://github.com/rajkumari-hub/jenkins-docker-demo.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/rajkumari-hub/jenkins-docker-demo.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 5996acf636b9738b0d8bdba0b7239f9440a58198 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 5996acf636b9738b0d8bdba0b7239f9440a58198 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D main # timeout=10
> git checkout -b main 5996acf636b9738b0d8bdba0b7239f9440a58198 # timeout=10



20. Verify the Image on Jenkins Server

SSH into the Docker/Jenkins agent and run:

docker images



21. Verify the Image in Docker Hub

Open:

Docker Hub

Go to:

Repositories

↓

jenkins-docker-demo

↓

Tags

You should see:

1

2

3

For example:

jenkins-docker-demo:15